

FernUniversität in Hagen
Fakultät für Mathematik und Informatik
LG Kooperative Systeme

UDP Transport Options

Implementierung und Analyse von RFC 9868 in Rust

Masterarbeit
im Studiengang Master Informatik

vorgelegt von
Alan Bernstein

Matrikelnummer: 2068834

Erstprüfer: Prof. Dr. Christian Icking

Zweitprüferin: Dr. Lihong Ma

Hagen, 15.09.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iii
Tabellenverzeichnis	iv
Abkürzungsverzeichnis	v
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.2.1 Unidirektionalität	2
1.2.2 Herausforderungen	2
1.3 Zielsetzung	3
1.4 Abgrenzung und Umfang	4
1.5 Aufbau der Arbeit	4
1.6 KI-Einsatz	5
2 Grundlagen	6
2.1 IPv4	6
2.2 Das User Datagram Protocol (UDP)	9
2.2.1 Protokollstruktur	9
2.2.2 Eigenschaften	12
2.3 Die Surplus Area	12
2.3.1 Lage und Kompatibilität	12
2.3.2 Aufbau und Ausrichtung	14
2.3.3 OCS und Fehlerbehandlung	15
2.3.4 TLV-Format und Empfangsentscheidung	17
2.3.5 Entwurfsprinzipien und Optionsklassen	19
2.4 Betriebssystem und Netzpfad	20
3 Methodik und KI-gestützte Entwicklung	25
3.1 Anlass des KI-Einsatzes und veröffentlichte Fälle	25
3.2 Forschungsdesign und KI-gestützte Arbeitsweise	26
3.3 Bekannte Schwächen der Sprachmodelle	30
3.4 Testarchitektur für FF1	32
3.4.1 Prüfmittel und ihre Reichweite	32
3.4.2 Prüfkette bis zur Freigabe	33

3.5	Reproduzierbarkeit und Grenzen	34
4	Analyse und Anforderungsmodell	35
4.1	Anwendung der Extraktionsmethode	35
4.2	Anforderungskatalog und FF1-Maßstab	36
4.3	FF2-Pfad- und Auswertungsmodell	37
4.4	Technologieauswahl	39
4.4.1	Programmiersprache	39
4.4.2	Zugang zur Surplus Area	41
4.4.3	Mitschnitt und Auswertung	41
4.4.4	Fuzzing und formale Gegenprobe	42
5	Entwurf und Implementierung	43
5.1	Architektur	43
5.2	Sendepfad und gemeinsamer Optionskern	44
5.3	Empfangspfad und öffentliche Schnittstelle	46
5.4	Fragmentierung und weitere Pflichtoptionen	49
5.5	Betriebs- und Sicherheitsgrenzen	53
6	Evaluation und Diskussion	55
6.1	Test- und Messkonzept	55
6.2	Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden . .	59
6.2.1	Kontrollierte Stufen: Loopback, eigene Topologien, Tunnel . .	59
6.2.2	Öffentliche Pfade in Europa	60
6.2.3	Interkontinentale Pfade	66
6.2.4	Mechanismenklassen und Folgen	68
6.3	Soll-Ist-Abgleich mit RFC 9868	70
6.4	Reflexion der KI-gestützten Arbeitsweise	76
6.5	Diskussion und Grenzen der Aussagekraft	78
7	Fazit und Ausblick	80
7.1	Beantwortung der Forschungsfragen	80
7.2	Ausblick	81
7.3	Schlusswort	81
	Literaturverzeichnis	82
	Erklärung	86

Abbildungsverzeichnis

2.1	Der IPv4-Header nach RFC 791	6
2.2	Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente	8
2.3	Der UDP-Header nach RFC 768	9
2.4	Der Pseudo-Header der UDP-Prüfsumme für IPv4	11
2.5	IP Transport Payload und Surplus Area	12
2.6	Aufbau eines Datagramms mit UDP Options	14
2.7	Pad-Byte bei geradem und ungeradem Startoffset	15
2.8	Empfangsentscheidung für die Surplus Area im Standardfall	18
2.9	Die zwei Sendepfade durch den Linux-Kernel	21
2.10	Der Empfangspfad durch den Linux-Kernel	22
2.11	Messmodell des Datagrammwegs	23
3.1	Aufbau der Untersuchung und Zuordnung der Forschungsfragen	26
3.2	Von den Normquellen zum geprüften Anforderungsbestand	28
3.3	Kernzyklus eines Implementierungsschritts	29
3.4	Prüfprozess im Sollzustand	34
4.1	Verteilung der markierten normativen Absätze über die Abschnitte	35
5.1	Module der Bibliothek, Messprogramme und ihre Abhängigkeiten	43
5.2	Entstehung des Beispieldatagramms mit MDS in drei Zuständen	45
5.3	Schutzbereiche der APC und Folgen von Empfangsfehlern	47
5.4	Zerlegung und Reassemblierung eines Datagramms	51
6.1	Ankunfts-TTL je Richtung	58
6.2	Kontrollierte Messumgebungen der Pfadstufen 1 bis 3	60
6.3	Europäisches Messdreieck	61
6.4	Reroute-Fenster im Szenario S-53	62
6.5	Prüfsummen-Gate-Kreuzzellen je Messrichtung	63
6.6	Zugangspfad über den Mobilfunk-Hotspot	64
6.7	Interkontinentale Messendpunkte mit Belegstufe und Rundlaufzeiten	66
6.8	Verbleib der Surplus-Klasse nach Asien-Pazifik	68
6.9	Kantenmodell eines Messpfads	69
6.10	Drei beobachtete Ergebnisse für Beispieldatagramme auf realen Pfaden	69

Tabellenverzeichnis

2.1	OCS-Rechnung einer 6-Byte-Surplus-Area bei geradem Startoffset . .	16
3.1	Operationalisierung der Forschungsfragen	27
3.2	Rollen bei Erzeugung, Prüfung und Freigabe	30
3.3	Schwächen der Sprachmodelle und methodische Antworten	31
4.1	Auszug aus dem RFC-Arbeitsindex der Implementierung	36
4.2	Implementierungsumfang	38
4.3	Messinfrastruktur	39
4.4	Vergleich der Sprachkandidaten	40
6.1	Messläufe der Arbeit nach Pfad und Einstufung	56
6.2	Summen der geprüften Einzelzuordnung	71
6.3	Fragmentierung und Soft-State: Norm, Umsetzung und Messbeleg . .	76
7.1	Synthese der Forschungsfragen im geprüften Umfang	81

Abkürzungsverzeichnis

AUTH Authentication Option (reserviert).

AWS Amazon Web Services.

CRC32c Cyclic Redundancy Check (32 Bit, Castagnoli-Polynom).

ECMP Equal-Cost Multi-Path.

EID Errata ID (Kennung eines RFC-Erratums).

GCP Google Cloud Platform.

ICMP Internet Control Message Protocol.

KVM Kernel-based Virtual Machine.

NTP Network Time Protocol.

PCAP Packet Capture.

STUN Session Traversal Utilities for NAT.

TIME Timestamp Option.

TTL Time to Live.

UCMP UNSAFE Compression (reserviert).

UENC UNSAFE Encryption (reserviert).

UEXP UNSAFE Experimental.

XDP eXpress Data Path.

1. Einleitung

1.1 Motivation

Die ursprüngliche Spezifikation des User Datagram Protocol (UDP), RFC 768 aus dem Jahr 1980, umfasst 663 Wörter [1]. Die Erweiterung RFC 9868 wurde im Oktober 2025 veröffentlicht und führt unter dem Titel *Transport Options for UDP* erstmals einen allgemeinen Optionsmechanismus für UDP ein [2]. Zwischen beiden Dokumenten liegen 45 Jahre; die Klartextfassung des RFC 9868 ist mehr als 27-mal so umfangreich wie die des RFC 768.

Diese Arbeit verwendet durchgängig die folgenden Begriffe. *Header* bezeichnet den festen Kopf einer Protokolleinheit, etwa eines IP- oder UDP-Datagramms; die englische Form ist in der deutschen Fachsprache üblich und entspricht den Feldnamen der RFCs. *Nutzlast* ist der Datenteil eines IP-Datagramms oder Rahmens hinter dessen Header; *Nutzdaten* sind die Bytes hinter dem UDP-Header bis zu dem Ende, das `UDP Length` angibt, im RFC *user data*. Ein *Altempfänger*, im RFC *legacy receiver*, ist ein Endpunkt, der UDP nach RFC 768 verarbeitet und RFC 9868 nicht kennt; in der Regel liefert er nur die Nutzdaten an die Anwendung. *Altsysteme* umfassen solche Endpunkte und die Zwischensysteme auf dem Pfad, die RFC 9868 nicht kennen. Den Gegenbegriff zum Altempfänger bildet der *optionsfähige Empfänger*, im RFC *option-aware receiver*.

UDP besitzt einen acht Byte langen Header aus vier 16-Bit-Feldern: Quellport, Zielport, Länge und Prüfsumme [1]. RFC 9868 beschreibt UDP als weit verbreitet und stellt fest, dass der ursprüngliche Header keinen Raum für Optionen bietet [2, Sec. 4].

RFC 9868 greift Funktionen auf, die UDP ursprünglich der Anwendung oder einer darüberliegenden Schicht überließ: Transportfragmentierung, eine zusätzliche Nutzdatenprüfsumme sowie Optionen zur Mitteilung maximaler Datagrammgrößen und zum Austausch von Zeitstempeln [2, Sec. 11.3 bis 11.6 und 11.8]. Für ihre Bereitstellung kommen grundsätzlich drei Wege in Betracht:

1. Erweiterung innerhalb der Nutzdaten: Jede Anwendung definiert dort ein eigenes Rahmenformat.
2. Ein neues Transportprotokoll: Alle gewünschten Mechanismen können so von Beginn an vorgesehen werden.

3. Erweiterung des bestehenden Formats: UDP selbst erhält einen Optionsmechanismus, der für Altempfänger möglichst unsichtbar bleibt.

RFC 9868 wählt den dritten Weg und nutzt dafür die sogenannte *Surplus Area*: den möglichen Bereich zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms. Das UDP-Length-Feld gibt die Länge des UDP-Datagramms an. Diese kann kleiner sein als die Größe des im IP-Datagramm für UDP verfügbaren Bereichs. Der verbleibende Raum hinter den UDP-Nutzdaten kann deshalb für Erweiterungen genutzt werden (Abschnitt 2.3). Auf dieser Grundlage definiert RFC 9868 ein erweiterbares Rahmenwerk für Transportoptionen.

Damit stellt sich die Frage, die diese Arbeit als Leitfrage begleitet:

Wie lässt sich UDP um Transportoptionen erweitern, ohne seine grundlegenden Eigenschaften aufzugeben?

Die Arbeit untersucht diese Frage anhand einer Referenzimplementierung und auf ausgewählten realen Netzpfeilen.

1.2 Problemstellung

Die Implementierung von UDP-Optionen gemäß RFC 9868 wirft drei Problemfelder auf, die diese Arbeit behandelt: die Beschaffenheit der Surplus Area selbst (Abschnitt 2.3), die bewusste Unidirektionalität des Protokolls sowie die praktischen Implementierungsherausforderungen vom Zugriff aus dem Benutzerraum bis zum Verhalten von Zwischensystemen. Die folgenden Abschnitte behandeln die beiden letzten.

1.2.1 Unidirektionalität

RFC 9868 stellt explizit klar: „*UDP is unidirectional; UDP Options do not change that fact.*“ [2, Sec. 6].

Für Mechanismen, die Bidirektionalität voraussetzen, fordert RFC 9868 eine getrennte Spezifikation [2, Sec. 6]; RFC 9869 definiert einen solchen Einsatz der Optionen REQ (Echo Request) und RES (Echo Response) für die Ermittlung der Pfad-MTU (Maximum Transmission Unit) [3, Sec. 3]. Die Optionsschicht erzeugt also keine Antwort selbstständig; das Prinzip vertieft Abschnitt 2.3.5.

1.2.2 Herausforderungen

Die zentrale Herausforderung bei der Implementierung von UDP-Optionen liegt im Zugriff auf die Surplus Area. RFC 9868 berichtet von Interoperabilitätstests

mit Linux, macOS und Windows Cygwin. Diese Systeme lieferten nur die durch das UDP-Length-Feld begrenzten Nutzdaten an die Anwendung und verwarfen die Surplus Area still. Außerdem berichtet der RFC von einem eingebetteten Gerät, das das gesamte IP-Datagramm an die UDP-Anwendungsschicht weitergab [2, Sec. 18]. Den Weg eines UDP-Pakets durch den Linux-Kernel bis zu dieser Socket-Schnittstelle dokumentieren Stephan und Wüstrich [4].

Eine weitere Herausforderung stellt die Kompatibilität mit Zwischensystemen dar. RFC 9868 berichtet Interoperabilitätstests, in denen Pakete mit Surplus Area Geräte zur Netzadressübersetzung (NAT, *Network Address Translation*) passierten, nennt aber auch ein Intrusion Detection System, das die Diskrepanz zwischen UDP-Length und IP-Length in der Standardeinstellung als Angriff meldet [2, Sec. 18]. Ob reale Netzpfade solche Pakete tatsächlich durchlassen, ist eine empirische Frage dieser Arbeit (FF2).

1.3 Zielsetzung

Die Problemstellung verdichtet sich zu zwei getrennten Hürden: Ein UDP-Options-Paket muss normgerecht erzeugt und empfangen werden, und seine Surplus Area muss die untersuchten Netzpfade unverändert überstehen.

Eine normgerechte Implementierung sagt nichts darüber aus, ob Zwischensysteme Datagramme mit Surplus Area unverändert passieren lassen. Ebenso nützt ein durchlässiger Netzpfad wenig, wenn Erzeugung und Auswertung der Optionen fehlerhaft sind. Aus dieser Trennung ergeben sich zwei Forschungsfragen:

- **FF1:** Welche Anforderungen des RFC 9868 lassen sich unter Linux und IPv4 mit einer Raw-Socket-Implementierung im Benutzerraum vollständig, teilweise oder nicht erfüllen?
- **FF2:** In welchem Umfang bleibt die *Surplus Area* auf den untersuchten realen Netzpfeiden bis zur Gegenstelle unverändert erhalten?

Das Ziel dieser Arbeit ist es, beide Fragen zu beantworten. Dazu entsteht zunächst eine an RFC 9868 ausgerichtete und systematisch auf Konformität geprüfte Bibliothek für UDP-Transportoptionen in Rust. Die auf Linux und IPv4 begrenzte Implementierung arbeitet im Benutzerraum mit Raw Sockets. Sie umfasst Parser und Serialisierer für UDP-Optionen nach dem TLV-Schema (*Type-Length-Value*), die Validierung der Optionsprüfsumme (*Option Checksum*, OCS) gemäß Abschnitt 9 des RFC 9868 sowie die verpflichtend zu unterstützenden Optionen EOL, NOP, APC, FRAG, MDS, MRDS, REQ und RES (Langformen in Abschnitt 5.4) [2, Sec. 10].

Die Verifikation erfolgt mehrstufig: Komponententests prüfen die einzelnen Bestandteile, Integrationstests den Loopback-Pfad, und für reine Regeln wird ergänzend eine Lean-Spezifikation gepflegt. Lean dient als formale Gegenprobe für Prüfsummenarithmetik sowie Längen- und Anordnungsregeln; das Verhalten von Raw Sockets, Betriebssystemen und Zwischensystemen bleibt Gegenstand empirischer Prüfungen. Dieses Prüfprogramm schafft die Grundlage für die Beantwortung von FF1.

Für FF2 vergleicht die Evaluation Datagramme ohne Surplus Area mit gleich großen Datagrammen, die eine Surplus Area enthalten, auf ausgewählten realen Netzpfeilen. Sie unterscheidet zwischen unveränderter oder veränderter Zustellung, Entfernung der Surplus Area und Paketverlust. Die Schlussfolgerungen bleiben auf die beobachteten Pfade begrenzt.

1.4 Abgrenzung und Umfang

Diese Arbeit setzt einen bewussten Rahmen. Sie beschränkt sich auf eine Umsetzung des RFC 9868 im Linux-Benutzerraum für IPv4. Gegenstand sind das TLV-Rahmenwerk, die OCS sowie die verpflichtend zu unterstützenden Optionen. Die Beschränkung auf den Benutzerraum ist eine vorab gesetzte Rahmenbedingung der Aufgabenstellung. Abschnitt 4.4 begründet die dafür gewählte Umsetzung mit Raw Sockets gegenüber kernelnahen Verfahren.

Außerhalb des Umfangs liegt der REQ/RES-Anwendungsfall zur Pfad-MTU-Ermittlung, den RFC 9869 definiert [3]. Die Implementierung unterstützt nur die in RFC 9868 festgelegten Formate von REQ und RES. Ebenfalls nicht implementiert wird die TIME-Option. RFC 9868 beschreibt zwar ihr Format und den Anwendungszweck, die Schätzung der Paketumlaufzeit (*Round-Trip Time*, RTT), legt aber kein nutzendes Protokoll fest. Die Kennungen für AUTH, UCMP und UENC sind in RFC 9868 lediglich reserviert und werden daher ebenfalls nicht implementiert [2, Sec. 11.9, 12.1 und 12.2]. Kernel-Module werden nicht entwickelt. Ein Schutz gegen verdeckte Kanäle über UDP-Optionen ist kein Ziel dieser Arbeit [2, Sec. 25.2].

IPv6 wird weder implementiert noch auf Konformität geprüft. Die hier untersuchten Mechanismen des RFC 9868 (*Surplus Area*, OCS, TLV-Optionen und FRAG) lassen sich vollständig über IPv4 zeigen. Die Implementierung, ihre Konformitätsprüfung und die Pfadmessungen beziehen sich daher auf IPv4.

1.5 Aufbau der Arbeit

Nach dieser Einleitung folgen sechs Kapitel, die drei Blöcke bilden:

- **Grundlagen und Methodik:** Kapitel 2 führt die technischen Grundlagen ein: das Internet Protocol, UDP sowie RFC 9868 mit der *Surplus Area* und seinen Entwurfsprinzipien. Kapitel 3 legt die Forschungsmethodik offen, bevor mit ihr gearbeitet wird: das Forschungsdesign, die RFC-Auswertungsmethode und der Einsatz von KI.
- **Anwendungsteil:** Kapitel 4 überführt RFC 9868 mit dieser Methode in ein prüfbares Anforderungsmodell und begründet die Technologieauswahl. Kapitel 5 beschreibt Entwurf und Implementierung der Bibliothek entlang ihrer Datenpfade. Kapitel 6 berichtet die Ergebnisse zu beiden Forschungsfragen sowie eine deskriptive Auswertung der KI-gestützten Arbeitsweise.
- **Schluss:** Kapitel 7 wiederholt die Forschungsfragen, beantwortet sie einzeln und zieht die Bilanz zur Leitfrage.

Diese lineare Kapitelfolge ist eine rekonstruktive Darstellungsstruktur und kein Abbild des tatsächlichen Arbeitsprozesses. Die Umsetzung verlief iterativ, sodass Entwurf, Implementierung und Test einander fortlaufend bedingten.

1.6 KI-Einsatz

Diese Arbeit ist unter durchgehendem Einsatz künstlicher Intelligenz (KI) entstanden. Der Einsatz betrifft Text und Software. Auf der Textseite erstellten KI-Werkzeuge Entwürfe, überarbeiteten Formulierungen, übernahmen die LaTeX-Formatierung, setzten die Abbildungen als TikZ-Grafiken um und bedienten die LaTeX-Werkzeugkette. Zudem prüften sie den Text wiederholt gegen den Primärtext des RFC 9868. Auf der Softwareseite unterstützten sie Planung, Implementierung, Tests und Verifikation der Bibliothek. Der Autor entschied über die Übernahme der Ergebnisse und verantwortet sie.

Zum Einsatz kommen zwei KI-Werkzeuge, die ein Sprachmodell mit Dateizugriff und Kommandoausführung verbinden: Claude Code von Anthropic und Codex von OpenAI. Ihre Rollenverteilung, die bekannten Schwächen dieser Arbeitsweise und die Prüfkette vor der Übernahme eines Ergebnisses behandelt Kapitel 3.

2. Grundlagen

Dieses Kapitel erklärt zunächst IPv4 und UDP (Abschnitt 2.1, Abschnitt 2.2). Danach folgen die Surplus Area (Abschnitt 2.3) sowie Betriebssystem und Netzpfad (Abschnitt 2.4).

2.1 IPv4

Das Internet Protocol (IP) vermittelt Datagramme zwischen Endsystemen und bildet die Vermittlungsschicht für UDP [5]. **UDP Length** und das durch den IP-Header bestimmte Datagrammende legen Lage und Größe der Surplus Area fest [2, Sec. 7].

Entsprechend der Abgrenzung in Abschnitt 1.4 behandelt dieser Abschnitt nur die für RFC 9868 und die Messung nötigen IPv4-Felder: **IHL** (Internet Header Length), **Total Length**, **Protocol**, Adressen, Header-Prüfsumme, Fragmentierungsfelder und **Time to Live**.

Abbildung 2.1 zeigt den Aufbau des IPv4-Headers.

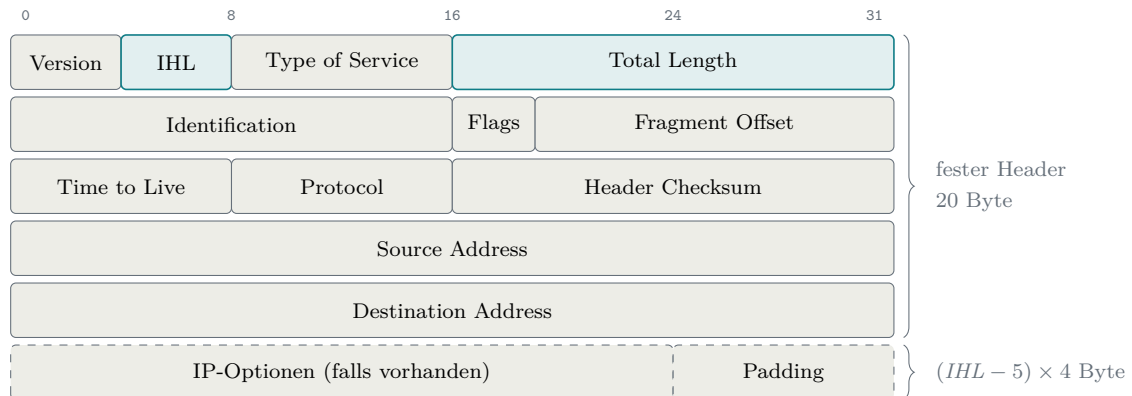


Abbildung 2.1: Der IPv4-Header nach RFC 791 [5]. Hervorgehoben sind die beiden Längenangaben, auf denen die Bestimmung der Surplus Area beruht; gestrichelte Felder sind nur vorhanden, wenn IHL über dem Minimalwert 5 liegt.

Total Length gibt die Gesamtlänge des Datagramms in Byte an, Header und Nutzlast zusammen; als 16-Bit-Feld erlaubt es höchstens 65 535 Byte.

Die zweite Längenangabe, die **IHL**, beziffert die Länge des Headers. **IHL** zählt in 32-Bit-Wörtern zu je 4 Byte. Das 4-Bit-Feld erreicht höchstens den Wert 15 und kann damit Headerlängen bis 60 Byte angeben [5]. Die Headerlänge ergibt sich daher als

$$\text{Headerlänge} = IHL \times 4 \text{ Byte.} \quad (2.1)$$

Der Minimalwert 5 entspricht dem optionslosen 20-Byte-Header. Eine IPv4-Headerlänge ist damit stets ein Vielfaches von 4. Diese Eigenschaft ist für die Ausrichtung der UDP Options wichtig.

Zwischen festem Header und Nutzlast können zudem IP-Optionen liegen: zusätzliche Steuerfelder für Sonderfälle, die die gewöhnliche Kommunikation nicht braucht; **Record Route** etwa sammelt die Adressen der durchlaufenen Router ein [5]. Ihre Verarbeitung belastet je nach Architektur und Konfiguration den allgemeinen Prozessor des Routers. Zudem ist das Verwerfen von Paketen mit IP-Optionen verbreitet [6, Sec. 1 und 3.1]. Der Beginn der Nutzlast folgt damit stets aus der IHL, nie aus einer Konstante.

Total Length und **IHL** bestimmen die Größe der IPv4-Nutzlast: Sie beginnt nach $\text{IHL} \times 4$ Byte und endet bei **Total Length**. Für ein UDP-Datagramm fordert RFC 9868 deshalb [2, Sec. 7]¹

$$\text{UDP Length} \leq \text{Total Length} - \text{Headerlänge}. \quad (2.2)$$

Die rechte Seite der Ungleichung ist der Platz hinter dem IP-Header, die linke die Länge, die das UDP-Datagramm für sich beansprucht: Das UDP-Datagramm muss vollständig in die Nutzlast des IP-Datagramms passen. Bei Gleichheit füllt es sie exakt aus; das Datagramm hat dann keine Surplus Area. Ist **UDP Length** kleiner, bildet die Differenz die Surplus Area (Abschnitt 2.3).

Protocol benennt das Transportprotokoll der Nutzlast; der Wert 17 steht für UDP. Die **Header Checksum** sichert ausschließlich den Header, nicht die Nutzlast; deren Fehlererkennung bleibt der UDP-Prüfsumme überlassen (Abschnitt 2.2.1). Quell- und Zieladresse geben Absender und Empfänger des Datagramms an; die UDP-Prüfsumme bezieht beide später über einen sogenannten Pseudo-Header in ihre Berechnung ein. **Version** und **Type of Service** berühren den Optionsmechanismus nicht.

Time to Live hat keinen Einfluss auf den Optionsmechanismus; die Evaluation nutzt den Wert als Messgröße. Jeder weiterleitende Router verringert **Time to Live** um mindestens eins und verwirft das Datagramm bei null [5]. Wenn jeder Router den Wert bei der Weiterleitung um genau eins verringert, entspricht die Differenz zwischen gesendetem und empfangenem Wert der Zahl der durchlaufenen IPv4-Router. Diese Arbeit nutzt die Differenz unter dieser Annahme, um die Stabilität der Messpfade zu prüfen (Kapitel 6).

¹ Zusätzliche Header zwischen IPv4- und UDP-Header (Shim-Header) verringern den für das UDP-Datagramm verfügbaren Platz um ihre Gesamtlänge; das Feld **Protocol** zeigt dann nicht UDP an [2, Sec. 7]. Diese Arbeit betrachtet solche Ketten nicht weiter.

Wie groß ein Datagramm praktisch werden kann, entscheidet allerdings nicht das 16-Bit-Längenfeld, sondern der Übertragungsweg. Die Maximum Transmission Unit (MTU) begrenzt die Größe eines IP-Pakets, das eine Verbindung ohne Fragmentierung übertragen kann. Bei Ethernet beträgt sie üblicherweise 1500 Byte. Ein Datagramm kann mehrere Netze mit unterschiedlichen MTUs durchqueren; IP muss deshalb festlegen, was mit einem zu großen Datagramm geschieht [5].

Ist ein Datagramm größer als die MTU und das Flag DF (Don't Fragment) nicht gesetzt, zerlegt die IP-Schicht es in mehrere Fragmente; bei IPv4 darf das auch jeder Router unterwegs. Jedes Fragment erhält einen eigenen IP-Header. **Identification**, **Fragment Offset** und das Flag MF (More Fragments) steuern die Zuordnung. Der Empfänger setzt das Datagramm anhand von Quelladresse, Zieladresse, Protokoll und **Identification** wieder zusammen, bevor er es an die Transportschicht übergibt [5]. Ist DF dagegen gesetzt, verwirft ein Router das zu große Datagramm und meldet dies dem Absender per ICMP; auf dieser Rückmeldung baut die Path MTU Discovery auf, mit der ein Sender die kleinste MTU seines Pfades ermittelt [7, Sec. 3.2].

Der UDP-Header gehört aus Sicht von IP zur Nutzlast und liegt deshalb vollständig im ersten Fragment. Was das für die übrigen Fragmente bedeutet, zeigt Abbildung 2.2.

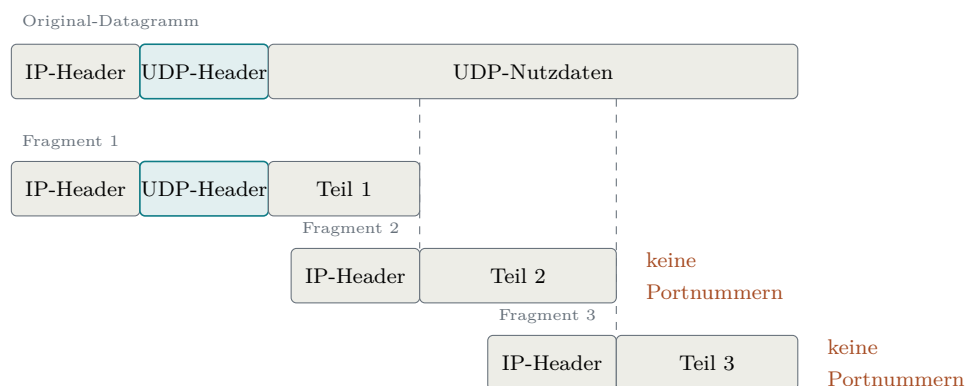


Abbildung 2.2: Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente

Aus Abbildung 2.2 geht hervor, dass nur das erste Fragment den UDP-Header enthält: Alle Folgefragmente tragen keine Portnummern. Manche NATs und Firewalls benötigen vollständige IP-Pakete, setzen Fragmente jedoch nicht zusammen und verwerfen sie deshalb pauschal. Außerdem macht der Verlust eines einzelnen Fragments das gesamte Datagramm unbrauchbar. RFC 8085 beschreibt diese Probleme und rät UDP-Anwendungen deshalb, IP-Fragmentierung zu vermeiden [7, Sec. 3.2]. TCP zerlegt seinen Datenstrom selbst in passende Segmente und vermeidet so meist IP-Fragmentierung [8]. UDP erzeugt je Nachricht genau ein Datagramm (Abschnitt 2.2.2). Ein zu großes UDP-Datagramm kann bislang nur die IP-Schicht

zerteilen. Genau diese Lücke schließt die FRAG-Option von RFC 9868: Sie verlagert die Fragmentierung auf die Transportschicht und wiederholt die Portnummern in jedem Fragment [2, Sec. 11.4]; den Verlust einzelner Fragmente behebt allerdings auch FRAG nicht (Kapitel 6).

2.2 Das User Datagram Protocol (UDP)

Das User Datagram Protocol (UDP) ist ein verbindungsloses, nachrichtenorientiertes Transportprotokoll. UDP benötigt keinen vorherigen Verbindungsaufbau und bietet selbst keine Garantie für Zustellung, Reihenfolge oder Duplikaterkennung [1]. Zuverlässigkeit und Staukontrolle muss ein nutzendes Protokoll bei Bedarf oberhalb von UDP bereitstellen [7, Sec. 3.1 bis 3.3]. UDP verwendet die IP-Protokollnummer 17 [1]. RFC 768 wurde seit seiner Veröffentlichung im August 1980 über 45 Jahre hinweg nicht formell aktualisiert. Erst RFC 9868 trägt den Header-Eintrag **Updates: 768** und erweitert damit den ursprünglichen UDP-Standard erstmalig direkt [2].

UDP geht auf die Aufspaltung des ursprünglichen *Transmission Control Program* zurück, in dem Cerf, Dalal und Sunshine Ende 1974 Netzwerk- und Transportfunktionen noch gemeinsam beschrieben [9]. Die Trennung in IP und TCP schuf Raum für UDP als transaktionsorientierte Alternative ohne den Zusatzaufwand von TCP [1]. Begleitdokumente wie RFC 8085 geben Einsatzrichtlinien für UDP vor, ohne den Standard formell zu ändern [7].

2.2.1 Protokollstruktur

Ein UDP-Datagramm besteht aus einem Header mit fester Länge von 8 Byte und den darauf folgenden Nutzdaten; den Aufbau des Headers zeigt Abbildung 2.3 [1].

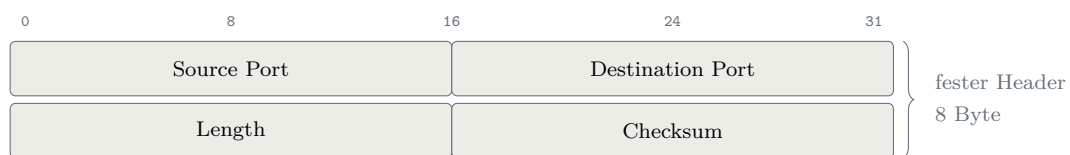


Abbildung 2.3: Der UDP-Header nach RFC 768 [1]

Der UDP-Header besteht aus vier Feldern zu je 16 Bit (Abbildung 2.3):

- **Source Port:** Die Portnummer des sendenden Prozesses. Dieses Feld ist optional; wird es nicht benötigt, wird es auf null gesetzt [1].² In der Praxis nutzt die empfangende Anwendung den *Source Port*, um den Absender zuzuordnen und Antworten an dessen Port zu senden.
- **Destination Port:** Die Portnummer des Zielprozesses auf dem empfangenden Host. Anhand von IP-Zieladresse und Zielpport ordnet der Empfänger eingehende Datagramme der richtigen Anwendung zu.
- **Length:** Die Gesamtlänge des UDP-Datagramms in Byte, bestehend aus Header *und* Nutzdaten. Der Minimalwert beträgt 8, was einem Datagramm ohne Nutzdaten entspricht [1]. Da das Feld 16 Bit breit ist, ergibt sich eine theoretische Maximalgröße von 65 535 Byte. Die durch IP gesetzte Obergrenze steht in Gleichung 2.2 (Abschnitt 2.1).
- **Checksum:** Eine Prüfsumme zur Fehlererkennung über einen Pseudo-Header, den UDP-Header und die Nutzdaten; ihr Verfahren und ihre Sonderwerte erläutert der folgende Absatz.

Prüfsumme. Die Prüfsumme dient der Fehlererkennung: Mit ihr erkennt der Empfänger, ob einzelne Bits des Datagramms auf dem Weg verändert wurden. Diese Prüfung auf der Transportschicht ist nötig, weil nicht jede Teilstrecke eines Pfades eigene Fehlererkennung bietet und Bits auch im Speicher eines Routers kippen können [10]. Der Feldwert null bedeutet: keine UDP-Prüfsumme.³ Der Sender berechnet die Prüfsumme aus den 16-Bit-Wörtern von Pseudo-Header, UDP-Header und Nutzdaten. Er addiert sie im Einerkomplement. Dabei addiert er einen Übertrag wieder zum 16-Bit-Ergebnis. Danach invertiert er die Bits des Ergebnisses und trägt den Wert in das Prüfsummenfeld ein [1, 12]. Zur Prüfung einer verwendeten Prüfsumme summiert der Empfänger dieselben Wörter einschließlich des Prüfsummenfelds und erwartet lauter Einsen, also 0xFFFF; jedes andere Ergebnis zeigt einen Übertragungsfehler an [12].

Das Einerkomplement kennt zwei Darstellungen der Null: 0x0000 und 0xFFFF. Eine übertragene 0x0000 bedeutet, dass der Sender keine Prüfsumme berechnet hat. Ergibt die Berechnung selbst den Wert null, wird stattdessen 0xFFFF übertragen. Der Empfänger kann dadurch eine berechnete Prüfsumme mit dem Ergebnis null von einer ungenutzten Prüfsumme unterscheiden [1].

² Dies ist etwa möglich, wenn weder Antworten noch eine Zuordnung des Senders über den Quellport nötig sind; RFC 8085 rät jedoch von Quellport 0 ab [7, Sec. 5.1].

³ Manche Anwendungen verzichten auf die UDP-Prüfsumme, um Rechenaufwand für ihre Berechnung und Prüfung zu sparen [11, Sec. 4.1.3.4].

UDP kann Fehler nur erkennen und nicht beheben; ein beschädigtes Datagramm wird typischerweise verworfen [10].

Pseudo-Header. Der *Pseudo-Header* bezieht ausgewählte IP-Angaben in die UDP-Prüfsumme ein. So können unbeabsichtigte Änderungen an IP-Adressen auffallen, selbst wenn UDP-Header und Nutzdaten unverändert sind [1]. Die UDP-Implementierung des Senders bezieht den Pseudo-Header zusammen mit dem UDP-Header und den Nutzdaten in die Berechnung ein und trägt das Ergebnis in das UDP-Prüfsummenfeld ein. Die UDP-Implementierung des Empfängers bildet den Pseudo-Header aus den empfangenen Angaben und verwendet ihn für die Prüfung einer UDP-Prüfsumme ungleich null [1]. Der Pseudo-Header wird nicht als eigener Header übertragen; seine Werte gehen nur in die Rechnung ein. Seinen Aufbau zeigt Abbildung 2.4.

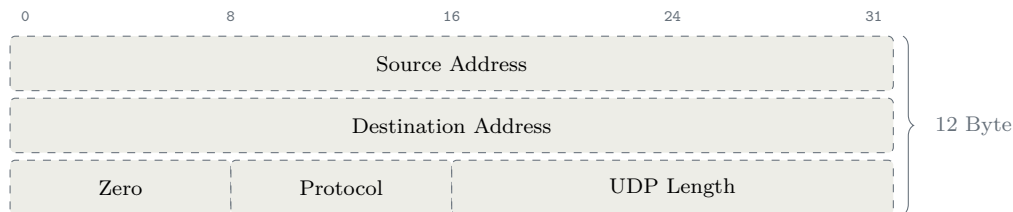


Abbildung 2.4: Der Pseudo-Header der UDP-Prüfsumme für IPv4 [1]

Der Pseudo-Header besteht aus beiden IP-Adressen, einem Nullbyte, **Protocol** und der wiederholten **UDP Length**. Sender und Empfänger verwenden dafür die Werte aus den jeweiligen IP- und UDP-Headern [1]. Ohne Änderungen an den Headern verwenden Sender und Empfänger dieselben Werte. Eine NAT kann Adressen oder Ports umschreiben und die UDP-Prüfsumme entsprechend anpassen, weil diese Felder in deren Berechnung eingehen [2, Sec. 11.3]. Das Nullbyte ergänzt das 8-Bit-Protokollfeld zu einem 16-Bit-Wort. Auch Nutzdaten ungerader Länge werden für die Prüfsummenrechnung mit einem Nulloktett aufgefüllt [1, 12]. Ein veränderliches Feld wie **Time to Live** wäre ungeeignet, weil Sender und Empfänger dann verschiedene Summen bildeten (Abschnitt 2.1).

2.2.2 Eigenschaften

UDP ist nachrichtenorientiert: Jede Sendeoperation erzeugt ein Datagramm, das der Empfänger als Einheit entgegennimmt. TCP stellt dagegen einen Bytestrom bereit, segmentiert ihn selbst und überlässt der Anwendung die Kennzeichnung von Nachrichtengrenzen [8, Sec. 2.2 und 3.7]. UDP nach RFC 768 hält zudem keine Sequenznummern, Fenstergrößen oder Zeitgeber je Kommunikationsbeziehung vor. Nur die Reassemblierung von FRAG-Datagrammen bildet in RFC 9868 eine begrenzte Ausnahme [2, Sec. 6].

Da kein Verbindungsaufbau nötig ist, kann die erste Nachricht sofort gesendet werden [7, Sec. 1]. UDP garantiert weder Zustellung noch Reihenfolge und wiederholt verlorene Datagramme nicht [1]. Später eintreffende Datagramme werden auf UDP-Ebene deshalb nicht wegen eines Verlusts zurückgehalten; dort entsteht kein *Head-of-Line-Blocking* wie bei einem gesicherten Bytestrom. Benötigte Zuverlässigkeit oder Reihenfolge muss die Anwendung selbst herstellen [7, Sec. 3.3].

2.3 Die Surplus Area

Der UDP-Header bietet keinen Platz für zusätzliche Optionsfelder. Zusätze innerhalb der UDP-Nutzdaten würden auch an Anwendungen weitergegeben, die keine Optionen unterstützen. RFC 9868 legt die Optionen deshalb hinter die UDP-Nutzdaten in die Surplus Area [2, Sec. 4 und 7].

2.3.1 Lage und Kompatibilität

UDP `Length` umfasst nur den UDP-Header und die UDP-Nutzdaten. Die Längenangaben im IP-Header können für UDP mehr Platz angeben, als UDP `Length` umfasst (Abschnitt 2.1). Die gesamte Nutzlast hinter dem IP-Header heißt in RFC 9868 *IP Transport Payload*, in dieser Arbeit kurz IP-Nutzlast. Der Teil hinter den UDP-Nutzdaten bis zum Ende des IP-Datagramms heißt *Surplus Area*. Dort liegen die UDP Options. Die Lage beider Bereiche zeigt Abbildung 2.5 an einem Beispieldatagramm mit `Total Length` 60, `IHL` 5 und UDP `Length` 32 [2, Sec. 7]:

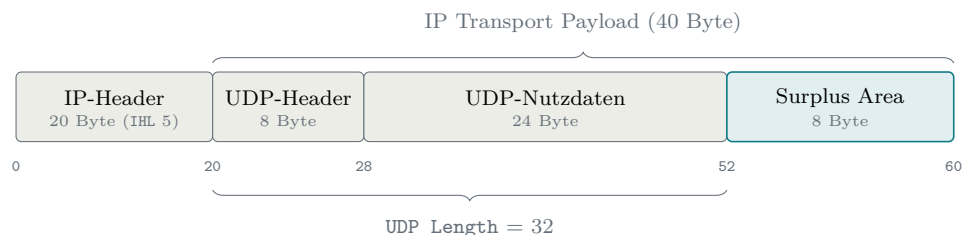


Abbildung 2.5: IP Transport Payload und Surplus Area eines Datagramms mit `Total Length` 60

In Abbildung 2.5 beginnt die IP Transport Payload nach dem IP-Header. `UDP Length` umfasst darin UDP-Header und Nutzdaten; die Surplus Area reicht bis zum Ende des IP-Datagramms. Im Beispiel bleiben von $60 - 20 = 40$ Byte IP Transport Payload nach Abzug von `UDP Length` = 32 genau 8 Byte Surplus Area. Allgemein gilt

$$\textit{Surplus-Länge} = \textit{Total Length} - \textit{IP-Headerlänge} - \textit{UDP Length}. \quad (2.3)$$

Eine negative Surplus-Länge bedeutet, dass `UDP Length` die Obergrenze aus Gleichung 2.2 überschreitet. Der Empfänger verwirft ein solches Datagramm still als ungültig [2, Sec. 10]. Ohne Surplus Area ist die Differenz null. RFC 768 verlangt diese Gleichheit jedoch nicht; RFC 9868 nutzt die seit 1980 mögliche Längendifferenz [2, Sec. 7].

Ein Altempfänger verarbeitet gewöhnlich nur die durch `UDP Length` begrenzten Nutzdaten und ignoriert den Rest. RFC 9868 nennt auch abweichendes Verhalten: Ein eingebettetes Gerät reichte das ganze IP-Datagramm weiter, ein Erkennungssystem meldete die Differenz als Angriff [2, Sec. 18]. Die Erweiterung ist daher für Endsysteme typischerweise, nicht ausnahmslos, abwärtskompatibel; reale Pfade untersucht Kapitel 6. Für IP bleibt die Surplus Area gewöhnliche IP-Nutzlast innerhalb von `Total Length`; neue IP-Felder oder eine geänderte IPv4-Verarbeitung entstehen nicht [2, Sec. 16].

Die Surplus Area darf an jedem gültigen Byte-Offset, auch einem ungeraden, beginnen (Abschnitt 2.3.2). `UDP Length` gibt weiterhin die Gesamtlänge von UDP-Header und UDP-Nutzdaten an. Der Empfänger kann damit zugleich den Beginn einer vorhandenen Surplus Area bestimmen: Sie beginnt unmittelbar hinter diesem Bereich [2, Sec. 7 und 21].

2.3.2 Aufbau und Ausrichtung

Wenn die Surplus Area UDP Options enthält, ist die Verwendung aller Bytes festgelegt. An der ersten 2-Byte-Grenze der Surplus Area steht das 2 Byte große Prüfsummenfeld *Option Checksum* (OCS). Die Bytepositionen werden ab dem Beginn des IP-Datagramms gezählt. Beginnt die Surplus Area an einem ungeraden Offset, setzt der Sender genau ein Nullbyte vor das OCS, um es auszurichten (das *Pad*). Hinter dem OCS folgt die Optionsliste im TLV-Format (Type-Length-Value, Abschnitt 2.3.4). Bei gewöhnlichen Datagrammen reicht sie bis zum Ende der Surplus Area oder endet vorzeitig mit EOL; danach folgen nur noch Nullbytes [2, Sec. 8]. Bei UDP-Fragmenten folgen hinter dem Optionsbereich noch die Fragmentdaten (Abschnitt 2.3.5). Diesen Aufbau zeigt Abbildung 2.6 an einem Datagramm mit 5 Byte Nutzdaten und einer einzelnen Option:

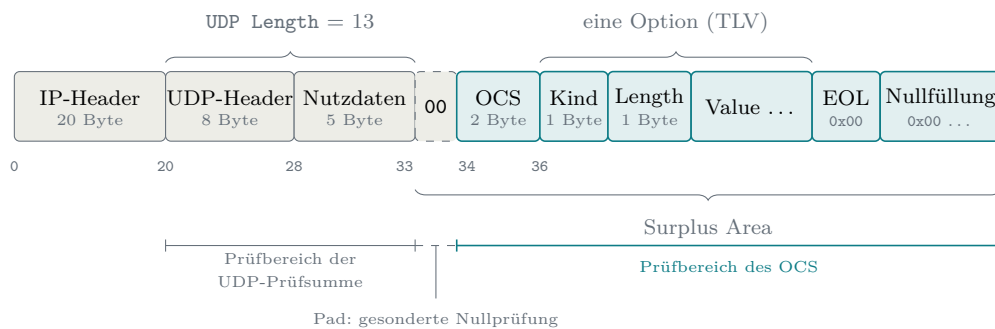


Abbildung 2.6: Aufbau eines Datagramms mit UDP Options

In Abbildung 2.6 gilt $\text{UDP Length} = 8 + 5 = 13$. Die Surplus Area beginnt deshalb bei Byte $20 + 13 = 33$. Dieser Offset ist ungerade, also fügt der Sender ein Pad-Byte ein. Das OCS beginnt bei Byte 34, die erste Option bei Byte 36. Abbildung 2.6 zeigt unten auch die Prüfbereiche bei verwendeten Prüfsummen: Die UDP-Prüfsumme erfasst den UDP-Header und die Nutzdaten; zusätzlich geht der Pseudo-Header in ihre Berechnung ein (Abschnitt 2.2.1). Mit dem OCS prüft der Empfänger die Surplus Area ab dem OCS-Feld. Ein davorliegendes Pad-Byte prüft er gesondert auf null [2, Sec. 8 und 9].

Der Sender darf Optionen an jedem gültigen **UDP-Length**-Offset beginnen lassen. Ausrichtungsbytes vor dem OCS müssen null sein [2, Sec. 8]. Für Fehler in der Surplus Area gilt im Standardfall: Der Empfänger verwirft die Optionsliste still und stellt die Nutzdaten trotzdem zu. Das gilt auch bei einem Ausrichtungsbyte ungleich null und entspricht dem Verhalten eines Altempfängers [2, Sec. 8 und 14].

Dieser Standardfall setzt ein gültiges Datagramm voraus. Ist **UDP Length** ungültig oder schlägt die UDP-Prüfsumme fehl, verwirft der Empfänger das ganze Datagramm, noch bevor er Optionen betrachtet. Eine zulässig ungenutzte Prüfsumme

(Abschnitt 2.2.1) ist dagegen kein Fehler [2, Sec. 10 und 14]. Außerdem darf die Anwendung kein strengeres Verhalten angefordert haben. FRAG und UNSAFE sind von dieser vereinfachten Regel ausgenommen; ihre Sonderfälle behandelt Abschnitt 2.3.5 [2, Sec. 12 und 14].

Das OCS wird aus 16-Bit-Wörtern berechnet; die 2-Byte-Ausrichtung vermeidet dabei das Vertauschen von Bytes [2, Sec. 8]. Bei einem geraden Startoffset ist kein Pad-Byte nötig, bei einem ungeraden genau eines. Den Unterschied zeigt Abbildung 2.7 an zwei sonst gleichen Datagrammen, deren Nutzdaten sich um ein einziges Byte unterscheiden:

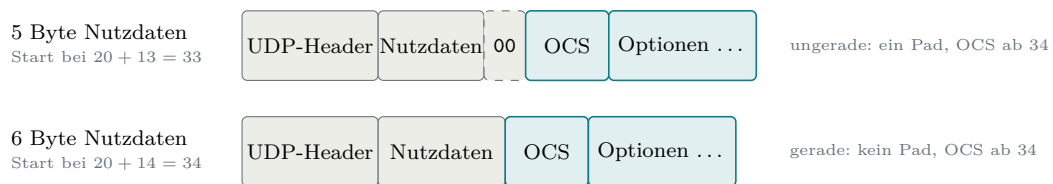


Abbildung 2.7: Pad-Byte bei geradem und ungeradem Startoffset

Bei einem IP-Header ohne Optionen beginnt die Surplus Area bei Byte $20 + \text{UDP Length}$. Durch ein zusätzliches Nutzdatenbyte wird aus einer geraden `UDP Length` eine ungerade oder umgekehrt. Dadurch ändert sich, ob ein Pad-Byte nötig ist (Abbildung 2.7). An der Größe der Surplus Area hängt zugleich, ob sie überhaupt Optionen trägt: Nötig sind mindestens 2 Byte bei geradem und 3 Byte bei ungeradem Start, damit das ausgerichtete OCS vollständig hineinpasst. Ein kleinerer Überschuss enthält keine Optionen, ist aber kein Fehlerfall. Passt genau das OCS hinein und sonst nichts, ist die Optionsliste leer und das Datagramm ebenfalls gültig [2, Sec. 7 und 8]. Auch ein Datagramm mit `UDP Length` 8 und damit ohne UDP-Nutzdaten kann Optionen tragen [2, Sec. 8]. Die FRAG-Option nutzt dies: Ihre Fragmente transportieren ihre Daten vollständig in der Surplus Area.

2.3.3 OCS und Fehlerbehandlung

Das OCS ist eine gewöhnliche Internet-Prüfsumme mit derselben Einerkomplementarithmetik, die Abschnitt 2.2.1 für die UDP-Prüfsumme beschreibt [12]. Es dient vor allem dazu, zufällige Fehler und nicht standardkonforme Nutzungen der Surplus Area zu erkennen. Es schützt nicht gegen gezielte Angriffe [2, Sec. 9].

Der Sender summiert die 16-Bit-Wörter ab dem ausgerichteten OCS-Feld bis zum Ende der Surplus Area. Das OCS-Feld behandelt er dabei als null [2, Sec. 8 und 9]. Zusätzlich addiert er die Länge der gesamten Surplus Area einschließlich des Pads als vorzeichenlosen 16-Bit-Wert. Diese Länge ergibt sich aus den Längenfeldern des

IP- und UDP-Headers (Gleichung 2.3); sie steht in keinem eigenen Feld des Pakets [2, Sec. 9]. Der Sender trägt das invertierte Ergebnis in das OCS-Feld ein.

Wenn die UDP-Prüfsumme ungleich null ist, muss auch das OCS ungleich null sein. Ein OCS-Wert von 0x0000 kennzeichnet ein ungenutztes OCS. Das ist nur zulässig, wenn auch die UDP-Prüfsumme null ist. Implementierungen müssen standardmäßig ein OCS ungleich null setzen [2, Sec. 9]. Abschnitt 9 enthält keine ausdrückliche Senderegel für den Fall, dass die OCS-Berechnung null ergibt. Aus der Kennzeichnung eines ungenutzten OCS mit 0x0000 folgt jedoch für ein verwendetes OCS: Der Sender trägt eine berechnete Null als 0xFFFF ein, wie bei der UDP-Prüfsumme [1].

Der Empfänger bestimmt die Surplus-Länge unabhängig vom Sender aus den Längensummanden. Ein Pad-Byte vor dem OCS prüft er gesondert auf null [2, Sec. 8 und 9]. Bei verwendetem OCS summiert er die Wörter ab dem OCS-Feld bis zum Ende der Surplus Area einschließlich des empfangenen OCS-Werts sowie den Längensummanden. Er erwartet wie bei der UDP-Prüfsumme in Abschnitt 2.2.1 lauter Einsen, also 0xFFFF [12]. Schlägt die OCS-Prüfung fehl, gilt die Grundregel aus Abschnitt 2.3.2: Der Empfänger verwirft im Standardfall die Optionen still und stellt die Nutzdaten bei bestandener oder zulässig ungenutzter UDP-Prüfsumme dennoch zu [2, Sec. 9].

Tabelle 2.1 zeigt die Berechnung an einem kleinen Beispiel.

Tabelle 2.1: OCS-Rechnung einer 6-Byte-Surplus-Area bei geradem Startoffset

Schritt	Sender	Empfänger
Optionswörter hinter dem OCS	0x0101, 0x0000	0x0101, 0x0000
Längensummand	0x0006	0x0006
OCS-Wort	0x0000 als Platzhalter	empfangenes 0xFEf8
Einerkomplementsumme	0x0107	0xFFFF
Ergebnis	Invertierung ergibt 0xFEf8	OCS ist gültig

Die Optionswörter stehen für zwei NOP, ein EOL und ein Nullbyte. Die zwei NOP dienen nur der Rechnung; als Füllung vor EOL rät RFC 9868 von ihnen ab. Ein regulärer Sender würde in diesem Beispiel EOL unmittelbar hinter das OCS setzen [2, Sec. 11.1].

Manche fehlerhafte Middleboxes berechnen die UDP-Prüfsumme über die gesamte IP-Nutzlast statt nur bis UDP Length und verwenden auch im Pseudo-Header diese größere Länge. Weil das OCS über die Surplus Area und deren Länge gebildet und anschließend invertiert wird, gleicht es den Beitrag der fälschlich einbezogenen

Surplus Area und den Längenzuschlag im Pseudo-Header aus. Mit gültigem OCS fällt die UDP-Prüfsumme in diesem Fall genauso aus wie bei korrekter Berechnung [2, Sec. 9].

2.3.4 TLV-Format und Empfangsentscheidung

Die Optionen hinter dem OCS folgen dem TLV-Format. Das 1 Byte große Feld **Kind** identifiziert die Option. Das 1 Byte große Feld **Length** gibt die Gesamtlänge einschließlich **Kind** und **Length** an. Dahinter kann ein Wert folgen [2, Sec. 8 und 10]. Im Standardformat ist die gesamte Option einschließlich **Kind** und **Length** höchstens 254 Byte lang. Größere Optionen verwenden ein erweitertes Format: Der Wert 255 im **Length**-Feld kündigt ein zusätzliches 16-Bit-Längenfeld an. Damit sind je Option höchstens 65535 Byte einschließlich der Headerfelder kodierbar [2, Sec. 10].

RFC 9868 setzt keine zusätzliche Gesamtgrenze für den Optionsraum; maßgeblich ist die Grenze des UDP-Pakets selbst (Abschnitt 2.3.5). Bei einem nicht fragmentierten Datagramm begrenzt der im IP-Datagramm verfügbare Platz, wie viele Optionen es trägt.

Die beiden Ein-Byte-Codes **NOP** und **EOL** besitzen kein Längenfeld. **NOP** (No Operation, Kind 1) ist ein Byte lang und dient der Ausrichtung zwischen Optionen. Dafür darf **NOP** mehrfach vorkommen. **EOL** (End of Options List, Kind 0) beendet die Liste vorzeitig; die Bytes dahinter muss der Sender bis zum Ende der Surplus Area beziehungsweise des Optionsbereichs eines UDP-Fragments auf null setzen [2, Sec. 8 und 11.1]. Der Empfänger darf diese Nullfüllung prüfen, muss es aber nicht. Findet er dabei ein Byte ungleich null, verwirft er nach der Grundregel aus Abschnitt 2.3.2 die gesamte Optionsliste still [2, Sec. 11.1].

Für die Anzahl der Optionen gibt es keine feste Obergrenze. Bei gewöhnlichen Datagrammen endet die Liste ohne vorzeitiges **EOL** erst am Ende der Surplus Area. Außer **FRAG**, **NOP** und den Experimentaloptionen soll jede Option höchstens einmal je Datagramm vorkommen. Wiederholt sich eine der übrigen Optionen, wertet der Empfänger nur ihr erstes Vorkommen aus. Mehrfaches **FRAG** macht die gesamte Optionsliste ungültig [2, Sec. 10]. **NOP** soll höchstens siebenmal in Folge stehen und nicht als Ersatz für **EOL** samt Nullfüllung dienen [2, Sec. 11.2].

Jede Option muss vollständig in die Surplus Area passen. Reicht eine Option laut ihrem **Length**-Feld über das Ende der Surplus Area hinaus, gilt die gesamte Surplus Area als fehlerhaft. Der Empfänger verwirft dann still alle Optionen [2, Sec. 10]. Das verifizierte Erratum EID 8834 bestätigt diese Regel und korrigiert die ältere Formulierung in Abschnitt 14 [13]. Der Sender muss in der Surplus Area von vornherein genügend Platz für das OCS und alle Optionen vorsehen [2, Sec. 8].

Abbildung 2.8 zeigt, in welcher Reihenfolge ein optionsfähiger Empfänger das Datagramm und die Surplus Area prüft:

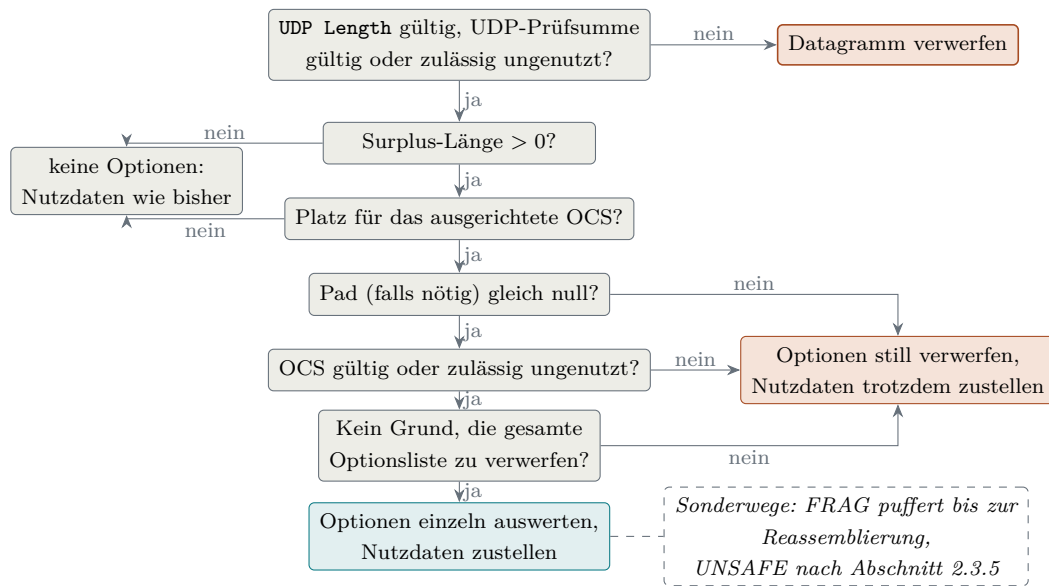


Abbildung 2.8: Empfangsentscheidung für die Surplus Area im Standardfall

Im Standardfall können die Nutzdaten nur bei der vorgelagerten Prüfung des Datagramms verworfen werden. Der Empfänger verwirft das ganze Datagramm, wenn `UDP Length` ungültig ist oder die UDP-Prüfsumme fehlschlägt [2, Sec. 10 und 14]. Ist keine Surplus Area vorhanden oder passt das ausgerichtete OCS nicht hinein, stellt er die Nutzdaten ohne Optionen zu. Bei einem Fehler im Pad oder OCS verwirft er im Standardfall nur die Optionen und stellt die Nutzdaten trotzdem zu. Dasselbe gilt, wenn er die gesamte Optionsliste verwirft (Abschnitt 2.3.2).

Der Empfänger ignoriert unbekannte SAFE-Optionen einzeln. Bei wiederholten Optionen wertet er nur das erste Vorkommen aus; ausgenommen sind FRAG, NOP und die Experimentaloptionen. Strukturelle Längenfehler und mehrfaches FRAG machen die ganze Optionsliste ungültig. Stehen verpflichtend zu unterstützende Optionen (im RFC *must-support*, ausgenommen NOP und EOL) nicht vor den übrigen SAFE-Optionen, darf der Empfänger alle Optionen verwerfen. Prüft er freiwillig die Nullfüllung hinter EOL und findet dabei ein Byte ungleich null, muss er alle Optionen verwerfen [2, Sec. 10 und 11.1].

Die dargestellte Empfangsentscheidung gilt für den Standardfall. Der Empfänger puffert UDP-Fragmente und stellt ihre Nutzdaten erst nach der Reassemblierung zu; darauf weist die gestrichelte Randnotiz hin. UNSAFE-Optionen und strengere, von der Anwendung angeforderte Empfangsregeln behandelt Abschnitt 2.3.5.

2.3.5 Entwurfsprinzipien und Optionsklassen

RFC 9868 legt sechs Entwurfsprinzipien für UDP Options fest, damit UDP seine bisherigen Eigenschaften behält [2, Sec. 6]:

- **Zustandslos:** Nötiger Zustand gehört in die Anwendung oder in eine Bibliothek, die in ihrem Auftrag arbeitet. Die einzige, ausdrücklich begrenzte Ausnahme ist die Zusammenführung von Fragmenten.
- **Unidirektional:** Die UDP-Schicht antwortet nicht selbstständig auf eine Option. Über eine Antwort entscheidet die Anwendung oder eine Schicht beziehungsweise Bibliothek in ihrem Auftrag. Verfahren, die auf Antworten angewiesen sind, verlagert der RFC in eigene Dokumente. Diese Arbeit wertet den Verzicht auf automatische Antworten als Schutz vor Missbrauch: Ohne solche Antworten kann der Grundmechanismus nicht dazu missbraucht werden, Antwortdatagramme an gefälschte Absenderadressen zu senden. Ein ausdrücklich aktiviertes Antwortverfahren muss diesen Missbrauch selbst abwehren. RFC 9869 schreibt beispielsweise vor, die Zahl eigens erzeugter Antwortdatagramme pro Zeit zu begrenzen [3, Sec. 3.1].
- **Keine eigene Längengrenze:** Für den Optionsraum gilt allein die Grenze des UDP-Pakets selbst; bei einem nicht fragmentierten Paket ist das der im IP-Datagramm verfügbare Platz. Der RFC nennt TCP-Optionen (40 Byte Optionsraum [8]) und IPv4-Adressen (32 Bit [5]) als Beispiele für feste Grenzen, die sich später als zu eng erwiesen haben. Grenzen darf eine Implementierung setzen, nicht die Spezifikation.
- **Kein Protokollersatz:** Optionen liefern Funktionen für den eigenen Verkehr einer Anwendung; sie sollen NTP, ICMP (insbesondere Echo) und Netzwerk-Testgeräte weder ersetzen noch nachbauen.
- **Rahmenwerk statt Protokoll:** RFC 9868 definiert Optionsformate auch dann, wenn sie noch kein vollständiges Verfahren ergeben; die Nutzung regeln andere Dokumente. Die Nutzung von REQ/RES regelt RFC 9869 [3]; wie TIME genutzt wird, bleibt bewusst offen.
- **Voreinstellung wie beim Altempfänger:** Empfangene Pakete werden standardmäßig auch dann zugestellt, wenn Prüfsummen-, Authentifizierungs- oder Entschlüsselungsprüfungen der Optionen scheitern; einzige Ausnahme sind UDP-Fragmente. Strengeres Verhalten muss die Anwendung ausdrücklich anfordern.

UDP bleibt ein minimales Transportprotokoll, und jede zusätzliche Fähigkeit wird von der Anwendung ausdrücklich aktiviert [2, Sec. 6].

Ein Altempfänger überliest die Surplus Area und ignoriert damit die Optionen. Deshalb unterscheidet RFC 9868 zwischen SAFE- und UNSAFE-Optionen. *SAFE*-Optionen (Kind 0 bis 191) sind genau dafür entworfen: Ein Empfänger, der sie nicht versteht, ignoriert sie, ohne dass sich die Bedeutung der Nutzdaten ändert. In diese Klasse gehören die Ein-Byte-Codes NOP und EOL ebenso wie FRAG [2, Sec. 10 und 11].

UNSAFE-Optionen (Kind 192 bis 255) können dagegen die Nutzdaten verändern oder eine Änderung ihrer Deutung signalisieren, etwa durch Kompression oder Verschlüsselung. Ein Empfänger, der die Option nicht versteht, würde die Nutzdaten falsch deuten. UNSAFE-Optionen dürfen nach RFC 9868 nur in UDP-Fragmenten auftreten. Deren regulärer Nutzdatenteil ist leer; die transportierten Daten liegen im Fragmentdatenbereich der Surplus Area hinter der Optionsliste [2, Sec. 11.4]. Ein Altempfänger sieht dann nur ein leeres Datagramm und verwirft die Daten still, statt sie falsch zu deuten. Unterstützt ein optionsfähiger Empfänger eine UNSAFE-Option nicht, muss er die reassemblierten Nutzdaten ebenfalls still verwerfen. Dasselbe gilt, wenn eine UNSAFE-Option außerhalb eines Fragment- oder Reassemblierungskontexts erscheint. In beiden Fällen wird höchstens ein leeres Datagramm zugestellt [2, Sec. 10, 12 und 14]. Den Maßstab für die Einzelbewertung führt Abschnitt 4.2 ein; den Soll-Ist-Abgleich enthält Abschnitt 6.3. Die Umsetzung der Optionen beschreibt Kapitel 5.

Die FRAG-Option kann ein logisches Anwendungsdatagramm (im RFC *user datagram*) auf mehrere IP-Datagramme verteilen. Ein Altempfänger sieht jedes UDP-Fragment als leeres Datagramm, ein optionsfähiger Empfänger das reassemblierte Datagramm mit Inhalt. Der RFC gleicht die unterschiedliche Anzahl zugestellter Datagramme bewusst nicht an, weil leere Datagramme als Lebenszeichen unzuverlässig sind [2, Sec. 6 und 18].

2.4 Betriebssystem und Netzpfad

Die Betriebssysteme der Endpunkte und die Geräte auf dem Netzpfad bestimmen mit, ob ein Datagramm mit Surplus Area sein Ziel erreicht. Betrachtet wird dabei durchgehend der Linux-Netzwerkstapel für IPv4, auf dem die Referenzimplementierung dieser Arbeit läuft; andere Betriebssysteme und IPv6 bleiben ausgeklammert.

UDP-Socket und Raw Socket. Bei einem gewöhnlichen UDP-Socket prüft der Kernel beim Empfang Länge und Prüfsumme. Anhand von Adresse, Port und Socketmerkmalen ordnet er das Datagramm einem Socket zu. Er liefert nur die Nutzdaten bis UDP `Length` [4]. Damit entspricht diese Sicht dem Altempfänger aus

Abschnitt 2.3; getestete Altempfänger schneiden die Surplus Area ab [2, Sec. 18]. Sendeseitig baut der Kernel den IP- und UDP-Header und lässt hinter den Nutzdaten keinen Raum. Ein UDP-Socket kann die Surplus Area daher weder lesen noch erzeugen.

Beim Senden führt der Weg von `write()` über die Socket- und UDP-Schicht zur IP-Schicht. Beim Empfang verläuft er entsprechend zurück zu `read()`; die Anwendung erhält die Nutzdaten erst nach der Prüfung der UDP-Prüfsumme [4, 14].

Zum Lesen und Erzeugen der Surplus Area ist ein *Raw Socket* nötig. Linux öffnet ihn nur Prozessen mit der Berechtigung `CAP_NET_RAW` [15]. Der Raw Socket arbeitet auf der IP-Schicht direkt mit IP-Datagrammen. Beim Empfang erhält die Anwendung vor dem UDP-Pfad eine Kopie des IP-Datagramms.

Zum Senden nutzt die Referenzimplementierung die Socket-Option `IP_HDRINCL`: Sie baut das vollständige Datagramm einschließlich IP- und UDP-Header selbst (Abschnitt 4.4). Der Kernel richtet die IP-Längenangabe nach dem übergebenen Puffer und nicht nach der UDP-Länge. Bytes, die in diesem Puffer hinter dem UDP-Datagramm liegen, werden dadurch zur Surplus Area [15]. Die Arbeitsteilung zwischen Anwendung und Kernel zeigt Abbildung 2.9:

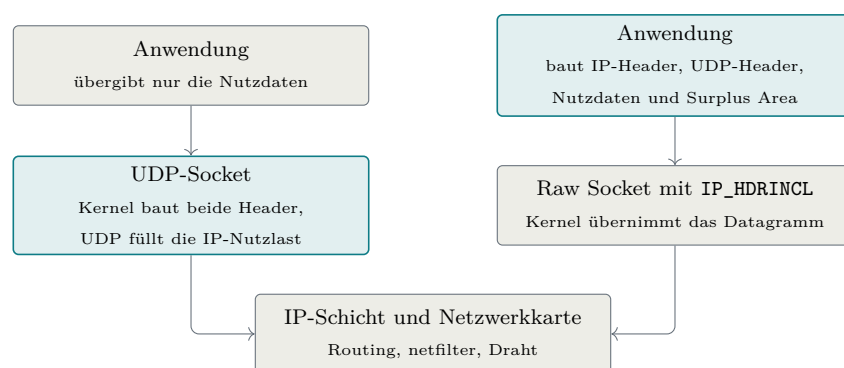


Abbildung 2.9: Die zwei Sendepfade durch den Linux-Kernel [4, 15]

Beide Wege führen über die IP-Schicht mit ihren netfilter-Stationen zur Netzwerkkarte [4].

Beim Senden über den Raw Socket übernimmt der Kernel weiterhin Routing, Nachbarauflösung und Link-Schicht. Im übergebenen IP-Header trägt er **Total Length** und die IP-Header-Prüfsumme stets selbst ein. UDP-Header und Surplus Area lässt er unangetastet; die netfilter-Stationen dieses Wegs können das Datagramm jedoch noch verwerfen oder verändern [4, 15]. Mit `IP_HDRINCL` fragmentiert der Kernel nicht. Ein Datagramm ist dann auf die MTU des Interfaces begrenzt; ein größerer Sendeaufruf schlägt fehl [15]. UDP-Anwendungen sollen IP-Fragmentierung ohnehin

vermeiden (Abschnitt 2.1) [7, Sec. 3.2]. Für große Datagramme stellt RFC 9868 die FRAG-Option auf Transportebene bereit.

Abbildung 2.10 zeigt den Empfangsweg von der Netzwerkkarte zum UDP-Socket und zum Raw Socket:

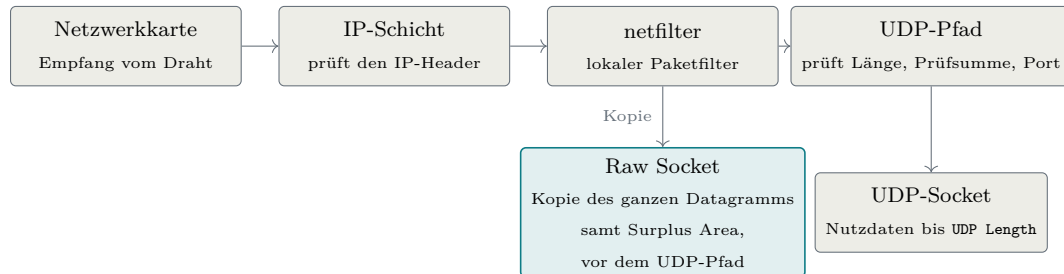


Abbildung 2.10: Der Empfangspfad durch den Linux-Kernel [4, 15]

Aus Abbildung 2.10 geht hervor, dass sich der Weg hinter dem Paketfilter gabelt: Der Raw Socket erhält eine Kopie des gesamten IP-Datagramms samt Surplus Area, und zwar vor den Prüfungen von Länge, Prüfsumme und Portzuordnung im regulären UDP-Empfangspfad [4, 15]. Den IP-Header hat die IP-Schicht zu diesem Zeitpunkt bereits geprüft, wie Abbildung 2.10 zeigt. Sofern vorgeschaltete Filter es passieren lassen, kann ein Datagramm mit falscher UDP-Prüfsumme oder einer `UDP Length`, die über die IP-Nutzlast hinausweist, unverändert am Raw Socket ankommen.

Die Raw-Empfangsschicht muss diese UDP-Prüfungen selbst ausführen. RFC 9868 verlangt folgende Reihenfolge: Zuerst prüft sie Mindestlänge und Längenkonsistenz des UDP-Headers, dann die UDP-Prüfsumme. Erst danach bestimmt sie die Grenzen der Surplus Area und prüft das OCS [2, Sec. 10 und 14]. Wie die Implementierung das umsetzt, zeigt Kapitel 5.

Die netfilter-Hooks bleiben beiden Sichten vorgeschaltet: Ein lokaler Paketfilter kann ein Datagramm verwerfen, bevor irgendeine Anwendung es sieht [4]. Ankommen- de IP-Fragmente setzt die IP-Schicht bereits vor der Zustellung wieder zusammen, auch für Raw Sockets; eine Raw-Empfangsschicht muss die IPv4-Reassemblierung deshalb nicht nachbauen [4, 15].

Akteure auf dem Netzpfad. Zwischen den Endpunkten kann das Datagramm auf *Middleboxes* treffen. RFC 8085 nennt damit Zwischensysteme wie NATs und Firewalls, die zusätzliche Funktionen ausführen und typischerweise Zustand je Fluss führen [7, Sec. 3.5].

Im Zugangsnetz ersetzt die NAT eines Heimrouters oder Hotspots Adressen und Ports. Sie speichert die Zuordnung für jeden Datenfluss für eine begrenzte Zeit. Die Provider-Variante *Carrier-Grade NAT* (CGNAT) verwendet dieselben öffentlichen

Adressen für viele Kunden. Firewalls werfen an Netzgrenzen und Endsystemen Datagramme nach Regelwerk, etwa anhand von Ports und Protokollen.

Auch der Netz- und NAT-Stack der Virtualisierungsschicht eines Testrechners kann Headerfelder auf erwartete Werte umschreiben. Im Transitnetz, der Kette eigenständig verwalteter Netze (autonomer Systeme, AS) zwischen den Zugangsnetzen, können einzelne Netze Datagramme filtern.

Ob ein einzelner Akteur UDP Options kennt, lässt sich von außen nicht feststellen. Unbekannte Zwischensysteme können eine Surplus Area unverändert weiterleiten, verändern, entfernen oder das gesamte Datagramm verwerfen. Zustandsbehaftete NATs und Firewalls können dabei je Richtung und Messzeitpunkt unterschiedlich reagieren [7, Sec. 3.5]. Eine Kapselung, etwa in einen WireGuard-Tunnel, verbirgt das innere Datagramm vor den Akteuren zwischen den Tunnelendpunkten, die nur noch das äußere Tunnelpaket behandeln. Welche dieser Eingriffe auf den untersuchten Pfaden beobachtet wurden, bewertet Kapitel 6.

Middleboxes lassen bevorzugt bekannte Paketformate durch. Dadurch erschweren sie Änderungen an Protokollen; das wird als *Ossifikation* bezeichnet. Protokollerweiterungen nutzen deshalb Bereiche, die Altsysteme nicht deuten. Anwendungen an Altempfängern erhalten die Surplus Area in der Regel nicht. Geräte auf dem Netzpfad können sie dagegen ohne Verschlüsselung lesen [2, Sec. 16]. RFC 9868 berücksichtigt bei der OCS-Berechnung auch Middleboxes, die UDP-Prüfsummen falsch berechnen (Abschnitt 2.3) [2, Sec. 9]. UDP-Relays können eine Surplus Area beim Weiterleiten vollständig abstreifen [2, Sec. 25.6].

Abbildung 2.11 ordnet alle Stationen zu einem Messmodell des Weges von der sendenden Anwendung bis zum Raw-Empfänger, der die Surplus Area auswertet; je nach Pfad können einzelne Stationen fehlen oder zusammenfallen.

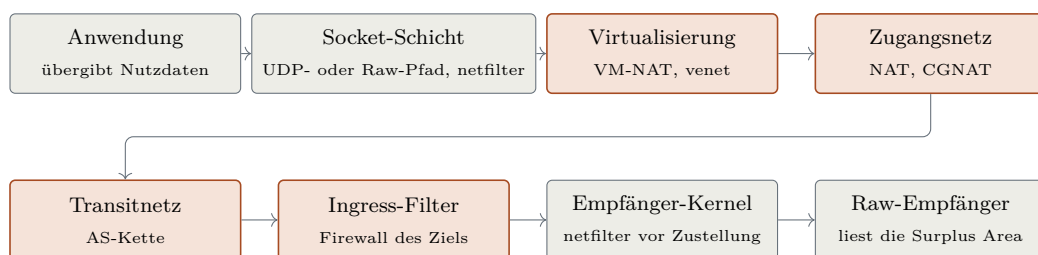


Abbildung 2.11: Messmodell des Datagrammwegs

Abbildung 2.11 zeigt zwischen Socket-Schicht und Empfänger-Kernel bis zu vier Stationen außerhalb der Endpunkte. Einige verarbeiten Datagramme abhängig von gespeichertem Zustand. Ihr Verhalten lässt sich von außen nicht direkt beobachten. Geht ein optionsbehaftetes Datagramm verloren oder kommt es verändert an, unter-

sucht diese Arbeit zunächst mögliche Ursachen auf dem Datagrammweg, bevor sie einen Fehler der Implementierung oder des Standards annimmt. Mit Messpunkten an beiden Enden lässt sich eingrenzen, in welchem Abschnitt des Weges ein Verlust oder eine Veränderung auftritt. Welches Gerät eingreift, bleibt ohne zusätzliche Messpunkte oder unabhängige Evidenz offen (Kapitel 6).

Die Auswertungsmethode folgt in Kapitel 3.

3. Methodik und KI-gestützte Entwicklung

Der Autor setzte große Sprachmodelle bei der Auswertung des RFC-Primärtexts, bei Entwurf und Implementierung, bei der Analyse der Messkampagnen und bei der Prüfung von Zwischenergebnissen ein. Wissenschaftlich belastbar ist dieses Vorgehen nur, wenn die Arbeit offenlegt, wie Modellausgaben geprüft werden und wer die Ergebnisse verantwortet. Diese Regeln müssen vor der Ergebnisbewertung feststehen.

3.1 Anlass des KI-Einsatzes und veröffentlichte Fälle

Diese Arbeit entsteht als Einzelarbeit und verbindet die Auswertung einer neuen Norm, systemnahe Programmierung, formale Modellierung und Messkampagnen. Die Arbeit legt den Einsatz von KI-Werkzeugen offen und beschreibt die Regeln zur Prüfung ihrer Ergebnisse. Zur Einordnung dienen drei veröffentlichte Fälle, in denen KI-Systeme an Forschungs- und Entwicklungsergebnissen beteiligt waren. Sie bilden keine systematische Literaturübersicht, belegen keine einheitliche Prüfkette und erlauben keine Aussage über Häufigkeit oder Wirksamkeit eines Verfahrens. Die folgende Auflistung nennt nur die in den Quellen ausdrücklich dokumentierten Rollen: den Beitrag des KI-Systems, die prüfende Instanz und die Grenze der jeweiligen Prüfung.

- **AlphaProof:** Ein Modell erzeugte formale Beweiskandidaten, die der Kern des Beweisassistenten Lean einzeln nachprüfte; jeder Beweis gilt für die formalisierte Aussage und ihre Annahmen [16].
- **C-Parameter:** Ein Modell steuerte unter Aufsicht Rechnungen, Numerik und Manuskriptteile bei; Physiker prüften mit einer Gegenprobe durch Monte-Carlo-Simulationen, und der Preprint legt den Einsatz offen [17].
- **Bun-Migration:** Bei der Migration von Bun von Zig nach Rust erzeugten und prüften Modelle derselben Familie den Quelltext. Trotz getrennter Rollen blieb ein Fehler unentdeckt, der erst nach dem Merge als undefiniertes Verhalten gemeldet wurde [18–20].

Die Fälle belegen nur, dass die Prüfinstanz zum Artefakt passen muss. Normtext, ausführbare Tests, formale Beweise, Messdaten und menschliche Freigabe haben ver-

schiedene Aufgaben; ihre tatsächliche Nutzung muss dokumentiert sein. Diese Arbeit folgt außerdem der ausdrücklichen Offenlegung des C-Parameter-Falls (Abschnitt 1.6) und versteht auch eine mehrstufige Prüfung nicht als Garantie. Sie wendet denselben Maßstab auf sich selbst an: Jede Artefaktart erhält eine passende Prüfinstanz, und die Arbeitsteilung zwischen Autor und Werkzeugen steht fest, bevor Ergebnisse bewertet werden.

3.2 Forschungsdesign und KI-gestützte Arbeitsweise

Die Untersuchung verbindet die Auswertung der Norm, die Referenzimplementierung und eine empirische Pfaduntersuchung. Abbildung 3.1 zeigt, wie die beiden Forschungsfragen auf diesen Teilen aufbauen:

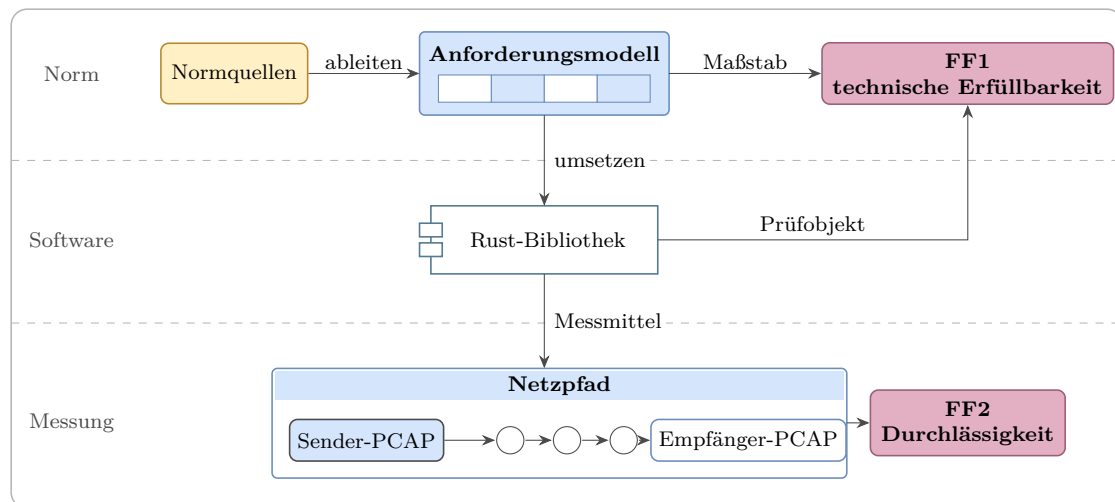


Abbildung 3.1: Aufbau der Untersuchung und Zuordnung der Forschungsfragen

Aus Abbildung 3.1 geht die Doppelrolle der Referenzimplementierung hervor: Für FF1 ist sie der Prüfgegenstand, und das aus den Normquellen abgeleitete Anforderungsmodell bildet den Maßstab; für FF2 dient dieselbe Implementierung als Messmittel. Beide Bewertungen bleiben getrennt: Eine konforme Implementierung belegt keine Zustellbarkeit entlang realer Netzpfade, und eine unveränderte Zustellung belegt keine RFC-Konformität.

Die Forschungsfragen benötigen deshalb verschiedene Untersuchungseinheiten und Entscheidungsregeln. Tabelle 3.1 stellt sie gegenüber:

Aus Tabelle 3.1 geht hervor, dass FF1 je anwendbarer Anforderung und FF2 je kontrolliertem Szenario entschieden wird. Ein Szenario kann je ein Kontroll- und ein Optionsdatagramm, mehrere Wiederholungen oder bei FRAG mehrere

Tabelle 3.1: Operationalisierung der Forschungsfragen

FF	Untersuchungs- einheit	Nachweis	Bewertung und Grenze
FF1	jede anwendbare Zeile des Arbeitsindex aus Abschnitt 4.2 im Projektumfang (eine normative Aussage oder ein Bündel davon)	je nach Status: Normfundstelle sowie Quelltext und Prüfung oder Beleg einer technischen Grenze	vollständig, teilweise oder nicht erfüllbar; nicht anwendbar bleibt unbewertet
FF2	kontrolliertes Szenario je Pfad, Richtung und Messzeitpunkt	Paketmitschnitte an Sender und Empfänger, Sendemanifest und dokumentierte maschinelle Auswertung	Surplus erhalten, verändert oder entfernt; Datagramm verworfen oder Lauf ungültig. Gilt nur für Pfad, Richtung und Messzeitpunkt

übertragene Datagramme enthalten. Für FF1 wird zunächst die Anwendbarkeit bestimmt. Anwendbare **MUST**- und **SHOULD**-Aussagen gehen in die Bewertung ein; eine Abweichung von **SHOULD** benötigt eine dokumentierte Begründung. Eine **MAY**-Funktion wird nur bewertet, wenn sie für die Referenzimplementierung gewählt wurde.

Wie diese Regeln auf die einzelnen Anforderungen angewandt werden, legt Abschnitt 4.2 fest; die Auswertungsregeln der Messläufe für FF2 stehen in Abschnitt 6.1. Neben der Erfüllbarkeit wird der Umsetzungsstand berichtet. Dafür werden die Bibliothek und die Software, die sie aufruft, gemeinsam bewertet; die zugehörige Regel definiert Abschnitt 6.3.

Der Maßstab für FF1 entsteht aus dem Primärtext. Normativ ist RFC 9868; RFC 768 liefert ergänzend das UDP-Format, RFC 1071 als informative Referenz die Rechenregeln der Prüfsummen. Die Schlüsselwörter **MUST**, **SHOULD** und **MAY** werden nach BCP 14¹ gelesen. Nur die großgeschriebenen Formen tragen die dort festgelegte besondere Bedeutung; normative Aussagen können die Schlüsselwörter aber auch auslassen [21, 22]. RFC 9868 stellt Absätzen mit solchen Wörtern zusätzlich das Zeichenpaar >> voran. Die eigene Zählung von 103 markierten Absätzen ist deshalb nur eine Kontrollzahl und keine vollständige Anforderungsliste. Für die Belegform gilt: Normbezüge nennen den maßgeblichen RFC-Abschnitt im Fließtext oder im Quellenverweis; bei BCP-14-Aussagen sowie bei Zahlen und Grenzwerten trägt der Quellenverweis die Abschnittsangabe immer.

¹ BCP steht für Best Current Practice. BCP 14 umfasst RFC 2119 und dessen Aktualisierung RFC 8174.

Jede so gewonnene Aussage wird nach demselben Muster erfasst: Fundstelle, Normstufe, Anwendbarkeit und Projektumfang. Dieses Muster ist zugleich das Zeilenschema des Anforderungskatalogs im Repository der Implementierung, so dass jede Katalogzeile ihren RFC-Abschnitt mitführt. RFC 9869 und die Errata zu RFC 9868 werden gesondert behandelt. RFC 9869 beschreibt die Pfad-MTU-Bestimmung (DPLPMTUD, *Datagram Packetization Layer Path MTU Discovery*) mit UDP-Optionen und liegt außerhalb des Implementierungsumfangs [3]. Von den Errata ist das verifizierte technische EID 8834 angewendet; es löst den Widerspruch zwischen den Abschnitten 10 und 14 zur Behandlung überlanger Optionen [13]. Das redaktionelle EID 8708 bleibt für eine spätere Ausgabe des RFC zurückgestellt [23]. Abbildung 3.2 fasst diesen Weg zusammen:

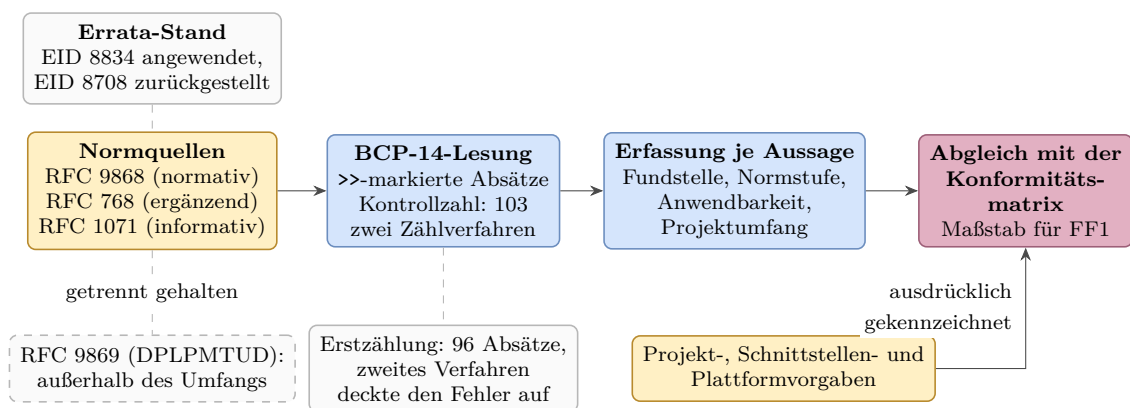


Abbildung 3.2: Von den Normquellen zum geprüften Anforderungsbestand

Abbildung 3.2 zeigt die beiden Grundlagen des Maßstabs: die Anforderungen des RFC unter Berücksichtigung der Errata und die davon getrennt erfassten eigenen Projekt-, Schnittstellen- und Plattformvorgaben. Die eigenen Vorgaben sind ausdrücklich gekennzeichnet. Diese Trennung verhindert, dass eine lokale Entwurfsentscheidung nachträglich als Forderung des RFC erscheint. Die Zahl der markierten Absätze wurde mit zwei Zählverfahren geprüft: Die Erstzählung ergab 96 Absätze; erst das zweite Verfahren deckte den Fehler auf. Zählung und Einordnung erfolgten beim Lesen des RFC-Texts. Ein Werkzeug zur automatischen Zerlegung des Texts wurde dafür nicht eingesetzt.

Diese Erfassung liegt nicht im Quelltext, sondern in versionierten Textdokumenten im Repository der Implementierung.² Die Dokumente halten den Wissensstand über die Arbeitssitzungen hinweg fest und dienen als Maßstab für die Prüfung der Modellausgaben. Ein Abgleich am Primärtext fand vier falsche RFC-Angaben in Dokumenten mit Arbeitsvorgaben; sie wurden dort berichtigt.

² Repository der Implementierung: `docs/requirements.md`, `docs/plan/ROADMAP.md` sowie die Schrittdaten unter `docs/plan/steps/`.

Die Versionsgeschichte schreibt diese Vorgaben dem Autor zu: Die beiden ersten Commits des Repositorys legten Roadmap, Schrittdateien, Anforderungskatalog sowie Architektur- und Formatreferenz an. Die Roadmap beschreibt einen Gesamtplan mit menschlicher Freigabe: Jeder Schritt soll mit einem geprüften Commit abgeschlossen werden. Der Autor legt damit Plan, Maßstab und Abschlusskriterien fest und gibt jedes Ergebnis frei; die Werkzeuge arbeiten innerhalb dieser Vorgaben. Aus den Sammelcommits lässt sich die Reihenfolge der Arbeiten innerhalb eines Schritts nicht rekonstruieren. Die Commit-Historie zeigt außerdem nicht, welchen Beitrag Autor und Werkzeug zu einer einzelnen Änderung geleistet haben.

Jeder Hauptschritt bekommt eine eigene Datei nach festem Schema mit Ziel, Anforderungen, Lean-Pflichten, Plan, Aufgaben und Abschlusskriterien; alle Schrittdateien lagen vor dem ersten Funktionsschritt vor. Wo Lean-Pflichten bestehen, verlangen neun Schrittdateien wörtlich die Spezifikation vor der Umsetzung und den Beweis danach; Abbildung 3.3 zeigt den Kernzyklus:

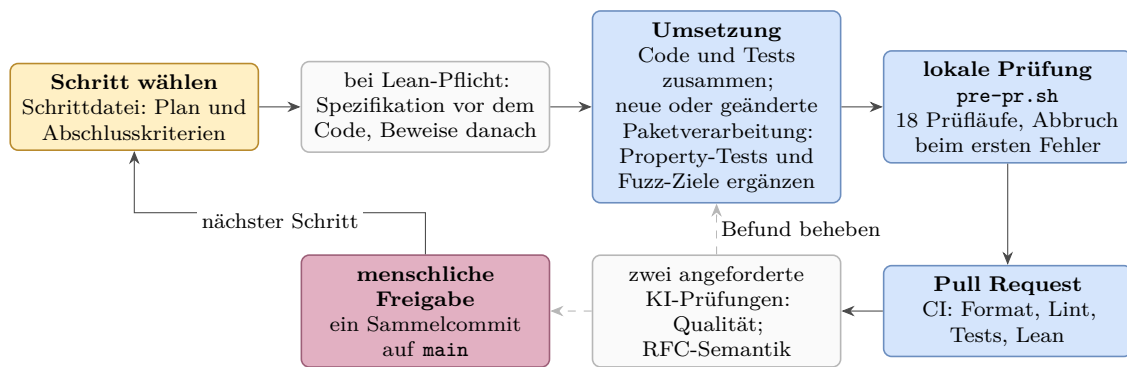


Abbildung 3.3: Kernzyklus eines Implementierungsschritts bis Schritt 18 (Ausnahmen gestrichelt)

Sprachmodelle sind in den Untersuchungen Werkzeuge, aber keine bewertende Instanz. Maßgeblich sind die Normquellen, die Messdaten, ausführbare Prüfungen und die Freigabe durch den Autor. Tabelle 3.2 trennt die Rollen von Autor und Werkzeugen nach Aufgabe und Grenze:

Beide Modellwerkzeuge verbinden ein Sprachmodell mit Dateizugriff, Kommandoausführung und Repository. Der Einsatz von Modellen verschiedener Hersteller soll das Risiko verringern, dass ihre Fehlermuster vollständig übereinstimmen. Unabhängige Prüfungen sind damit jedoch nicht belegt. Eine Prüfung durch dasselbe Modell ist keine getrennte Zweitprüfung. Wenn der Zweitprüfer selbst Änderungen vornimmt, braucht er dafür einen eigenen, vom Autor freigegebenen Auftrag. Diese Änderungen zählen nicht als getrennte Zweitprüfung des bearbeiteten Ergebnisses.

Die Art der Prüfung richtet sich nach der Art der Aussage. Protokollaussagen werden gegen RFC-Primärtext und Errata geprüft. Aussagen über die Implementierung

Tabelle 3.2: Rollen bei Erzeugung, Prüfung und Freigabe

Beteiligter	Aufgabe	Grenze
Autor	legt Plan, Anforderungskatalog, Architektur- und Formatvorgaben sowie die Schrittdateien fest; entscheidet, gewichtet, gibt frei und verantwortet	jedes Ergebnis gilt erst nach seiner Freigabe
Claude Code (Anthropic)	setzt innerhalb dieser Vorgaben um: Entwürfe, Quelltext, Tests und Analysen	Modellausgaben sind keine Nachweise
Codex (OpenAI)	getrennter Zweitprüfer	Herstellertrennung beweist keine Unabhängigkeit

werden durch Quelltext und ausführbare Prüfungen gestützt. Pfadbefunde benötigen Paketmitschnitte und Kampagnenprotokolle. Literaturbehauptungen werden an der jeweiligen Primärquelle geprüft. Ein Hashwert belegt nur die Unverändertheit eines Artefakts, nicht die Wahrheit seines Inhalts.

Die Arbeit mit den Modellen folgt drei schriftlich festgehaltenen Regeln:

1. Modellergebnisse werden gegen die jeweilige Primärquelle geprüft, nie gegen eigene Zusammenfassungen.
2. Aufträge werden neutral formuliert, ohne das Modell in eine gewünschte Richtung zu lenken.
3. Widerspruch wird ausdrücklich ermutigt: Ein Modell, das „nein“ sagt oder Unsicherheit meldet, ist wertvoller als eines, das gefällig zustimmt.

Diese Regeln beschreiben den Sollprozess; welche Prüfungen tatsächlich stattfanden, wird nicht aus ihm abgeleitet, sondern aus den Protokollen rekonstruiert (Abschnitt 3.5). Warum diese Trennung nötig ist, zeigt der folgende Abschnitt.

3.3 Bekannte Schwächen der Sprachmodelle

Tabelle 3.3 ordnet den bekannten Schwächen der Sprachmodelle die beobachtete Wirkung und die methodische Antwort zu.

Tabelle 3.3: Schwächen der Sprachmodelle und methodische Antworten

Schwäche	Wirkung und Beleg	Antwort der Methodik
Halluzination	überzeugend formulierter, aber falscher Inhalt; Trainings- und Bewertungsverfahren können Raten gegenüber eingestandener Unsicherheit belohnen [24]	Prüfung an der Primärquelle, nicht an eigenen Zusammenfassungen
Gefälligkeit (Sycophancy)	erkennbare Erwartungen können Widerspruch unterdrücken; das Verhalten tritt bereits bei reinen Vortrainingsmodellen auf, steigt bei größeren Modellen, bleibt nach Training mit menschlicher Rückmeldung bestehen und verursachte in Versuchen über elf Modelle messbare Schäden [25, 26]	neutrale Aufträge, ausdrücklich erwünschter Widerspruch und getrennter Zweitprüfer
Verfügbarkeit	Cloud-Dienste können ausfallen, gedrosselt oder verändert werden; wechselnde Modelle und nichtdeterministische Antworten verhindern die vollständige Reproduktion einer Modellausgabe	dauerhafte, ohne den Dienst prüfbare Artefakte: Quelltext, Tests, Protokolle und Messdaten

Halluzination und Gefälligkeit können zusammenwirken, wenn eine falsche Aussage unwidersprochen bleibt. Die Gegenmaßnahmen setzen deshalb an Auftrag, Prüfinstanz und Artefakten an. Keine Aussage gilt, weil ein Modell sie behauptet. Konformitätsmatrix und Testarchitektur prüfen die Implementierung; auch sie belegen nur die untersuchten Eigenschaften und keinen allgemeinen Korrektheitsbeweis.

Menschen und Modelle müssen die Verfahrensregeln aus Tabelle 3.3 anwenden. Ausführbare Prüfungen und der Beweisprüfer liefern nach ihrem Start automatisch ein Ergebnis. Wie diese Prüfungen für FF1 aufgebaut sind, legt der folgende Abschnitt fest.

3.4 Testarchitektur für FF1

Die aktuelle Prüfkette wird lokal gestartet, verbindet aber lokale und entfernte Prüfungen. Sie ist der Sollzustand vor dem Öffnen oder Aktualisieren eines Pull Requests. Historisch entstand diese Kette schrittweise.

Dieser Abschnitt beschreibt, wie die Implementierung während der Entwicklung für FF1 geprüft wird. Ein bestandener Entwicklungstest sagt nichts über reale Netzpfade aus, und eine Pfadmessung ersetzt keinen Entwicklungstest. Die Pfaduntersuchung für FF2 verwendet die fertigen Programme, Sendemanifeste sowie Paketmitschnitte an Sender und Empfänger; ihre Ergebnisklassen legt bereits Tabelle 3.1 fest, das Pfad- und Auswertungsmodell folgt in Abschnitt 4.3, die Belegregeln und die Auswertung der Messläufe in Abschnitt 6.1. Welche Auswerter dabei eingesetzt werden und was sie prüfen, beschreibt Abschnitt 4.4.3; die Bewertungsregeln beider Untersuchungen bleiben getrennt.

3.4.1 Prüfmittel und ihre Reichweite

Zu den festen Prüfungen gehören Formatierung, statische Analyse, Dokumentationsbau sowie Unit- und Integrationstests. Ein Property-Test formuliert eine Eigenschaft, die für alle Eingaben einer Klasse gelten soll, und prüft sie an vielen maschinell erzeugten Fällen statt an ausgewählten Beispielen [27].

Ein Fuzz-Ziel ist eine Funktion, die eine beliebige Bytefolge entgegennimmt und damit die echten Verarbeitungspfade der Bibliothek aufruft, vom Zerlegen über das Serialisieren bis zum Aufbau ganzer Datagramme. Der abdeckungsgeleitete Fuzzer erzeugt und verändert solche Eingaben fortlaufend und bevorzugt jene, die neue Programmpfade erreichen [28, 29]. Fuzzing kann Fehler finden, aber keine Fehlerfreiheit beweisen. Es läuft als lokale Pflichtstufe und nicht in der kontinuierlichen Integration.

Lean ist ein Beweisassistent: Regeln werden als Modell formuliert, Aussagen darüber als Theoreme bewiesen, und ein kleiner Kern prüft jeden Beweisschritt nach [30]. Die Lean-Prüfung baut diese Modelle und kontrolliert ihre Axiome. Bewiesen ist damit das handgeschriebene Modell mit seinen Annahmen, nicht der Rust-Quelltext.

Eine besondere Gefahr sind Fehler, die Implementierung und Test gemeinsam haben. Sender und Empfänger der Bibliothek verwenden getrennte Module für Serialisierung und Parsing, beruhen aber auf denselben projektspezifischen Formatannahmen, etwa der gemeinsamen Tabelle der `Kind`-Werte und der Längenkonstanten. Tests, die auf denselben fehlerhaften Annahmen beruhen, können den Entwurfsfehler deshalb übersehen. Bei der Wire-Prüfung zeichnet `tcpdump` die tatsächlich gesendeten

Bytes für eine getrennte Auswertung auf. Ein getrenntes Python-Programm dekodiert IPv4, UDP und die UDP-Optionen und berechnet die Prüfsummen. **tshark** bestätigt zusätzlich die IPv4- und UDP-Felder. Die im Projekt verwendete Version 4.6.4 besitzt jedoch keinen Dissektor für UDP-Optionen.

3.4.2 Prüfkette bis zur Freigabe

Das lokale Prüfskript **pre-pr.sh** führt neun feste Prüfläufe aus: Formatierung, statische Analyse, Dokumentationsbau, Tests mit Property-Fällen, Prüfung des eigenen Auswerters, Lean-Prüfung und drei Prüfläufe für das Linux-Zielsystem. Diese umfassen Cross-Kompilierung und Tests, Raw-Socket-Integration mit Root-Rechten und die Prüfung der tatsächlich gesendeten Pakete. Die Cross-Kompilierung erfolgt auf dem Entwicklungsrechner; die Testprogramme laufen auf der lokalen virtuellen Maschine **achim**. Für jedes der neun Fuzz-Ziele kommt ein weiterer Prüflauf hinzu. Insgesamt ergeben sich die 18 Prüfläufe aus Abbildung 3.3. Die kontinuierliche Integration (CI) wiederholt Formatierung, statische Analyse und Unit-Tests. Nach deren Erfolg baut sie die Lean-Modelle und stellt anschließend zwei Prüfanfragen: an Claude Code zu Codequalität und Sicherheit und an Codex als getrennten Zweitprüfer zur RFC-9868-Semantik.

Der aktuelle Sollprozess ist in Abbildung 3.4 als Sequenzdiagramm dargestellt: Jeder Beteiligte erhält eine senkrechte Lebenslinie, die schmalen Balken zeigen seine aktiven Phasen, durchgezogene Pfeile sind Aufträge und gestrichelte Pfeile Rückmeldungen.

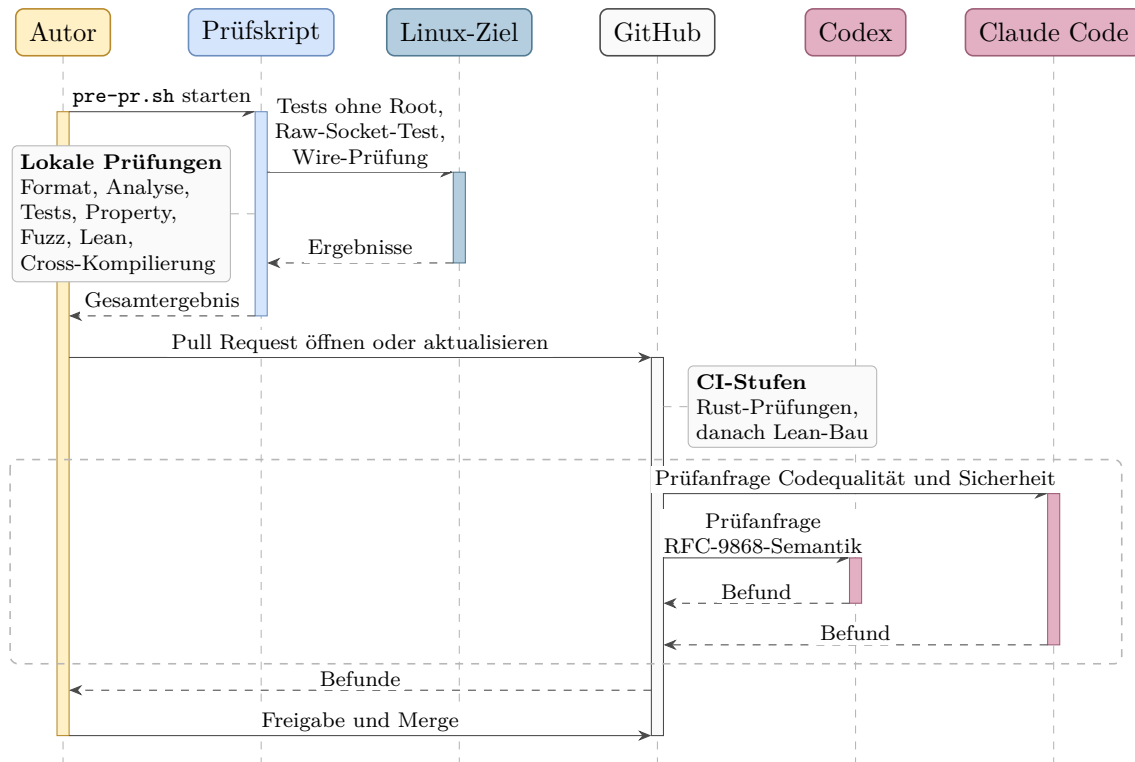


Abbildung 3.4: Prüfprozess im Sollzustand

Aus Abbildung 3.4 geht hervor, dass die Pfeilfolge nur die vorgesehene Reihenfolge vom Prüfskript über den Pull Request und die Rust/Lean-CI bis zur Freigabe durch den Autor zeigt; eine technische Merge-Bedingung ist sie nicht. Nach erfolgreicher Rust/Lean-CI werden beide KI-Prüfungen angefragt.

In dieser Arbeit gilt eine Prüfung nur dann als belegt, wenn sie protokolliert ist. Die Grenzen dieser Protokollierung behandelt der folgende Abschnitt.

3.5 Reproduzierbarkeit und Grenzen

Aussagen über den Entstehungsprozess sind nur so belastbar wie ihre Protokollierung. Maßgeblich sind die vorhandenen Arbeitsprotokolle, also die Repository-Historie, die Prüf- und Kampagnenprotokolle sowie die Messdaten. Technische Ergebnisse sind reproduzierbar, soweit Quellstand, Werkzeuge, Eingaben und Ausgaben archiviert sind. Die genaue Folge nichtdeterministischer Modellausgaben ist es nicht.

Kapitel 4 verwendet die hier festgelegten Quellen- und Bewertungsregeln, um aus RFC 9868 das Anforderungsmodell für FF1 abzuleiten.

4. Analyse und Anforderungsmodell

Kapitel 3 legt die Quellenregeln fest. Dieses Kapitel wendet sie auf RFC 9868 an (Abschnitt 4.1), überführt den Anforderungskatalog in den FF1-Maßstab (Abschnitt 4.2) und bildet das FF2-Pfad- und Auswertungsmodell (Abschnitt 4.3). Die Technologieauswahl (Abschnitt 4.4) schließt die Analyse ab.

4.1 Anwendung der Extraktionsmethode

Die Auswertung wendet die in Abschnitt 3.2 festgelegte Methode auf den Primärtext an. Die 103 mit >> markierten Absätze [2, Sec. 2] dienen als Kontrollzahl; unmarkierte normative Festlegungen bleiben Teil des Maßstabs [21, 22]. Die Auswertung erfasst Aussagen Satz für Satz statt Absatz für Absatz, denn ein markierter Absatz kann mehrere Pflichten mit verschiedenen Adressaten bündeln. Jede erfasste Aussage wird zu einem Prüfpunkt für den Anforderungskatalog der Implementierung (Abschnitt 4.2).

Abbildung 4.1 zeigt, wie sich die 103 markierten Absätze auf die Abschnitte verteilen:

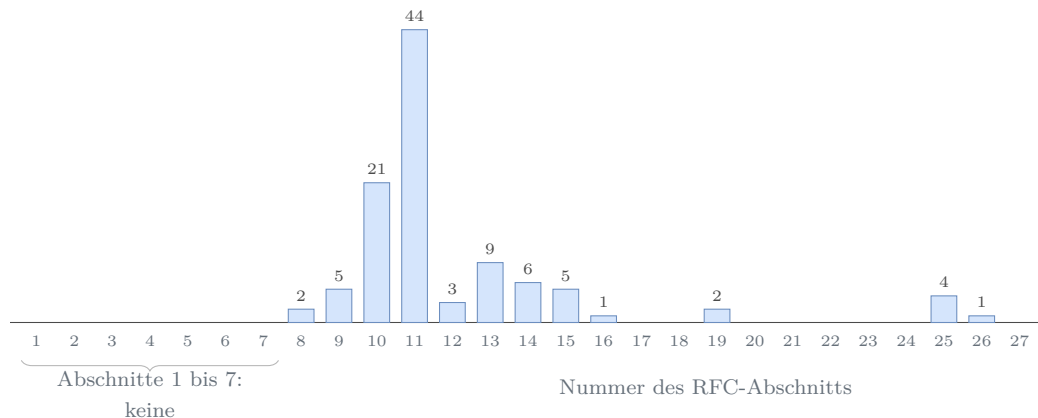


Abbildung 4.1: Verteilung der markierten normativen Absätze über die Abschnitte

Knapp zwei Drittel der markierten Absätze (65 von 103) liegen in den Abschnitten 10 und 11. Sie betreffen das Optionsformat, seine Verarbeitung und die einzelnen Optionen. Bis einschließlich Abschnitt 7 gibt es keine markierten Absätze. Abschnitt 7 enthält dennoch eine für die Arbeit relevante Längenregel: Zusätzliche Header zwischen IPv4- und UDP-Header, sogenannte Shim-Header, verringern die Obergrenze der UDP `Length` um ihre Gesamtlänge (Abschnitt 2.1). Die Implementierung verar-

beitet nur Protocol 17 und verfolgt keine Shim-Ketten. Das ist eine Umfangsgrenze dieser Arbeit.

4.2 Anforderungskatalog und FF1-Maßstab

Das Repository der Implementierung (Kapitel 5) führt in `docs/requirements.md` 49 funktionale Anforderungen (FR) und zwölf nichtfunktionale (NFR). Die Datei entstand vor dem ersten Funktionsschritt und wurde während der Umsetzung fortgeschrieben.

Die Konformitätsmatrix in dieser Datei umfasst 67 Zeilen. Sie dient als **RFC-Arbeitsindex**: Sie ordnet Aussagen des RFC den Anforderungen und ihrem Umsetzungsstand zu. Neben Normsätzen enthält sie Beschreibungen und Leitlinien. Die Prüfung aus Abschnitt 4.1 stuft deshalb jede Zeile erneut am Primärtext ein. Normstufe, Anwendbarkeit, FF1-Bewertung und Implementierungsstand (Abschnitt 6.3) stehen in einer eigenen Datei, der **geprüften Einzelzuordnung**.¹ Zeilen, die mehrere Normsätze oder Adressaten bündeln, etwa die Zeilen zu den Abschnitten 11.2, 16 und 19 sowie 25.2 bis 25.4, bleiben eine Zeile. Bewertet wird jeweils der Anteil, der die Endpunkte im Sinne von FF1 betrifft; die Transitzpflicht aus Abschnitt 16 gehört zu FF2. Tabelle 4.1 zeigt eine eigene übersetzte Auswahl; Normstufe und Abdeckung stehen im Wortlaut des Arbeitsindex:

Tabelle 4.1: Auszug aus dem RFC-Arbeitsindex der Implementierung

Fund-stelle	Normative Aussage	Normstufe	Abdeckung	FR/NFR
Sec. 8	Ausrichtungsbyte vor dem OCS ist null; sonst alle Optionen ignorieren und die Surplus Area still verwerfen	MUST	yes	FR-05
Sec. 9	OCS ungleich null, wenn die UDP-Prüfsumme ungleich null ist	MUST	yes	FR-21
Sec. 10	Optionen über 254 Byte nutzen das erweiterte Format; das kleinste Format ist empfohlen	MUST/SHOULD	yes (send); local strict receive	FR-09, FR-14
Sec. 11.2	Höchstens sieben NOPs in Folge; übermäßige Folgen protokollieren und Ressourcen begrenzen	SHOULD	partial	FR-16, NFR-05
Sec. 12	UCMP und UENC sind reserviert; UEXP ist die UNSAFE-Experimentoption	reserved	out	FR-39

¹ Die Datei `thesis/daten/konformitaet-kategorien.csv` liegt im öffentlichen Repository dieser Arbeit, <https://github.com/ab7z/mcs-thesis-docs> (Stand der abgegebenen Fassung).

Die Abdeckung ist eine Selbstauskunft des Arbeitsindex und kann von der eigenen Bewertung abweichen. Die Zeile zu Abschnitt 11.2 verbindet die NOP-Pflichten mit der bedingten Empfehlung aus Abschnitt 25.2, Ressourcen zu begrenzen [2, Sec. 25.2].

Bewertet wird gegen den Maßstab von FF1: welche Anforderungen sich unter Linux und IPv4 im Benutzerraum „vollständig, teilweise oder nicht“ erfüllen lassen (Abschnitt 1.3). Die Bewertung nutzt drei Kategorien und eine Sonderklasse, die Tabelle 3.1 bereits verankert. **Vollständig** erfüllbar ist eine Anforderung, wenn sie im gewählten Rahmen ohne Abstriche umsetzbar ist. Dieser Rahmen umfasst Linux, IPv4 und Raw Sockets im Benutzerraum sowie die Beschränkung auf `Protocol 17` ohne Shim-Ketten (Abschnitt 1.3, Abschnitt 4.1). **Teilweise** erfüllbar ist sie, wenn nur ein Teil erreichbar bleibt und die Ursache der Einschränkung belegt ist. **Nicht erfüllbar** ist sie, wenn das dokumentierte Verhalten des Betriebssystems oder seiner Schnittstellen die Umsetzung im Benutzerraum grundsätzlich ausschließt. **Nicht anwendbar** bleibt als Sonderklasse sichtbar, wird aber nicht bewertet. Hierhin gehören Aussagen außerhalb des Umfangs, nicht gewählte **MAY**-Funktionen und reine Entwurfsleitlinien, die keine zu bewertende Umsetzungspflicht festlegen.

4.3 FF2-Pfad- und Auswertungsmodell

FF2 fragt, in welchem Umfang die Surplus Area auf den untersuchten realen Netzpfaden bis zur Gegenstelle unverändert erhalten bleibt (Abschnitt 1.3). Ausgewertet wird je Pfad, Richtung und Messzeitpunkt ein kontrolliertes Szenario aus Kontroll- und Optionsverkehr, das mehrere Wiederholungen oder bei FRAG mehrere IP-Datagramme umfassen kann (Tabelle 3.1). Ein gültiger Lauf beschreibt die Surplus Area als **erhalten**, **verändert** oder **entfernt** oder das Datagramm als **verworfen**; fehlerhafte Messungen erhalten den Status **Lauf ungültig** und gehen nicht in die Bewertung ein (Abschnitt 6.1). Die fünf **Pfadstufen** bilden getrennte Messklassen mit unterschiedlicher Beteiligung fremder Netze:

1. **Loopback:** Sende- und Empfangspfad auf demselben Host; prüft Bibliothek und Messwerkzeuge ohne fremde Akteure.
2. **Lokale virtuelle Testumgebung:** Eine Ubuntu-VM unter VMware Fusion, mit Linux-Netzwerk-Namensräumen und `veth`; prüft Weiterleitung, Adressumsetzung und Paketfilter unter eigener Kontrolle.
3. **Tunnel:** WireGuard kapselt das innere Datagramm samt Surplus Area. Der äußere Pfad transportiert die verschlüsselten Tunnelpakete zwischen den Tunnelendpunkten.
4. **Öffentliche Pfade in Europa:** Cloud-Endpunkte in Deutschland und Finn-

land sowie ein Mobilfunk-Zugangsnetz; reale Zugangs-, Transit- und Provider-Kanten.

5. **Interkontinentale Pfade:** Endpunkte in Nordamerika und Asien-Pazifik; lange Transitzketten und die Netzstrukturen großer Cloud-Anbieter.

Vorversuche ohne beidseitige Mitschnitte oder Sendemanifest bleiben Piloten und werden nicht verallgemeinert (Abschnitt 6.1).

Die **Auswertung** verknüpft für jedes gerichtete Szenario die vorhandenen Sende- und Empfangsbelege. Bei vollständigen Kampagnen sind dies das Ergebnis des Sendeaufrufs, das Sendemanifest, Sender-PCAP, Empfänger-PCAP und Empfänger-ausgabe. Das Sendemanifest hält je Sendevorgang unter anderem Folge-Nummer, Adressen und Längen fest. Zusammen helfen die Belege, Senderfehler, Abweichungen auf dem Pfad und Verwerfungen im Empfänger zu unterscheiden.

Zusätzliche Mitschnitte, etwa am macOS-Messpunkt aus Tabelle 4.3, können Teile des Pfads weiter eingrenzen. Ein Befund wird nur dem Abschnitt zwischen den Messpunkten zugeschrieben, die ihn tatsächlich begrenzen. Dieser Abschnitt heißt **Attributionsintervall**. Liegen nur Mitschnitte an den beiden Endpunkten vor, umfasst es den gesamten Pfad dazwischen. Ein konkretes Gerät oder dessen interner Mechanismus lässt sich daraus nicht bestimmen.

Die aus Vorversuchen und Messungen abgeleiteten Mechanismenklassen stellt Kapitel 6 mit ihren Belegen dar.

Zwei Tabellen grenzen abschließend den Geltungsbereich ab: Tabelle 4.2 den Implementierungsumfang, Tabelle 4.3 die bewusst breitere Messinfrastruktur:

Tabelle 4.2: Implementierungsumfang

Merkmal	Festlegung	Außerhalb des Umfangs
Betriebssystem	Linux	andere Betriebssysteme
Vermittlungsschicht	IPv4	IPv6
Ausführungsort	Benutzerraum mit Raw Sockets	Umsetzung im Kernel
Artefakt	Bibliothek mit Mess- und Beispielprogrammen	eigenständiger Dienst; Protokollautomaten für REQ/RES und TIME
Optionsumfang	acht Pflichtoptionen (im RFC <i>must-support</i>) sowie allgemeine Empfangsregeln für unbekannte SAFE- und UNSAFE-Arten	typisierte Erzeugung und Verarbeitung weiterer Arten wie TIME, AUTH, EXP, UCMP, UENC und UEXP

Tabelle 4.3: Messinfrastruktur

Baustein	Rolle in der Messung
Linux-Endpunkte: lokale virtuelle Maschine und Cloud-Instanzen in Europa, Nordamerika und Asien-Pazifik	Sende- und Empfangsendpunkte der Kampagnen; Mitschnitt mit <code>tcpdump</code>
Mobilfunk-Zugang über einen Hotspot	reales Zugangsnetz mit Hotspot-NAT; der Anteil eines möglichen CGNAT bleibt ohne zusätzlichen Messpunkt offen
macOS-Messpunkt <code>en0</code> (<code>access_bpf</code>)	zusätzlicher Beobachtungspunkt am Zugangsnetz; die Bibliothek wird dort nicht eingesetzt
WireGuard-Tunnel	Kontrollpfad mit Kapselung (Pfadstufe 3)

4.4 Technologieauswahl

Die Implementierung entsteht als einbindbare *Bibliothek* mit Mess- und Beispielprogrammen. Diese Form passt zu den Entwurfsprinzipien von RFC 9868: Zustände verwaltet grundsätzlich die Anwendung oder eine Schicht oder Bibliothek in ihrem Auftrag; von dort gehen auch Antworten auf Optionen aus. Die Reassemblierung von Fragmenten ist die einzige, begrenzte Ausnahme beim Zustand (Abschnitt 2.3.5) [2, Sec. 6]. Zu begründen sind die Sprache, der Zugang zur Surplus Area, die Messwerkzeuge sowie Fuzzing und formale Gegenproben.

4.4.1 Programmiersprache

Die Sprache muss vier Bedingungen erfüllen. Erstens braucht die Bibliothek Zugriff auf das vollständige IP-Datagramm einschließlich der Surplus Area (Abschnitt 2.4). Zweitens muss der Serialisierer das Paketformat bytengenau in Netzwerkbyteordnung schreiben; native Speicherabbilder mit möglichem Padding werden nicht als Paketformat verwendet. Drittens verarbeitet der Parser auch Pakete unbekannter oder bösartiger Absender. An dieser *Vertrauensgrenze* muss er fehlerhafte und gezielt konstruierte Eingaben sicher behandeln. Viertens soll die Bibliothek keine verwaltete Laufzeitumgebung voraussetzen und ohne automatische Speicherbereinigung (Garbage Collector) auskommen. Das erleichtert ihre Einbindung und vermeidet GC-Pausen, garantiert aber kein allgemein vorhersagbares Zeitverhalten. Die ersten beiden Bedingungen betreffen den Protokollzugriff, die letzten beiden sind Qualitätsziele der Arbeit.

C, C++, Zig und Rust erlauben den benötigten Zugriff auf Speicher und System-schnittstellen und erfüllen die Vorgabe, ohne Garbage Collector auszukommen. Go und Java werden wegen dieser ausdrücklich gewählten Vorgabe nicht weiter betrachtet.

Den Ausschlag gibt die Speichersicherheit an der Vertrauensgrenze. Sie schließt Zugriffe außerhalb eines Speicherbereichs (*Buffer Overflow*), auf freigegebenen Speicher (*Use-after-free*) und doppelte Freigaben (*Double Free*) aus. Ein *Memory Leak* gewährt dagegen keinen unzulässigen Zugriff und bleibt auch in Rust möglich. Das Microsoft Security Response Center (MSRC) berichtete 2019, dass rund 70 Prozent der Schwachstellen, denen Microsoft jährlich eine CVE-Nummer (Common Vulnerabilities and Exposures) zuweist, auf Speicherfehler zurückgehen [31]. Chromium nennt rund 70 Prozent unter 912 seit 2015 erfassten Fehlern hoher oder kritischer Schwere in der stabilen Auslieferung (Stable Channel) [32]. Die National Security Agency (NSA) empfiehlt speichersichere Sprachen, darunter Rust [33].

Die für die Entscheidung wichtigen Eigenschaften fasst Tabelle 4.4 zusammen [34–37]:

Tabelle 4.4: Vergleich der Sprachkandidaten

Kriterium	C	C++	Zig	Rust
Speicher-sicherheit	nicht garantiert	nicht garantiert	nicht vollständig; Prüfungen je nach Build-Modus	garantiert in Safe Rust; bei unsafe nur unter Bedingungen
Speicher-verwaltung	manuell	manuell oder an Objektlebens-dauer gebunden	explizite Zuteilung und Freigabe	Ownership

Unter den vier Kandidaten verbindet allein Rust den Schutz vor unzulässigen Speicherzugriffen im sicheren Sprachumfang mit einer Speicherverwaltung ohne Garbage Collector. Ein Zugriff mit einem ungültigen Array- oder Slice-Index löst einen definierten *panic* aus [37, 38]. In C ist ein Zugriff außerhalb der Grenzen eines Arrays undefiniertes Verhalten [34]. Die Rust-Garantien setzen voraus, dass die Sicherheitsbedingungen an jeder **unsafe**-Grenze eingehalten werden.² Die Regeln zu Ownership und Lebensdauer prüft der Übersetzer statisch, also ohne den Quelltext auszuführen [37]; dynamische Prüfungen und Fuzzing erfassen dagegen nur die jeweils erreichten Programmpfade [33]. Auch Zig prüft zur Übersetzungszeit, seine Laufzeitprüfungen hängen aber vom Build-Modus ab [36]. Für die Vertrauensgrenze dieser Bibliothek fiel die Wahl deshalb auf Rust.

² Das Rust-Schlüsselwort **unsafe** ist von der UNSAFE-Optionsklasse aus Abschnitt 2.3.5 zu unterscheiden.

Der Parser liest die Optionsbytes als geliehene Slices ohne Kopie; ihre Lebensdauer ist an den Empfangspuffer gebunden [37]. Zwei `unsafe`-Blöcke stehen an den Schnittstellen zum Betriebssystem; ihre Sicherheitsbedingungen stehen als Kommentar am jeweiligen Block, ihre Lage im Modul `socket` nennt Kapitel 5. Externe Parser-Bibliotheken entfallen, weil diese Arbeit die Mechanik von RFC 9868 selbst untersucht.

Safe Rust verhindert keine Protokollfehler. Ein um ein Byte verschobener Startoffset bei ungeradem Beginn der Surplus Area überstand 23 selbstgeschriebene Tests und wurde erst bei der Pull-Request-Prüfung entdeckt. Der Fehler betraf die Protokolllogik, nicht die Speichersicherheit. Daraus entstand die Vorgabe, neue oder geänderte Verarbeitungspfade durch Property-Tests und neue oder erweiterte Fuzz-Ziele abzusichern (Abbildung 3.3).

4.4.2 Zugang zur Surplus Area

Ein gewöhnlicher UDP-Socket unter Linux kann die Surplus Area weder füllen noch auslesen (Abschnitt 2.4) [14]. Packet Sockets arbeiten auf Ebene der Netzgeräte, TUN/TAP benötigt ein virtuelles Netzgerät, und `AF_XDP` verlangt für den Empfang ein XDP-Programm sowie Speicherbereiche und Warteschlangen [39–41]. Ein Kernelmodul liegt außerhalb des Projektumfangs. Raw Sockets geben der Bibliothek ohne eigenes Netzgerät Zugriff auf das vollständige IP-Datagramm. Sie sind damit für den gewählten Umfang der einfachste passende Zugang [15].

Die Bibliothek nutzt diesen Zugang unter Linux. Sein Verhalten ist dokumentiert und im Kernel Quelltext nachprüfbar; außerdem laufen die Sende- und Empfangsprogramme der Kampagnen unter Linux (Abschnitt 4.3). Im Gegenzug prüft die Bibliothek IPv4-Header, Längen, UDP-Prüfsumme und Optionen selbst. Beim Senden liefert sie mit `IP_HDRINCL` den IPv4-Header. Da dieser Pfad nicht fragmentiert, muss jedes gesendete Datagramm mit Optionen in die MTU der ausgehenden Schnittstelle passen (Abschnitt 2.4). Diese Folgen bestimmen den Entwurf in Kapitel 5.

4.4.3 Mitschnitt und Auswertung

Die Kampagnenskripte zeichnen den Verkehr mit `tcpdump` in PCAP-Dateien auf. Eigene Python-Programme werten diese Dateien aus, weil Wireshark in der verwendeten Version 4.6 die UDP-Optionen nicht dekodiert. Ihr Prüfumfang unterscheidet sich je nach Versuch.

Bei der Wire-Prüfung rechnet `wire-check.py` die Prüfsummen einschließlich OCS selbst nach und prüft die Optionsstruktur. `tshark` liefert dazu eine Gegenprobe

der IPv4- und UDP-Felder (Abschnitt 3.4). Auch `eval-check.py`, das bei den lokalen Referenzläufen und in den Kampagnen eingesetzt wurde, prüft OCS selbst.

Die Auswerter der späteren Kampagnen, in Abschnitt 6.1 als P0, P1 und P2 geführt, verknüpfen die Sendebelege mit den Paketmitschnitten und den Empfängerangaben. Den OCS-Status übernehmen sie aus der Empfängerangabe; diese Angaben sind deshalb keine unabhängige OCS-Nachrechnung. Welche Messreihen mit welchen Nachweisen vorliegen, beschreibt Abschnitt 6.1.

4.4.4 Fuzzing und formale Gegenprobe

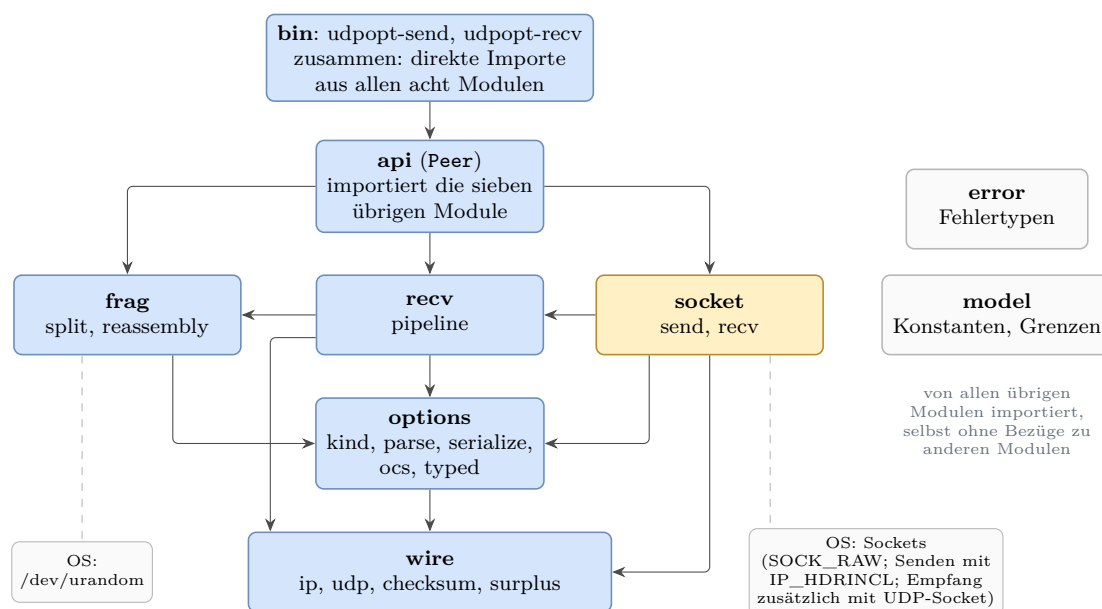
Fuzzing und formale Gegenproben ergänzen die Unit-, Integrations- und Property-Tests (Abschnitt 3.4.1). Sie sollen Fehler aufdecken, die bei der KI-gestützten Entwicklung und ihren bisherigen Prüfungen unentdeckt bleiben können (Abschnitt 3.3). Fuzz-Ziele und Property-Tests verwenden dabei gemeinsame Testmodule; fehlerhafte Annahmen in diesen Modulen können deshalb in beiden Prüfverfahren unentdeckt bleiben. Das über `cargo-fuzz` eingebundene libFuzzer verändert Eingaben des tatsächlichen Parsercodes und bevorzugt solche, die zusätzliche Programmpfade erreichen. Es sucht nach Abstürzen und verletzten Invarianten [29]. Lean beweist ausgewählte Eigenschaften handgeschriebener Modelle unter den Voraussetzungen der jeweiligen Theoreme; den Begriff führt Abschnitt 3.4.1 ein. Fuzzing beweist keine Fehlerfreiheit, und ein Lean-Beweis gilt nicht automatisch für den Rust-Code, der das Modell umsetzt. Auch erfolgreiche Prüfungen ersetzen den Abgleich mit dem RFC nicht. In der Schlussphase deckte ein erneuter Vergleich trotz bestandener Prüfkette weitere Konformitätslücken auf. Sie wurden in einem eigenen Korrekturschritt bearbeitet (Abschnitt 6.3).

5. Entwurf und Implementierung

Dieses Kapitel beschreibt die Referenzimplementierung und das Messinstrument. Es folgt den Datenpfaden der Bibliothek: von der Architektur über das Senden und Empfangen bis zu FRAG, den weiteren Pflichtoptionen und den Betriebsgrenzen.

5.1 Architektur

Die Implementierung umfasst eine Bibliothek sowie die Messprogramme `udpopt-send` und `udpopt-recv`, mit denen die Messungen in Kapitel 6 laufen. Abbildung 5.1 zeigt ihre Aufgaben und ausgewählte Abhängigkeiten.



Pfeil: ausgewählte Importbeziehung, vom importierenden zum importierten Modul. Gestrichelte Linie: ausgewählter Kontakt zum Betriebssystem. Hervorgehoben: das Modul mit den Netz-Systemaufrufen.

Abbildung 5.1: Module der Bibliothek, Messprogramme und ihre Abhängigkeiten

Die Kanten zeigen ausgewählte tatsächliche Importe und keine strikte Schichtung. `wire` verarbeitet IPv4- und UDP-Header sowie Prüfsummen und lokalisiert die Surplus Area. `options` serialisiert und liest den Optionsstrom. `recv` führt für eingehende UDP-Datagramme einschließlich ihrer UDP-Optionen die von RFC 9868 vorgegebenen Prüfschritte nacheinander aus [2, Sec. 14]. `frag` zerlegt und reassembliert Datagramme, und `api` bündelt diese Funktionen im Typ `Peer`. `error` und `model` stellen gemeinsame Fehlertypen, Konstanten und Grenzen bereit.

Im Quelltext unter `src/` liegen die Netz-Systemaufrufe der Bibliothek in `socket`; dort stehen auch die einzigen zwei `unsafe`-Blöcke. Beim Empfang bindet `socket`

zusätzlich einen gewöhnlichen UDP-Socket auf den Zielport; sonst würde der Kernel eintreffende Datagramme per ICMP als unerreichbar zurückmelden.

Für die Kennungsgeneratoren von `Peer` und `udpopt-send` liest `frag` beim Aufbau einmalig vier Bytes aus `/dev/urandom`, falls kein Startwert vorgegeben ist. Die Kennung ist das Feld `Identification` der FRAG-Option: Zusammen mit Adressen und Ports zeigt sie dem Empfänger, welche Fragmente zusammengehören [2, Sec. 11.4]. Der zufällige Start macht es unwahrscheinlich, dass ein neu gestarteter Sender Kennungen wiederholt, die ein Empfänger noch gespeichert hat (Abschnitt 5.4).

`api` und `socket` verwenden die monotone Uhr. Sie misst nur, wie viel Zeit vergangen ist, und wird durch Änderungen der Systemzeit nicht verstellt. `api` liefert damit den Zeitpunkt, mit dem `frag` unvollständige Fragmentsätze nach Fristablauf verwirft. `socket` kann damit die gesamte Empfangsfrist zuverlässig einhalten. Die Protokolllogik in `recv` und `frag` greift selbst weder auf das Netz noch auf die Uhr zu; sie erhält Zeitpunkte und Kennungen als Parameter. So lässt sich jede Protokollentscheidung ohne Netz und ohne erhöhte Rechte prüfen (Abschnitt 3.4.1). Beide Messprogramme schreiben auf die Konsole. Nur `udpopt-send` kann zusätzlich das Sendemanifest fortschreiben, je Sendevorgang eine Zeile im JSON-Format (JSONL).

Die folgenden Abschnitte nutzen drei Pflichtoptionen als Leitbeispiele. Die Option zur maximalen Datagrammgröße (MDS) veranschaulicht den Sendepfad, die zusätzliche Nutzdatenprüfsumme (APC) den Empfangspfad und die Fragmentierungsoption (FRAG) die Zerlegung und spätere Zusammensetzung großer Datagramme.

5.2 Sendepfad und gemeinsamer Optionskern

Beim Senden baut die Bibliothek aus Nutzdaten und Optionen ein vollständiges IPv4-Datagramm und übergibt es dem Raw Socket. Als Leitbeispiel dient eine Anwendung, die drei Byte Nutzdaten `odd` verschickt und ihrem Gegenüber dabei mitteilt, wie große Datagramme sie selbst empfangen kann. Dafür dient die Option Maximum Datagram Size (MDS). Im Beispiel nimmt die Anwendung an, dass IPv4-Datagramme bis 1.500 Byte ohne IP-Fragmentierung bei ihr ankommen. Nach Abzug des 20 Byte langen IPv4-Headers und des 8 Byte langen UDP-Headers trägt MDS deshalb den 16-Bit-Wert $1.500 - 20 - 8 = 1.472$ Byte [2, Sec. 11.5].

MDS ist nur ein Hinweis, keine bekannte Pfadgrenze, und lässt den Platz für Optionen unberücksichtigt. Über seine Nutzung und den nötigen Abzug entscheidet der Aufrufer. Die Bibliothek meldet einen empfangenen Wert daher nur. Für das Beispiel übergibt die Anwendung dem Sendepfad die Nutzdaten und genau eine MDS-Option. Abbildung 5.2 zeigt die drei Zustände bis zum fertigen Datagramm.

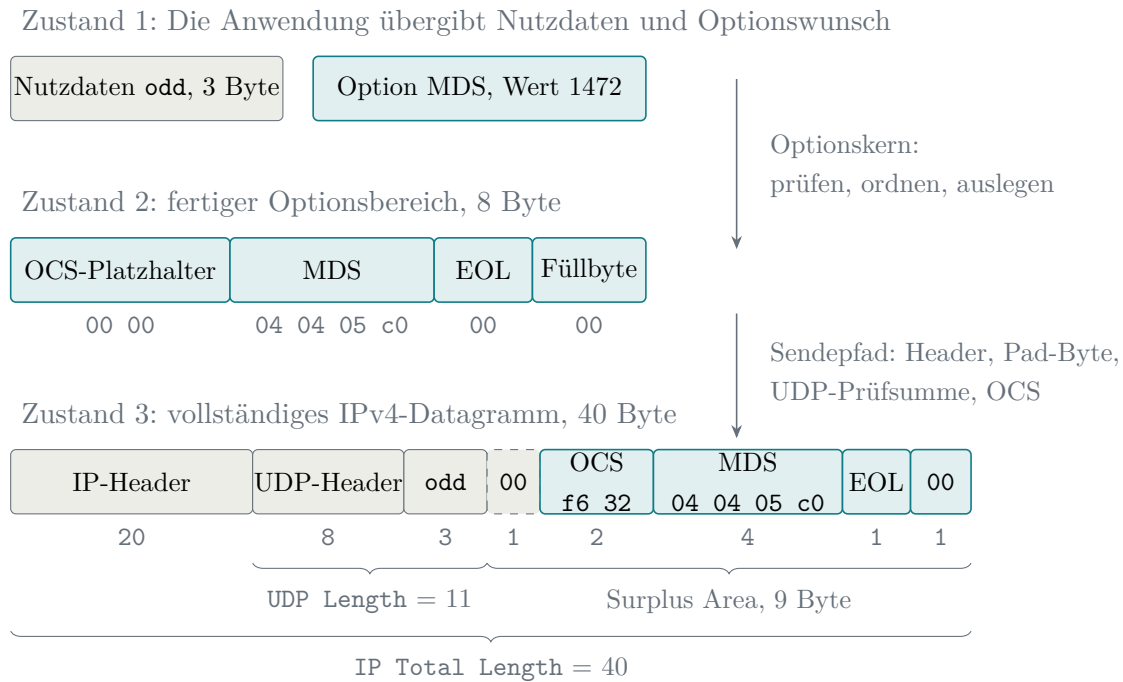


Abbildung 5.2: Entstehung des Beispieldatagramms mit MDS in drei Zuständen

Den Optionsbereich aus Zustand 2 erzeugt der gemeinsame Optionskern. Er prüft zuerst, ob die gewählte Option unterstützt wird und ihre Länge stimmt. Bei mehreren Einträgen stellt er die Pflichtoptionen außer EOL und NOP vor andere SAFE-Optionen [2, Sec. 10] und legt die Bytes danach in einer festen eigenen Reihenfolge ab. Im Beispiel entsteht so der acht Byte lange Bereich aus Zustand 2; der MDS-Wert `0x05C0` entspricht den angekündigten 1.472 Byte. Beim Empfang liest derselbe Optionskern diesen Aufbau wieder zurück.

Sobald der Optionsbereich feststeht, kann der Sendepfad die endgültigen Längen bestimmen. Die drei Byte Nutzdaten ergeben zusammen mit dem acht Byte langen UDP-Header eine `UDP Length` von 11. Hinter dem 20 Byte langen IP-Header beginnt die Surplus Area deshalb bei Byte 31. Da dieser Offset ungerade ist, fügt die Bibliothek vor dem OCS ein Nullbyte zur Ausrichtung ein (Abschnitt 2.3.2). Mit diesem Pad-Byte und dem acht Byte langen Optionsbereich wächst das IPv4-Datagramm auf 40 Byte. Die UDP-Prüfsumme schützt den UDP-Header und die Nutzdaten, aber nicht die Surplus Area.

Das OCS wird zuletzt eingesetzt, weil erst jetzt der vollständige Optionsbereich und die gesamte Surplus-Länge feststehen. Mit dem noch leeren OCS-Feld ergeben die vier Wörter des Optionsbereichs die Summe `0x09C4`. Hinzu kommt die gesamte Surplus-Länge von neun Byte, also `0x0009`; das Pad-Byte selbst gehört nicht zu den summierten Wörtern. Nach der Invertierung entsteht `0xF632`. Der Empfänger erhält mit $0x09C4 + 0xF632 + 0x0009 = 0xFFFF$ das erwartete Ergebnis

(Abschnitt 2.3.3). Ergibt eine verwendete Prüfsumme rechnerisch null, schreibt die Bibliothek stattdessen `0xFFFF`. Für UDP verlangt dies RFC 768. Beim OCS folgt es daraus, dass `0x0000` ein ungenutztes OCS kennzeichnet [1, 2, Sec. 9].

Mit `IP_HDRINCL` übergibt die Bibliothek anschließend das vollständige Datagramm an den Kernel; dieser trägt IPv4-Gesamtlänge und IP-Headerprüfsumme stets selbst ein und füllt die noch leere IPv4-Identifikation [15]. UDP-Header und Surplus Area bleiben auf diesem Weg unverändert; selbst die vom RFC erlaubte Kombination aus UDP-Prüfsumme null und OCS null gelangt so auf die Leitung (Abschnitt 6.3). Spätere Paketfilter können das Datagramm weiterhin verändern oder verwerfen (Abschnitt 2.4).

Eine weitere Grenze betrifft die Größe: Auf diesem Pfad fragmentiert der Kernel nicht, ein Datagramm über der MTU der Schnittstelle scheitert mit `EMSGSIZE` [15]. Größere Nachrichten muss die Bibliothek deshalb selbst mit FRAG zerlegen.

5.3 Empfangspfad und öffentliche Schnittstelle

Der Raw Socket erhält das IPv4-Datagramm, bevor der Kernel `UDP Length`, Prüfsumme und Ports prüft. Deshalb kontrolliert die Bibliothek diese Angaben selbst, bevor sie die Surplus Area und deren Optionen auswertet.

Als Leitbeispiel dient ein Datagramm mit den neun Byte Nutzdaten `123456789` und einer Additional Payload Checksum (APC). Die Option enthält eine 32-Bit-Prüfsumme nach dem Verfahren CRC32c über die Nutzdaten, hier `0xE3069283`. Der Sender trägt den Wert mit dem höchstwertigen Byte zuerst hinter Kind 2 und Länge 6 ein [2, Sec. 11.3].

Die APC schützt nur die Nutzdaten. Pseudo-Header und UDP-Header bleiben Aufgabe der UDP-Prüfsumme, sofern diese verwendet wird. Für die Surplus Area greifen zwei andere Prüfungen zusammen: Das Pad vor dem ausgerichteten OCS wird getrennt auf null geprüft. Die OCS-Rechnung beginnt am OCS-Feld und reicht bis zum Ende der Surplus Area; zusätzlich geht die Länge des gesamten Bereichs einschließlich Pad in die Rechnung ein. Die APC ersetzt keine dieser Prüfungen. Sie soll Nutzdatenfehler erkennen, die der UDP-Prüfsumme entgehen [2, Sec. 8, 9 und 11.3]. Abbildung 5.3 (a) zeigt die Schutzbereiche; Teil (b) zeigt die Folgen eines Fehlers.

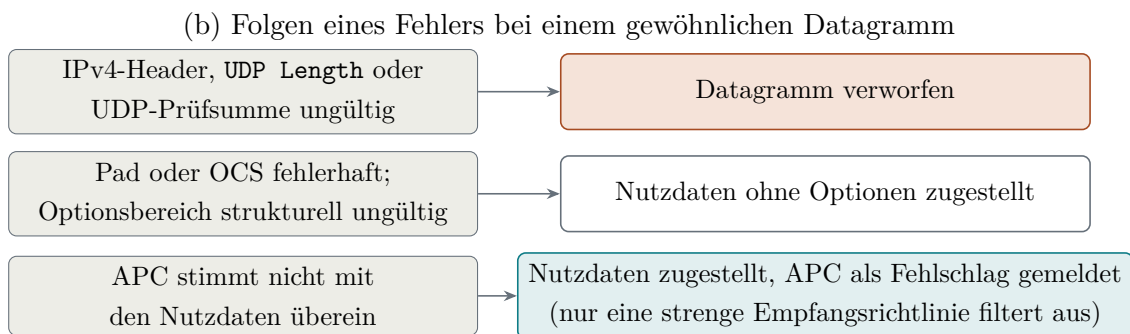
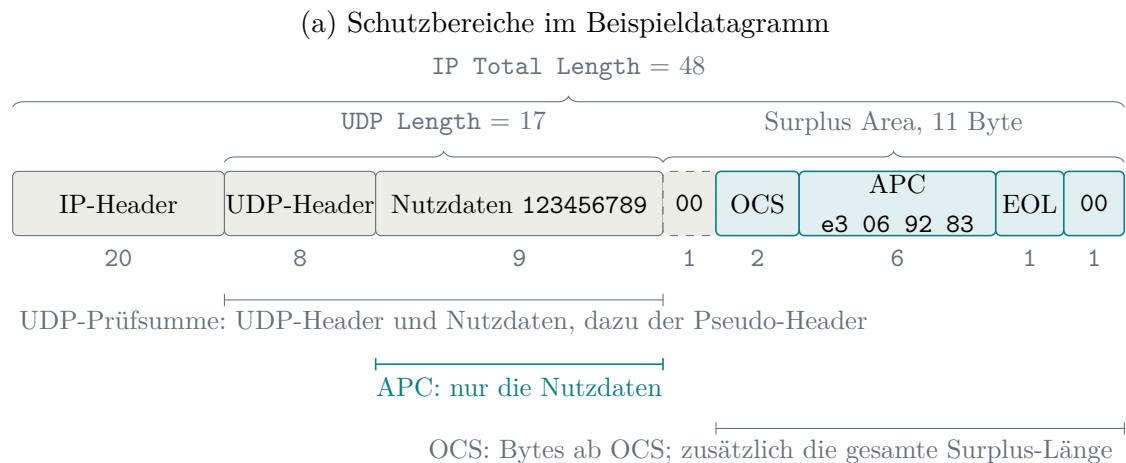


Abbildung 5.3: Schutzbereiche der APC und Folgen von Empfangsfehlern

Ein früher Vorversuch zeigte, warum die Bibliothek `UDP Length` und `UDP-Prüfsumme` selbst prüfen muss. Er enthielt vier gezielte Grenzfälle: eine `UDP-Prüfsumme` von null, eine absichtlich falsche `UDP-Prüfsumme`, eine `UDP Length` von 500 bei nur 48 verfügbaren Bytes und eine `UDP Length` von 4, die kürzer als der `UDP-Header` war. Der `Raw Socket` lieferte alle vier Fälle an den Benutzerraum.

Zuerst prüft die Bibliothek das Datagramm selbst. Der `IPv4-Header` muss gültig sein. `UDP Length` muss mindestens 8 betragen und darf die `IP-Nutzlast` nicht überschreiten. Eine `UDP-Prüfsumme` ungleich null muss stimmen; eine gespeicherte Null bleibt unter `IPv4` zulässig (Abschnitt 2.2.1). Scheitert eine dieser Prüfungen, wird das gesamte Datagramm verworfen [2, Sec. 10 und 14].

Danach sucht die Bibliothek die `Surplus Area`. Sie beginnt im Beispiel bei Byte 37 ($20 + 17$) und umfasst $48 - 37 = 11$ Byte. Der Start ist ungerade, also ist das erste Byte das `Pad` vor dem ausgerichteten `OCS`. Passt das `OCS` nicht mehr vollständig in die `Surplus Area`, sind keine Optionen vorhanden. Das ist kein Fehler: Die `Nutzdaten` werden ohne Optionen zugestellt, und der `OCS-Bericht` meldet ein fehlendes `OCS`.

Anschließend prüft die Bibliothek den `Optionsbereich`. Das `Pad-Byte` vor dem `OCS` muss null sein. Ein `OCS` ungleich null muss zusammen mit den übrigen Optionsbytes und dem `Längensummanden` aufgehen. Der Nullwert bedeutet nur dann ein zulässig

ungenutztes OCS, wenn auch die UDP-Prüfsumme null ist. Scheitert eine dieser Prüfungen, verwirft die Bibliothek bei einem gewöhnlichen Datagramm nur die Optionen und stellt die bereits geprüften Nutzdaten weiter zu [2, Sec. 8, 9 und 14].

Erst danach liest der Parser die Optionsliste. Zwei Längenfehler machen den gesamten Optionsbereich strukturell ungültig: Die angegebene Länge liegt unter dem bekannten Mindestmaß der Optionsart oder reicht über das Ende der Surplus Area hinaus. Ein zweites FRAG hat dieselbe Folge [2, Sec. 10]. Für den Überlängenfall löst Erratum 8834 den Widerspruch zu Abschnitt 14 zugunsten dieses Verfahrens auf [13].

Eine längere APC ist davon zu unterscheiden. Ihre gewöhnliche Gesamtlänge beträgt 6. Ist die angegebene Länge größer, passt aber noch vollständig in die Surplus Area, behandelt der RFC die Option wie eine falsche APC-Prüfsumme und nicht wie einen Strukturfehler [2, Sec. 11.3].

Bei einer gültig aufgebauten APC berechnet die Bibliothek die CRC32c erneut über die neun Byte Nutzdaten. Stimmen beide Werte überein, gilt die APC als erfolgreich und wird der Anwendung gemeldet. Andernfalls meldet die Bibliothek einen Fehlschlag und gibt eine gedrosselte Warnung aus. Enthält das Datagramm mehrere APC-Optionen, wertet die Bibliothek nur die erste aus und übergeht die weiteren [2, Sec. 10].

Zum Schluss entscheidet die Empfangsrichtlinie über die Zustellung. In der Voreinstellung erhält die Anwendung die Nutzdaten auch bei einer fehlgeschlagenen APC. Eine strenge Richtlinie kann dagegen eine erfolgreiche APC verlangen. Fehlt die APC oder schlägt sie fehl, liefert die Richtlinienauswertung den Ausgang **Filtered**. **Dropped** steht dagegen für ein Verwerfen in der Protokollverarbeitung, etwa bei einem gescheiterten Fragmentsatz. So bleibt intern sichtbar, ob eine Anwendungsrichtlinie oder die Verarbeitung selbst die Zustellung verhindert hat [2, Sec. 11.3, 14 und 19].

Die öffentliche Schnittstelle fasst Senden und Empfangen im Typ **Peer** zusammen. Er wird mit Adressen, Sendekonfiguration, Empfangsrichtlinie und Empfangsfrist erzeugt. Die Empfangsrichtlinie (**ReceivePolicy**) kann eine oder mehrere meldefähige Pflichtoptionen verlangen (alle außer EOL, NOP und FRAG), alle options-tragenden Datagramme ausfiltern oder ein bestätigtes OCS fordern. Die Berichtsarten **OcsReport** und **OptionReport** halten fest, welche Prüfungen die Bibliothek ausgeführt hat und wie sie endeten [2, Sec. 14 und 15].

Vor dieser Verarbeitung muss die Socket-Schicht zunächst ein passendes Datagramm finden. Ein Raw Socket sieht auch fremden UDP-Verkehr. In einer frühen Fassung beendete bereits das erste fremde Datagramm den Empfangsaufwurf ohne Ergebnis; eine laufende Messung konnte dadurch still enden. Die heutige Fassung

überspringt in einer Empfangsschleife Datagramme mit unlesbarem Header oder nicht passenden Ports, auf Anforderung auch Datagramme vom eigenen Absender.

Eine gesetzte Empfangsfrist gilt für die ganze Schleife; ohne Frist wartet sie unbegrenzt. Vor jedem weiteren Leseversuch erhält der Socket nur die verbleibende Zeit; ständiger fremder Verkehr verlängert die Wartezeit deshalb nicht. Erst wenn bis zum Ablauf der Frist kein passendes Datagramm eintrifft, meldet die Socket-Schicht einen Fristablauf. Die öffentliche Schnittstelle fasst diesen Ausgang weiterhin mit Dekodierfehlern zusammen. Dasselbe gilt für ein noch unvollständig gepuffertes Fragment, ein Verwerfen und eine Richtlinienentscheidung. Alle erscheinen dort nur als „kein Datagramm“; die Ursache ist nicht unterscheidbar. Ob diese Zusammenfassung für die Messkampagnen ausreicht, klärt Abschnitt 6.3.

5.4 Fragmentierung und weitere Pflichtoptionen

Die Pflichtoptionen umfassen acht Arten: End of Options List (EOL), No Operation (NOP), Additional Payload Checksum (APC), Fragmentation (FRAG), Maximum Datagram Size (MDS), Maximum Reassembled Datagram Size (MRDS), Echo Request (REQ) und Echo Response (RES) [2, Sec. 10]. EOL und NOP entstehen beim Optionsbau, FRAG beim Zerlegen. APC, MDS, MRDS, REQ und RES stehen als typisierte Sendeoptionen bereit und entstehen nur auf Anforderung.

TIME, AUTH und EXP (Experimentaloption) besitzen keine typisierten, sendbaren Varianten. Empfangsseitig erscheinen ihre Kind-Werte als `Other`. Nur für TIME und EXP prüft der Parser eine Mindestlänge; das ist keine vollständige Unterstützung dieser Optionen.

Mit FRAG überträgt der Sender ein Datagramm in einem oder mehreren Fragmenten, die der Empfänger wieder zusammensetzt. Dazu stehen die Fragmentdaten nicht in den UDP-Nutzdaten, sondern in der Surplus Area. Deshalb beträgt `UDP Length` in jedem Fragment 8, und jedes Fragment trägt erneut beide UDP-Ports [2, Sec. 11.4]. Die FRAG-Option ist im nichtterminalen Fragment zehn Byte und im terminalen Fragment zwölf Byte lang.

`Frag.Offset` bezeichnet die Lage der Fragmentdaten im ursprünglichen UDP-Datagramm, gemessen ab dessen UDP-Header. `Frag.Start` bezeichnet den Beginn der Fragmentdaten im aktuellen Fragment, ebenfalls ab dem UDP-Header. Er beträgt in dieser Implementierung 20 Byte im nichtterminalen und 22 Byte im terminalen Fragment. Nur das terminale Format enthält den Reassembled Datagram Option Start (RDOS). Dieser Zeiger markiert im wiederhergestellten Datagramm das Ende der Nutzdaten und den Beginn der Surplus Area. RDOS ergibt zugleich die UDP

`Length` des wiederhergestellten Datagramms; etwaige Datagrammoptionen folgen dahinter in dessen Surplus Area [2, Sec. 11.4].

MRDS begrenzt die Größe des wiederhergestellten UDP-Datagramms einschließlich UDP-Header und Datagrammoptionen. Ein zweites Feld begrenzt die Zahl der Fragmente [2, Sec. 11.6]. Ohne empfangene MRDS legt die Implementierung für IPv4 die vom RFC verlangten Mindestwerte von 2.926 Byte und zwei Fragmenten zugrunde.

Empfangene MRDS- und MDS-Werte übernimmt die Bibliothek nicht automatisch in ihre Sendekonfiguration. Der Aufrufer muss MRDS-Werte selbst in `PeerFragmentLimits` übertragen. Auch MRDS lässt sich je Sendung erzeugen.

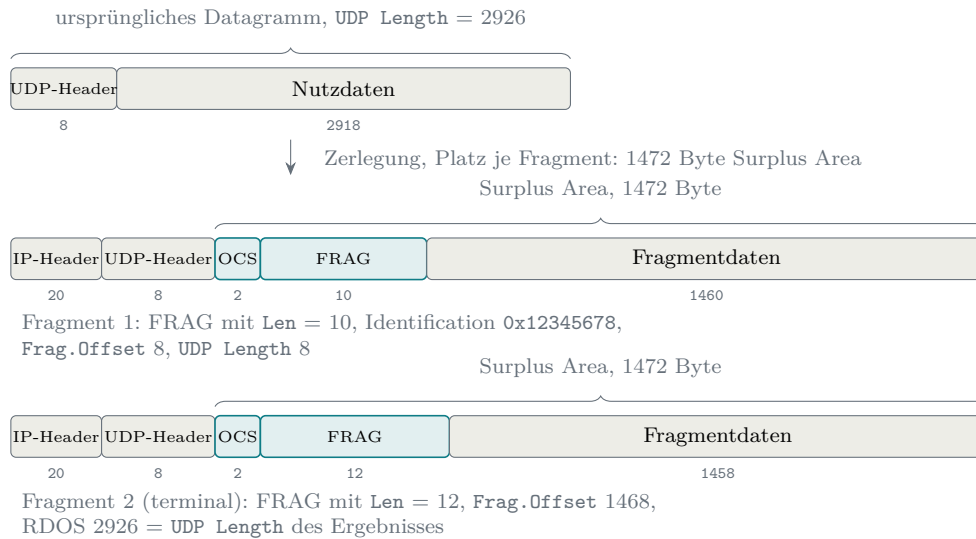
Bei einer maximalen Datagrammlänge von 1.500 Byte bleiben nach IPv4- und UDP-Header 1.472 Byte für die Surplus Area. OCS und FRAG verringern die Datenkapazität auf 1.460 Byte im nichtterminalen und 1.458 Byte im terminalen Fragment. Das Beispiel in Abbildung 5.4 zerlegt deshalb 2.918 Byte Nutzdaten in genau zwei Fragmente.

Passt das ganze Datagramm in ein einziges terminales Fragment, trägt dieses `Frag.Offset` null. Sonst erzeugt eine Schleife nichtterminale Fragmente und überlässt den Rest dem terminalen Fragment. Beim Zerlegen prüft die Bibliothek zuerst die MRDS-Größe und danach die Zahl der Fragmente. Ausgelöst wird die Zerlegung durch die Datagrammkapazität, nicht durch MRDS.

`Peer` und `udpopt-send` besitzen je einen monotonen Kennungsgenerator (Abschnitt 5.1). Er wiederholt keinen Wert. Nach `u32::MAX` meldet er einen Fehler. Die tiefere Funktion `build_outgoing_datagrams` besitzt keinen Generator und verlangt bei notwendiger Fragmentierung eine explizite Kennung.

Der Empfangscache ordnet Fragmente nach UDP-Vierertupel und Kennung [2, Sec. 11.4]. Das Ende des terminalen Fragments bestimmt das Ende des Satzes; sein RDOS liefert die `UDP Length` des Ergebnisses. Listing 5.1 zeigt die Prüfung, ob die Fragmentdaten lückenlos vorliegen.

(a) Zerlegung im Sendepfad



(b) Reassemblierung im Empfangspfad

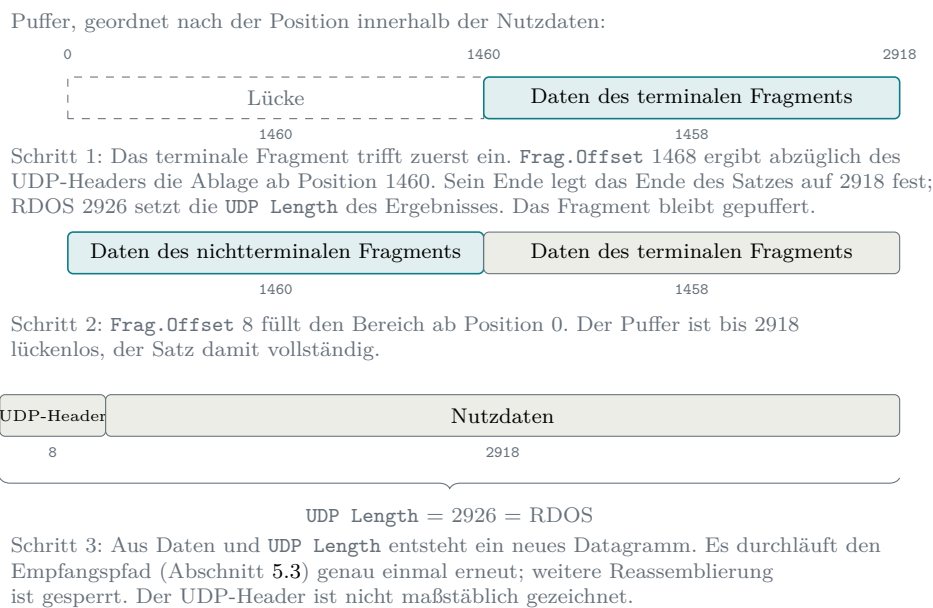


Abbildung 5.4: Zerlegung und Reassemblierung eines Datagramms mit 2.918 Byte Nutzdaten

```

1 fn complete(&self) -> InsertResult {
2     let (Some(terminal_end), Some(udp_length)) =
3         (self.terminal_end, self.udp_length)
4     else {
5         return InsertResult::Incomplete;
6     };
7
8     let mut cursor = 0;
9     for segment in &self.segments {
10         if segment.start > cursor {
11             return InsertResult::Incomplete;
12         }
13         if segment.start < cursor {
14             return InsertResult::Abort(AbortReason::Overlap);
15         }
16         cursor = segment.end;
17     }
18     if cursor != terminal_end {
19         return InsertResult::Incomplete;
20     }
21     ..
22 }

```

Listing 5.1: Lückenprüfung eines Reassemblierungssatzes

Nur ein lückenloser Bereich bis zum terminalen Ende erzeugt ein Ergebnis; eine Überlappung verwirft den Satz.

Die Bibliothek übergeht ein Duplikat nur, wenn Lage, Endwert, Daten, Fragmentoptionen und Fehlerliste übereinstimmen [2, Sec. 11.4]. Andernfalls gelten die normalen Grenz- und Überlappungsprüfungen. Den OCS-Befund der Fragmente führt die Bibliothek davon getrennt zusammen.

Die Fragmentberichte werden ebenfalls zusammengeführt. Bei MDS gilt der kleinste Wert. Bei MRDS werden Größe und Fragmentzahl getrennt minimiert. Von REQ und RES bleibt jeweils das zuletzt gesehene Token [2, Sec. 11.5 bis 11.7]. Schlägt eine Option in einem Fragment fehl, gilt sie im ganzen Satz als fehlgeschlagen [2, Sec. 15]. Das neu gebildete Datagramm durchläuft danach den Empfangspfad genau einmal erneut (Abbildung 5.4). Erst in diesem Durchlauf wird eine APC des Ursprungsdatagramms geprüft; eine APC in einem einzelnen Fragment wertet die Bibliothek nicht aus [2, Sec. 11.3].

Der Parser wahrt die Abwärtskompatibilität auch bei unbekannten Optionen. Eine unbekannte SAFE-Option übergeht er und meldet sie als übergangen. Bei einer unbekannten UNSAFE-Option darf er die Nutzdaten dagegen nicht weitergeben. Ein gewöhnliches oder bereits zusammengesetztes Datagramm wird deshalb mit leeren Nutzdaten zugestellt [2, Sec. 10, 12 und 14]. Die Sendeseite erzeugt selbst keine UNSAFE-Optionen; der Optionskern weist solche Arten schon beim Aufbau zurück.

Innerhalb eines Fragments bildet eine unbekannte UNSAFE-Option einen Sonder-

fall. Steht zuvor ein gültiges FRAG, kann die Bibliothek das Fragment zuordnen und verwirft den ganzen Fragmentsatz. Steht die UNSAFE-Option vor FRAG, ist noch keine Zuordnung möglich. Dann stellt die Bibliothek leere Nutzdaten zu und verwirft keinen gespeicherten Satz [2, Sec. 10, 12 und 14].

Für den Aufbau aller Optionen gelten gemeinsame Regeln. Die Reihenfolge legt der Optionskern fest (Abschnitt 5.2). Der interne Optionskern lässt für Tests Wiederholungen gewöhnlicher Optionen zu. `Peer` und `build_outgoing_datagrams` weisen wiederholte meldefähige Optionen dagegen ab; FRAG darf schon im Optionskern nur einmal vorkommen [2, Sec. 10].

EOL beendet die Liste, NOP richtet die folgende Option aus. Nach EOL fügt der Serialisierer höchstens ein Füllbyte für eine gerade Länge an. Die öffentliche Schnittstelle kann weder eine größere Nullfüllung noch eine Mindestlänge des Optionsbereichs anfordern. Beim Empfang prüft der Parser die Bytes hinter EOL nicht; der RFC stellt diese Prüfung frei [2, Sec. 11.1]. Im Empfangsbericht bleiben EOL, NOP und FRAG unsichtbar.

Beim erweiterten Längenformat trifft der Parser eine eigene strenge Entscheidung. Verwendet eine Option dieses Format für eine Gesamtlänge unter 255, verwirft er alle Optionen. RFC 9868 verlangt hier nur vom Sender die kürzere Darstellung und legt keine besondere Reaktion des Empfängers fest [2, Sec. 10].

REQ und RES tragen jeweils ein vier Byte langes Token. Die Bibliothek serialisiert und dekodiert beide Arten, sendet aber keine automatische Antwort. Die Anwendung muss das Token eines empfangenen REQ für ein zugehöriges RES übernehmen [2, Sec. 11.7].

5.5 Betriebs- und Sicherheitsgrenzen

Raw Sockets verlangen unter Linux `CAP_NET_RAW` oder Root [15, 42]. Dieses Privileg erlaubt den Zugriff auf rohe IPv4-Datagramme. Mit `IP_HDRINCL` kann der Prozess auch eine von null verschiedene Quelladresse vorgeben. Die Bibliothek kapselt den Zugriff, hebt seine Wirkung aber nicht auf.

Der `ReassemblyCache` ist der einzige Empfangszustand mit Protokollwirkung, der über ein Datagramm hinaus erhalten bleibt. Jeder `Peer` besitzt einen Cache, filtert eingehende Datagramme jedoch nur nach Ports. Verschiedene Adresspaare mit gleichen Ports teilen deshalb Cache und Mengengrenze. Die im Modul `frag` dokumentierte Aufruferpflicht, je UDP-Vierertupel einen eigenen Cache zu führen, erfüllt `Peer` damit nur teilweise.

Globale Warnzähler dienen nur der Diagnose. Der Parser begrenzt die Zahl der Optionen nicht. Mehr als sieben aufeinanderfolgende `NOP` erzeugen nur eine Warnung [2, Sec. 11.2]. Sie erscheint beim ersten und danach bei jedem 64. Vorkommen derselben Warnung.

`ReassemblyLimits` begrenzt Datagrammgröße, Fragmentzahl, Frist und die Zahl gleichzeitig unvollständiger Sätze. Die Voreinstellungen sind 2.926 Byte, zwei Fragmente und zwei Minuten [2, Sec. 11.4 und 11.6] sowie 64 unvollständige Sätze je Cache. Die Zahl 64 ist eine lokale Schutzgrenze; RFC 9868 empfiehlt nur eine Begrenzung [2, Sec. 11.4 und 25.4].

Die Frist beginnt mit dem ersten Fragment und wird nicht erneuert. Abgelaufene Sätze verschwinden bei der nächsten Einfügung oder bei einem Bereinigungsaufruf. Ein voller Cache weist neue unvollständige Sätze ab und verdrängt keine vorhandenen. Ein für sich vollständiges Fragment kann weiterhin zugestellt werden.

Die maximale Sendelänge gibt der Aufrufer vor; die Bibliothek ermittelt die Pfad-MTU nicht. Oberhalb der MTU meldet der Kernel `EMSGSIZE` [15]; die Bibliothek reicht den E/A-Fehler weiter (Abschnitt 5.2).

Damit sind die Referenzimplementierung und ihre Betriebsgrenzen beschrieben. Kapitel 6 prüft sie für FF1 und setzt sie für FF2 als Messmittel ein.

6. Evaluation und Diskussion

RFC 9868 berichtet in Abschnitt 18 von Interoperabilitätstests mit Linux, macOS und Windows Cygwin sowie vom erfolgreichen Durchgang durch NAT-Geräte. Der Abschnitt nennt jedoch weder den genauen Versuchsaufbau noch die verwendeten Netzpfade. Ob diese Tests im Labor oder über öffentliche Netze stattfanden, bleibt damit offen [2, Sec. 18].

Messungen auf realen Internetpfaden sind in der 2020 veröffentlichten Studie von Zullo, Jones und Fairhurst dokumentiert, auf die RFC 9868 in den Abschnitten 9 und 25.6 verweist [2, Sec. 9 und 25.6]. Die Messungen fanden im November 2018 statt [43, Sec. III und IV]; Grundlage war ein damaliger Entwurf der UDP-Optionen. Die Studie prüfte somit nicht die endgültige Fassung von RFC 9868, die erst im Oktober 2025 veröffentlicht wurde [2].

Die Bibliothek aus Kapitel 5 wird an den Kategorien aus Abschnitt 4.2 und am Pfadmodell aus Abschnitt 4.3 geprüft. Nach dem Messkonzept (Abschnitt 6.1) folgen die Pfadergebnisse (Abschnitt 6.2), der Soll-Ist-Abgleich (Abschnitt 6.3), die KI-Reflexion (Abschnitt 6.4) und die Grenzen (Abschnitt 6.5).

6.1 Test- und Messkonzept

Die Messläufe tragen unterschiedliche, vorab festgelegte Beweiskraft. Eine **Kampagne** umfasst beidseitige Mitschnitte, Sendemanifest, Auswerterlauf und ein versiegeltes Archiv; eine **vorregistrierte Messreihe** (in Tabelle 6.1 kurz Messreihe) folgt einem festen Zellplan, verzichtet aber auf einen Teil dieses Vollpakets; ein **Pilot** ist eine Funktionsprobe ohne Mitschnitt oder Manifest; eine **Vorstufe** prüft eine Richtung mit einem Datagramm je Klasse; ein **Referenzlauf** (kurz Referenz) ist eine kontrollierte lokale Messung ohne fremde Netzbeteiligung. Eine **Zelle** ist eine Messung mit fester Datagrammzahl je Zellform und Richtung außerhalb einer Kampagne; ihre Stufe folgt den Artefakten: mit beidseitigen Mitschnitten und Sendemanifest wie eine Messreihe, ohne Senderaufzeichnung wie ein Pilot. Alle Läufe ordnet Tabelle 6.1 ein:

Aus Tabelle 6.1 geht hervor, dass die Ergebnisse der vollständigen Suiten auf Kampagnen beruhen. Piloten, Vorstufen und Messreihen schließen Lücken und isolieren Mechanismen, behalten aber ihr jeweils ausgewiesenes niedrigeres Evidenzniveau. Werte aus diesen Läufen werden nicht mit Kampagnensummen verrechnet.

Tabelle 6.1: Messläufe der Arbeit nach Pfad und Einstufung

Lauf	Pfad	Umfang	Stufe	Archiv
Lokale Läufe	4 Topologien	5 Szenarien	Referenz	Impl.
Extern 10.08.	achim → mcs	IPv4, Tunnel	Kampagne	extern
Bidir 11.08.	achim ↔ mcs	9/9 normalisiert, 0/6 Rückweg	Kampagne	bidir
Piloten 12./13.08.	achim, Mac, 1blue, mcs	1blue 6/6; achim/Mac 0/13, 0/10	Pilot	piloten
Vorstufe 14.08.	mcs → 1blue	11 Szenarien, $n = 1$	Vorstufe	lokal, unversie- gelt
P0 14.08.	1blue ↔ mcs	P0 >10k	Kampagne	1blue
P1/P2 15.08.	1blue ↔ mcs	P1/P2-Suiten	Kampagne	1blue
Helsinki 15.08.	mcs, hel, 1blue	18 Szenarien je Paar	Kampagne	hel
Gate-Zellen 15.08.	drei EU-Paare	G/C und dokumentierte Kontrolle je Richtung	Messreihe	hel; lokal unversie- gelt
Hotspot 15.08.	achim ↔ mcs, 1blue	$n = 30$, ATOM $n = 10$	Messreihe	hotspots; ttl-pfad
AWS-Suiten 16.08.	aws, mcs, hel, 1blue	18 Szenarien, 3 Paare	Kampagne	aws
GCP/AWS 16.08.	gcp, aws, aws2	Piloten; Zellen $n = 20$	Pilot/Zelle	aws-us; gcp-us
AP 16.08.	6 Regionen, mcs, 1blue	Zellen $n = 20$	Messreihe	aws-ap

Alle Aussagen über Quelltext und Tests beziehen sich auf den Auswertungsstand der Implementierung; die Stände der einzelnen Läufe verzeichnet die Evidenz-Übersicht im Repository dieser Arbeit. Jedes der elf versiegelten Archive besitzt eine passende äußere SHA-256-Prüfsumme. Die inneren Manifeste sind nicht einheitlich portabel; das begrenzt die portable Innenprüfung, ist aber kein Hinweis auf beschädigte Archivdaten.¹

Die Kampagnenskripte zeichnen den Verkehr mit `tcpdump` auf; der Prüfumfang der Auswerter unterscheidet sich je nach Messreihe (Abschnitt 4.4.3). Insbesondere übernehmen die P0-, P1- und P2-Auswerter den OCS-Status aus der Empfängerangabe; sie rechnen OCS nicht unabhängig nach. Vier Eigenschaften der Mitschnittkette bestimmen den Messbetrieb: Auf der Loopback-Schnittstelle werden ausgehende Pakete nicht über Richtungsfilter erfasst, Mitschnittdateien wechseln nach dem Rechteabstieg von `tcpdump` den Eigentümer, die Schnittstelle meldet sich mit einem künstlichen Ethernet-Rahmen, und ein vom Filter gezähltes Paket ist noch kein geschriebenes Paket. Deshalb beweist vor jedem Lauf ein Aufwärmpaket die Mitschnittbereitschaft;

¹ Die Archive und ihre Sollwerte liegen im Verzeichnis `thesis/evidence/` des öffentlichen Repositories dieser Arbeit <https://github.com/ab7z/mcs-thesis-docs>; die dortige Übersicht ordnet die Läufe aus Tabelle 6.1 den Dateinamen, Quellständen und SHA-256-Sollwerten zu. Ein Hashwert belegt die Unverändertheit eines Artefakts, nicht die Wahrheit seines Inhalts (Kapitel 3).

die lokale Wire-Prüfung beendet den Mitschnitt erst nach drei unveränderten Dateigrößen, der Namespace-Lauf nach fester Wartezeit, und die öffentlichen Kampagnen senden **SIGINT**, warten eine Sekunde und prüfen, dass **tcpdump** keinen Kernelverlust meldet. Im Sendeweg gilt die Arbeitsteilung: Auf dem **IP_HDRINCL**-Pfad setzt der Kernel die IPv4-Gesamtlänge und die Headerprüfsumme und, weil die Bibliothek die IPv4-Identification null lässt, gegebenenfalls auch dieses Feld. UDP-Header und Surplus Area bleiben unverändert; deshalb sind Grenzfälle wie ein Datagramm mit UDP-Prüfsumme null und OCS null weiterhin exakt konstruierbar.

Den Prüfumfang der Realpfadkampagnen legen die Treiberskripte fest: je Hostpaar 18 Szenarien, nämlich elf P0-Szenarien für den regulären Betrieb, vier P1-Szenarien für Negativ- und Störfälle und drei P2-Szenarien für Randfälle und die Cachegrenze. Hinzu kommen 21 Options- und FRAG-Zellen des P2-Zellplans sowie S-50 als eigene vorregistrierte Cache-Reihe mit zwölf Läufen. Der Zellplan hält je Zelle fest, ob die Erwartung aus dem RFC, aus dem Implementierungsstand oder aus einer Pfadannahme stammt.² Die lokalen Referenzläufe verwenden dagegen fünf eigene Szenarien in vier Namespace-Topologien und sieben Urteilklassen. Sie liefern einen kontrollierten Nullpunkt für ausgewählte Wire-, Options- und FRAG-Fälle, bilden die Realpfadszenarien aber nicht vollständig nach.

Die externen Kampagnen vergleichen zeitnah gesendete, gleich große Kontroll- und Optionsdatagramme auf demselben gerichteten Pfad. Ein externer Kampagnenlauf ist nur auswertbar, wenn die Senderaufzeichnung den Versand und die erwartete Surplus Area belegt. Die maschinelle Auswertung vergleicht Sender- und Empfänger-aufzeichnung über Zielport und Szenario, Sequenznummern, bei leeren Nutzdaten die Klassen- und Längeninformation sowie bei FRAG die Kennung des Fragment-satzes. Abweichende Paketzahlen werden gesondert gemeldet und machen einen Lauf nur dann ungültig, wenn die Auswertung ein bestimmtes Sollergebnis erzwingt. Bei IP-fragmentierten Datagrammen überspringen die Kampagnenauswerter die Folgefragmente; der Referenzauswerter bricht ab.

Ob die Messpfade während der Kampagnen stabil blieben, prüft eine eigene Auswertung der Lebensdauerfelder aller Mitschnitte:

² Im Repository der Implementierung liegen der Treiber **scripts/pair-campaign.sh**, der Zellplan **scripts/p2-cellplan.json** und die Auswerter **scripts/p0-eval.py** bis **scripts/p2-eval.py**; die lokalen Topologien und Urteilklassen definiert **docs/evaluation.md**.

Richtung	Ankunfts-TTL (Pakete)	Router*
mcs → 1blue	TTL 53: 7345	11
1blue → mcs	TTL 53: 7198 TTL 54: 147	11 / 10
mcs → hel	TTL 51: 7206	13
hel → mcs	TTL 51: 7206	13
hel → 1blue	TTL 52: 7206	12
1blue → hel	TTL 54: 7136 TTL 53: 70	10 / 11

Gesendet: alle 43618 erfassten Egress-Pakete einheitlich TTL 64; * abgeleitet unter der Annahme von genau einem Dekrement je Weiterleitung

Abbildung 6.1: Ankunfts-TTL je Richtung über die Mitschnitte der europäischen Messpfade

Aus Abbildung 6.1 ist nicht für jede Richtung eine einheitliche Ankunfts-TTL abzulesen. Auf dem Rückweg von 1blue nach mcs kommen 7198 Pakete mit TTL 53 und 147 mit TTL 54 an; die zweite Gruppe umfasst 137 Pakete desselben Flusses in einem zusammenhängenden Zeitfenster sowie zehn Einzelpakete mit eigenen Flusskennungen; am sparsamsten ist ein zeitweises Rerouting des Messflusses, und die Einzelpakete passen zu flussabhängiger Wegewahl. Von 1blue nach Helsinki kommen 7136 Pakete mit TTL 54 und 70 mit TTL 53 an; dort liegen 40 der 70 Pakete auf Kontrollen mit jeweils eigener Flusskennung, was eine flussabhängige Aufteilung stützt, aber nur für die Messfenster gilt. Keine der Gruppen liefert einen eigenständigen Beleg für eine klassenspezifische Behandlung oder für einen allgemein stabilen Pfad; die abgeleitete Routerzahl beruht auf der Annahme aus Abschnitt 2.1, dass jede Weiterleitung den Wert um genau eins senkt. Abbildung 6.1 zählt für das Paar 1blue und mcs auch die lokale, nicht versiegelte Kreuztestreihe mit, für die Helsinki-Paare nur die Kampagnen-Mitschnitte. Zählt man allein die Kampagnen-Mitschnitte, stimmen Egress und Ingress auf allen sechs Richtungen exakt überein; mit der Kreuztestreihe fehlen auf dem Paar 1blue und mcs je Richtung genau deren 52 Negativzellen. Diese Differenz ist kein Messverlust, sondern das Verwerfen, das Abschnitt 6.2.2 gerade nachweist. Für die späteren Klassenbefunde sind deshalb die jeweiligen Kontrollpakete und Messfenster maßgeblich.

Zwei Verzichte gehören zum Konzept. Das Szenario S-24 (unterschiedliche Optionssätze je Fragment) wird über die automatische Sende-API nicht erzeugt, weil diese je logischem Datagramm einen Optionssatz führt; die aus der Verarbeitungspflicht abgeleitete Empfängerpflicht ist mit der Ersatzzelle S-24r real belegt. Das netem-

Störszenario S-51 entfiel aus vier Gründen. Jeden Mechanismus, den es zufällig anstoßen würde, belegen bereits deterministische Läufe mit exakten Sollwerten (S-41 bis S-44, S-49, S-50). Seine Verlustzelle L1 sah nur unfragmentierten Verkehr vor und hätte den FRAG-Gegenstand verfehlt. Die Verlustzuordnung ist am realen Pfad stärker belegt als unter künstlicher Störung. Und eine Störwarteschlange (netem-Qdisc) auf dem einzigen Messserver, dessen Verwaltungszugang über dieselbe Schnittstelle läuft, stand ohne eigenen Messpunkt hinter der Störstufe in keinem Verhältnis zum Nutzen. Als Folge bleibt das Ende-zu-Ende-Verhalten unter stochastisch gestörtem Pfad ungemessen; die Ergebnisse gelten für saubere, kalibrierte Pfade (Abschnitt 6.3).

Mit diesen Läufen ist das Messprogramm abgeschlossen. Zwei Grenzen begleiten alle Ergebnisabschnitte: Als Zugangsnetz wurde nur ein Mobilfunk-Hotspot-Pfad untersucht, und ohne Messpunkt im Betreibernetz endet jede Ursachenzuordnung am Intervall zwischen zwei eigenen Messpunkten. Der folgende Abschnitt wendet das Konzept auf die realen Pfade an.

6.2 Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden

FF2 fragt, in welchem Umfang die Surplus Area auf den untersuchten realen Netzpfeiden bis zur Gegenstelle unverändert erhalten bleibt (Abschnitt 1.3). Die Ergebnisse folgen den fünf Pfadstufen aus Abschnitt 4.3 in aufsteigender Netzferne.

Die Messpfade umfassten IP-Weiterleitung, Adressumsetzung und Paketfilterung. Im kontrollierten Aufbau waren diese Funktionen bekannt und gezielt konfiguriert. Auf öffentlichen Netzpfeiden ließ sich das beobachtete Verändern oder Verwerfen von Datagrammen dagegen meist keinem konkreten Gerätetyp zuordnen. Jede beobachtete Abweichung wird daher nur dem Intervall zwischen den eigenen Messpunkten zugeschrieben. Die Mechanismenklassen in Abschnitt 6.2.4 beschreiben dieses Verhalten, nicht den Gerätetyp. Kein einzelner untersuchter Pfad beantwortet FF2 vollständig; die Reichweite jedes Befunds grenzt Abschnitt 6.5 ein.

6.2.1 Kontrollierte Stufen: Loopback, eigene Topologien, Tunnel

Die ersten drei Pfadstufen liefern den Nullpunkt, an dem jeder Realpfadbefund gespiegelt wird. Ihre Umgebungen zeigt Abbildung 6.2:

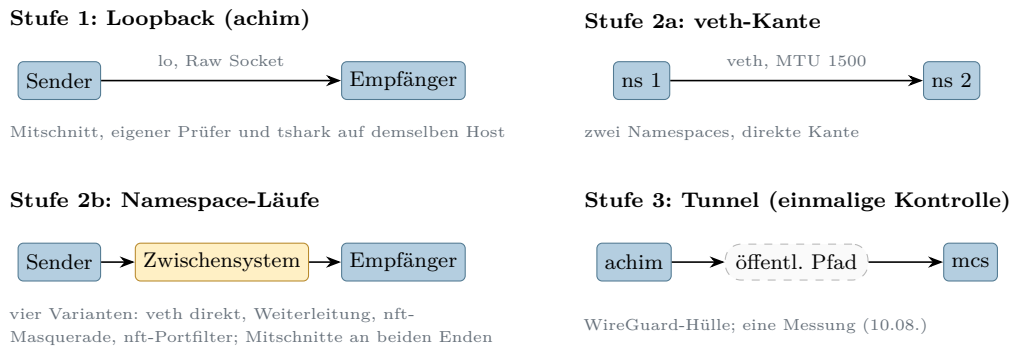


Abbildung 6.2: Kontrollierte Messumgebungen der Pfadstufen 1 bis 3

Die Stufen 1 und 2 (Abbildung 6.2) sind so gebaut, dass jede Kante entweder bekannt oder absichtlich konfiguriert ist: Ein abweichender Befund wäre dort dem Endpunkt oder der ausdrücklich eingerichteten Topologie zuzuordnen, nicht einem fremden Akteur. Die fünf lokalen Szenarien belegen die Zustellung ausgewählter gültiger Options- und FRAG-Formen sowie den vollständigen Verlust in der Filtertopologie. Die Negativzellen mit beschädigtem Pad, beschädigter OCS oder fehlerhafter TLV-Kodierung stammen dagegen aus den Realpfadkampagnen und werden durch Parser- und Pipeline-Tests ergänzt. Die Tunnelstufe steuert eine einzelne Kontrolle bei: Durch die WireGuard-Kapselung erreichte das innere Datagramm die Gegenstelle mit unveränderten 26 Surplus-Bytes und gültiger OCS; wie in Abschnitt 2.4 festgehalten, belegt das die Zustellung nach der Entkapselung, nicht den nativen Transport.

6.2.2 Öffentliche Pfade in Europa

Die vierte Pfadstufe umfasst zwei sehr verschiedene Umgebungen: das Cloud-Dreieck aus den Endpunkten mcs, helsinki und 1blue sowie den Mobilfunk-Zugangspfad des Entwicklungsrechners. Den Aufbau des Dreiecks zeigt Abbildung 6.3:

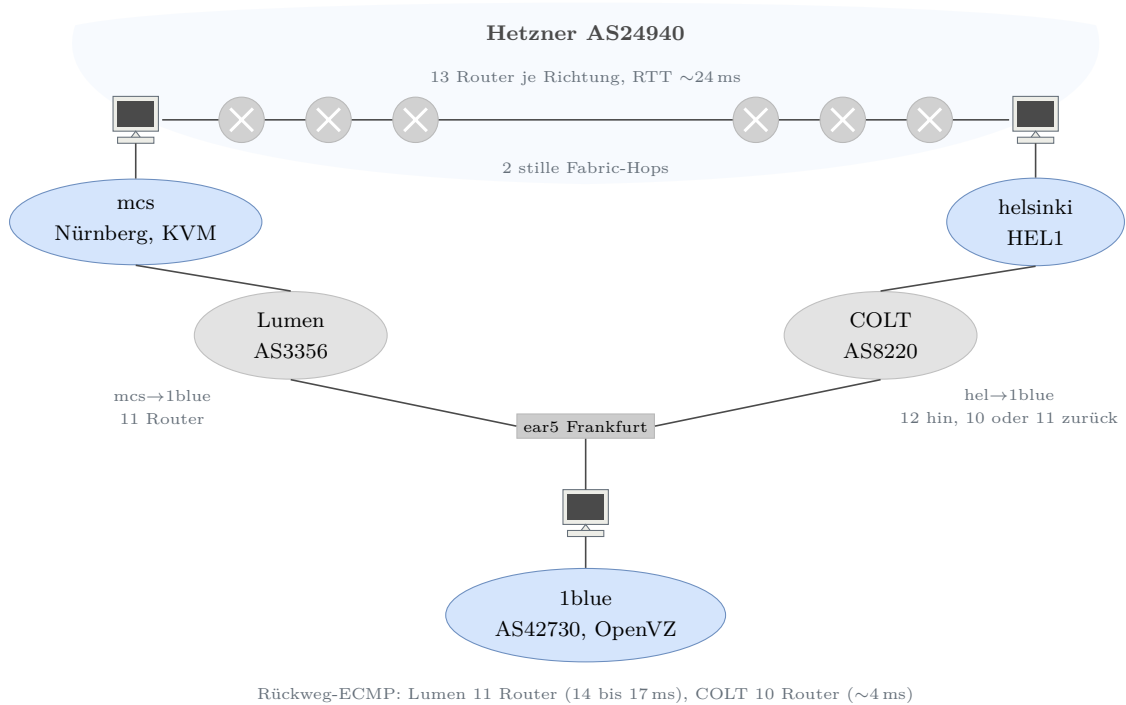


Abbildung 6.3: Europäisches Messdreieck mit zwei Endpunkt- und zwei beobachteten Transit-Netzen

Abbildung 6.3 zeigt, dass die drei Messstrecken zwei Endpunkt-Netze (AS24940, AS42730) und zwei beobachtete Transit-Netze (Lumen, COLT) berühren; die abgeleiteten Routerzahlen beruhen auf der TTL-Annahme aus Abschnitt 2.1, und zwei Weiterleitungen existieren nur als TTL-Differenz, weil sie nie antworten. Auf diesem Dreieck wurde zuerst die Verlustbasislinie geprüft. Sie stellte je Hostpaar 3000 von 3000 Paketen zu, zusammen 9000 von 9000, gesendet als Dreiergruppen aus Kontrolle, Optionsdatagramm und Kontrolle gleicher Gesamtlänge; bei 500 Paketen je Klasse liegt die einseitige obere 95-Prozent-Vertrauensgrenze der klassenweisen Verlustrate unter 0,6 Prozent je Richtung, unter der Annahme unabhängiger Einzelverluste. Der MTU-Sweep trug bis exakt 1500 Byte Gesamtlänge vollständig, und über zehntausend P0-Pakete blieben ohne unerklärten Verlust. Ferner überstanden die elf festen Prüfscenarien der Wire-Prüfung (Abschnitt 3.4.1) den Pfad von mcs nach 1blue unverändert: 15 Datagramme mit 0 bis 308 Byte Surplus, an beiden Enden aufgezeichnet und alle bestanden; als Vorstufe belegt das Pfadtransparenz, nicht die Zustellung an die Anwendung. Zusammen mit dem Piloten vom 13.08. (je Richtung sechs von sechs Optionsdatagrammen intakt) zeigt der OpenVZ-Endpunkt, dass auch dieser Netzstapel die Surplus Area nicht normalisiert. Die vollständigen Suiten erreichten anschließend auf beiden Helsinki-Paaren die erwarteten Zustellmengen (je 18 Szenarien und Richtung).

Dennoch ist das Dreieck kein neutrales Medium, und zwar in zwei Punkten:

der Wegevielfalt des Rückpfads und einem Prüfsummen-Gate. Die Wegevielfalt dokumentiert Abbildung 6.4:

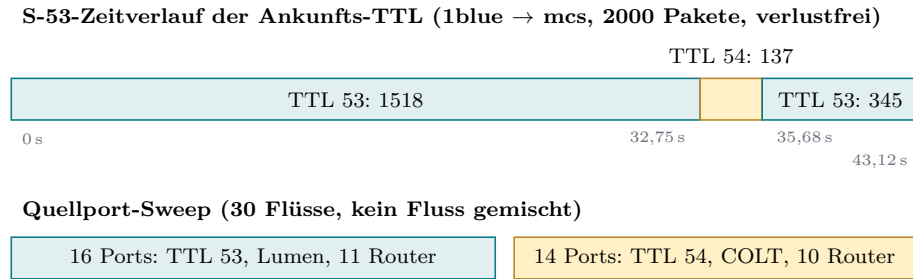


Abbildung 6.4: Reroute-Fenster im Szenario S-53 und Aufteilung des Quellportraums auf die ECMP-Zweige

Aus Abbildung 6.4 geht hervor, dass derselbe Fluss für 2,93 Sekunden verlustfrei auf eine um einen Router kürzere Variante wechselte und zurück, und dass die 1blue-Aggregation den Portraum nahezu hälftig auf einen Lumen- und einen COLT-Zweig verteilt; der Port-Sweep ist dabei eine Stichprobe des Hash-Raums, keine Lastmessung. Außerhalb des Fensters lag der feste Messfluss durchgehend auf der längeren Lumen-Variante. Ein Pfad kann seine Struktur also ohne jedes Verlustsignal ändern; jede Pfadaussage dieser Arbeit gilt deshalb nur für ihr Messfenster.

Der zweite Befund ist ein Zustellmuster, das einer fehlerhaften Prüfung der UDP-Prüfsumme über die gesamte IP-Nutzlast entspricht, im Folgenden **Prüfsummen-Gate**. Es wurde in einer lokalen, nicht versiegelten Voranalyse erkannt und durch die versiegelten Kreuztests auf weiteren Paaren repliziert. In der Voranalyse von 248 einzeln nachgerechneten Paketen stimmte die Zustellung ohne Abweichung mit folgender Bedingung überein: Das UDP-Prüfsummenfeld war null oder die Einerkomplementsumme über Pseudo-Header und gesamte IP-Nutzlast ging auf, wobei der Pseudo-Header die Länge der IP-Nutzlast trug statt des UDP-Längensums. Der entscheidende Kreuztest bestand aus zwei Zellen: Zelle G trägt ein Pad-Byte **ff** bei rechnerisch gültiger OCS; sie ist wegen des Null-Pad-Gebots aus Abschnitt 2.3.2 regelwidrig gebaut, ihre Gate-Summe, also diese Einerkomplementsumme, geht nicht auf, und sie verschwindet vollständig. Zelle C kompensiert denselben Pad-Fehler durch eine gegenkorrigierte OCS **5b53**; sie ist ebenso regelwidrig, ihre Gate-Summe geht auf, und sie kommt vollständig an. In dieser Stichprobe folgt die Zustellung damit der Gate-Summe und nicht der OCS. Algebraisch schließt sich das: Bei gültiger UDP-Prüfsumme und RFC-korrektur OCS ist die Gate-Summe gleich dem Pad-Byte, denn der Längenzuschlag des Pseudo-Headers hebt sich exakt gegen den Längensummanden der OCS-Definition auf (Abschnitt 2.3.3). Damit ist zugleich die entscheidende Anschlussfrage beantwortet, ob regelkonform gebaute Datagramme ein solches Gate passieren: Ja. Ein Datagramm mit Null-Pad und korrekter OCS

besteht ein Gate dieses Typs zwingend; das ist mit den verlustfreien P0-Läufen vereinbar, und die Messung bestätigt das in Abschnitt 9 genannte Entwurfsziel der OCS, Datagramme durch Zwischensysteme zu tragen, die den Surplus fälschlich in die UDP-Prüfsumme einbeziehen [2, Sec. 9]. Zugleich folgt eine Prüfbarkeitsgrenze für Abschnitt 6.3: Die Empfängerpflichten für verfälschte Pads und Prüfsummen sind auf solchen Pfaden nur messbar, wenn das Datagramm pfadneutral gehalten wird, entweder durch die kompensierte OCS oder durch eine genullte UDP-Prüfsumme. Beide Umgehungen sind versiegelt belegt, die kompensierte OCS als Zelle C, die genullte UDP-Prüfsumme als P1-Klasse S-05.³

Wo dieses Gate sitzt, klärt die systematische Wiederholung des Kreuztests auf allen Paaren, ergänzt um einen reinen Amazon-Vergleichspfad; Abbildung 6.5 stellt alle Richtungen zusammen:

Richtung	G: Pad ff, OCS gültig	C: OCS 5b53 kompensiert	Kontrolle
EU-Paare			
1blue → mcs	× 0/20	✓ 20/20	✓ 20/20
mcs → 1blue	× 0/20	✓ 20/20	✓ 20/20
mcs → hel	× 0/20	✓ 20/20	✓ 10/10
hel → mcs	× 0/20	✓ 20/20	✓ 10/10
hel → 1blue	× 0/20	✓ 20/20	✓ 10/10
1blue → hel	× 0/20	✓ 20/20	✓ 10/10
AWS-Paare			
aws → mcs	× 0/20	✓ 20/20	✓ 10/10
mcs → aws	× 0/20	✓ 20/20	✓ 10/10
aws → hel	× 0/20	✓ 20/20	✓ 10/10
hel → aws	× 0/20	✓ 20/20	✓ 10/10
aws → 1blue	× 0/20	✓ 20/20	✓ 10/10
1blue → aws	✓ 20/20	✓ 20/20	✓ 10/10
Amazon-Backbone			
aws → aws2	✓ 20/20	✓ 20/20	✓ 10/10
aws2 → aws	✓ 20/20	✓ 20/20	✓ 10/10

Zugestellt (✓) heißt: am fernen Ende aufgezeichnet beziehungsweise ausgeliefert; bei den zugestellten G-Datagrammen verwirft der Empfänger anschließend regelkonform die Optionen (Pad ungleich null), die Nutzdaten stellt er zu

Abbildung 6.5: Prüfsummen-Gate-Kreuzzellen je Messrichtung; die beiden Richtungen des Paares 1blue zu mcs stammen aus einer lokalen, nicht versiegelten Reihe mit abweichender Kontrollzelle

³ Die 248-Paket-Reihe ist nicht Bestandteil der elf versiegelten Evidenzarchive und diente der Hypothesenbildung. Die übertragenen Pfadaussagen stützen sich auf die späteren versiegelten Kreuztests.

Die Zelle C passiert alle 14 dargestellten Richtungen vollständig, die Zelle G nur drei; in den übrigen elf verschwindet sie vollständig (Abbildung 6.5). Zwölf dieser Richtungen sind versiegelt belegt, die beiden Richtungen des Paares 1blue zu mcs stammen aus der lokalen Reihe. Das Muster der zustellenden Richtungen trägt die Verortung: Der Amazon-Backbone lässt G beidseitig durch, ebenso der Weg von 1blue zu aws; die Ausfälle konzentrieren sich auf alle Richtungen mit Hetzner-Beteiligung und auf den 1blue-Eingang. Als sparsamstes Modell bleibt ein kanten- oder empfangsseitiges Prüfsummen-Gate bei Hetzner in beiden Richtungen und bei 1blue nur eingehend, keines bei Amazon; das ist eine Schlussfolgerung aus Endpunkt-Mitschnitten, denn zwischen Provider-Kante und angrenzendem Transitabschnitt kann ohne Messpunkt im Netz nicht unterschieden werden. Der Asien-Durchlauf replizierte beide Kantenbefunde über sechs AP-Zielregionen und ergänzte drei weitere Richtungen mit zugestellter G-Zelle, von 1blue nach Mumbai, Singapur und Osaka. In der Richtung von 1blue nach aws zeigte sich zudem das Empfängerbild einer zugestellten G-Zelle: Der Empfänger verwirft die Optionen wegen des Pad-Bytes regelkonform und stellt die Nutzdaten zu. Die kompensierte Zelle C kam in 33 gemessenen Richtungen an; in 27 davon war ein Datagramm mit regelwidrigem Pad unzustellbar, bevor ein Empfänger urteilen konnte. Dort ist die Null-Pad-Regel keine Formalie, sondern Zustellbedingung.

Die zweite europäische Umgebung ist der Zugangspfad über einen Mobilfunk-Hotspot; Abbildung 6.6 zeigt beide Betriebsarten:

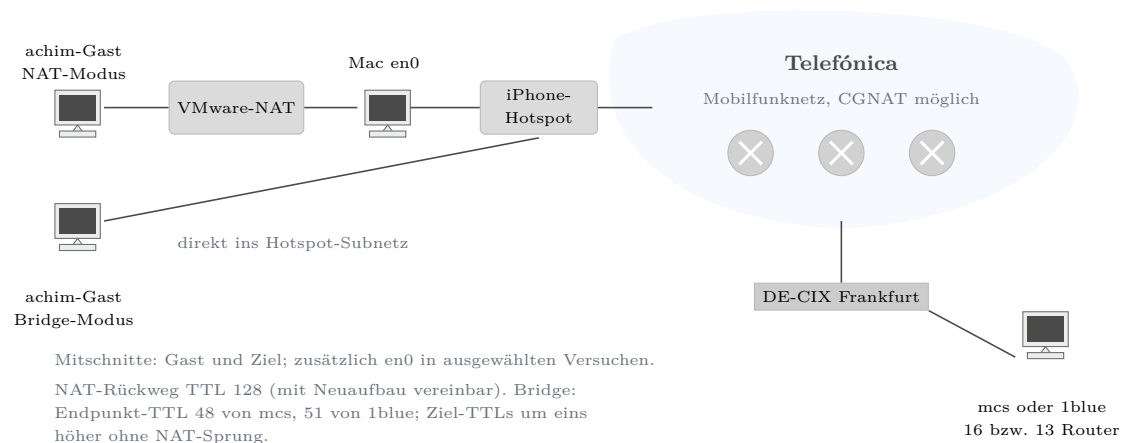


Abbildung 6.6: Zugangspfad über den Mobilfunk-Hotspot in den Betriebsarten NAT und Bridge

Auf diesem Pfad wirkten zwei Mechanismen zugleich auf die Optionsdatagramme, und beide ließen sich später trennen. Im NAT-Modus kamen abgehende Optionsdatagramme zwar an, aber verändert: neun von neun ohne ihre Surplus Area, gegen 18 von 18 intakte Kontrollen (Fisher exakt, einseitig, $p = 2,13 \cdot 10^{-7}$). Rückantworten

mit TTL 128 und neu geschriebenen IPv4-Kennungen sind mit einem Neuaufbau aus dem Verbindungszustand der NAT vereinbar (Abbildung 6.6). Direkt belegt ist allein die Normalisierung auf IPv4-Headerlänge plus UDP-Länge bei unveränderten Nutzdaten und entfernter Surplus Area; die Messpunkte trennen weder `vmnet-natd` von der macOS-VMnet-API noch legen sie den internen Mechanismus offen. Diese Beobachtung definiert die Mechanismenklasse des **Normalisierers** mit der Ergebnis-klasse entfernt. Sein deutlichster Fall (vorregistrierte Wiederholung vom 15.08.) betrifft atomare FRAG-Datagramme, deren gesamter Inhalt in der Surplus Area liegt: Sie kommen als leere 28-Byte-Hüllen an und werden zugestellt, zehn von zehn; die Anwendung erhält leere statt fehlender Datagramme, ohne jedes Verlustsignal. RFC 9868 hält es für unproblematisch, dass Altempfänger und optionsfähige Empfänger unterschiedlich viele Nulllängen-Pakete sehen; als Lebendsignal nennt er sie unzuverlässig, und Anwendungen mit UNSAFE-Optionen sollten sie nicht als Signal nutzen [2, Sec. 6, 12 und 18]. Der hier beobachtete Fall ist ein anderer: Der Inhalt wurde auf dem Pfad vernichtet. In der Rückrichtung verschwanden Optionsdatagramme dagegen vollständig: null von sechs gegen 15 von 15 gleich große Kontrollen (Fisher exakt, einseitig, $p = 1,84 \cdot 10^{-5}$); offen blieb zunächst, welches Merkmal der Klasse den Ausschlag gibt. In dieser frühen Reihe trugen Kontrollen und Optionsdatagramme zudem verschiedene DF-Werte (Kontrollen DF=1, Optionsdatagramme DF=0); die beiden p-Werte gelten deshalb für die Klasse aus Surplus-Präsenz und DF-Lage, und die Vorregistrierung der Wiederholung eliminierte diese Kovariate ausdrücklich.

Diese Frage beantwortete die vorregistrierte Wiederholung mit Kontrollzellen zu je 30 Paketen. Der Lauf wich in einem Punkt von der Vorregistrierung ab: Vorgesehen war DF=1 für alle Sendungen, die Sendermitschnitte belegen jedoch durchgehend DF=0, in beiden Richtungen und ebenso in den beiden Folgereihen. Weil Kontrollen und Proben dieselbe DF-Lage hatten, war DF in diesem Lauf kein Diskriminator. Kontrollen ohne Surplus Area passieren in beiden Größen, 58 und 44 Byte, vollständig. Gültige Optionen, ein strukturfreier Rumpf aus zwölf Bytes `fe` mit korrekter OCS und die Variante mit genullter UDP-Prüfsumme verschwinden dagegen gleichermaßen vollständig, jeweils null von 30. Jeder Vergleich ergibt im einseitigen exakten Fisher-Test $p = 8,5 \cdot 10^{-18}$. Damit diskriminiert der Verwerfer die Präsenz einer Surplus Area, also UDP-Länge kleiner als IP-Nutzlast, nicht den Optionsinhalt, die Prüfsummenlage oder die Größe. Ein Prüfsummen-Gate ist zusätzlich ausgeschlossen, weil auch die genullte UDP-Prüfsumme verworfen wird. Dieses Muster definiert die Mechanismenklasse des **Präsenz-Verwerfers**.

Die Fisher-Tests behandeln jedes Datagramm als unabhängige Ziehung; bei vollständiger Trennung hängt der p-Wert nur von den Gruppengrößen ab. Er gibt unter der Nullhypothese und bei festen Randhäufigkeiten die Wahrscheinlichkeit

eines mindestens ebenso extremen Ergebnisses im Messfenster an, nicht die Stabilität des Mechanismus. Die Datagramme eines Laufs sind Wiederholungen desselben Pfads im selben Messfenster, ihre statistische Unabhängigkeit ist nicht belegt, und die Werte werden nicht als Inferenz über eine Pfadpopulation gelesen; maßgeblich sind die vollständige Trennung der Zellen und ihre Replikation an getrennten Endpunkten. Der Fisher-Test war für die Wiederholung vorregistriert, seine Einseitigkeit nicht; für die Erstreihe sind Test und Seitigkeit nachträglich gewählt. Eine Korrektur für die sechs paarweisen Vergleiche der Wiederholung ändert an diesem Ergebnis nichts. Die Triangulation mit getauschtem fernen Ende (1blue statt mcs, anderes Endpunkt-Netz, anderer Transit, identisches Zellenbild) verortet den Mechanismus in das gemeinsame Segment aus Hotspot-Anschluss und Telefónica-Mobilfunknetz. Der Bridge-Betrieb schließlich nahm die VMware-NAT und damit den Normalisierer aus dem Pfad; nun verwarf der Pfad die Surplus-Datagramme auch im Uplink vollständig (null von 30 je Zelle, atomare Zelle null von zehn, an beiden Zielen; die um genau eins erhöhten Ziel-TTLs belegen den entfallenen NAT-Sprung). Der Zugangspfad verwirft die Klasse also in beiden Richtungen; im NAT-Modus hatte die Normalisierung den Uplink-Verwerfer verdeckt, weil sie jedes Datagramm surplus-frei machte, bevor es den Carrier erreichte. Was ohne Messpunkt im Telefon oder Betreibernetz offen bleibt, ist die Trennung zwischen Hotspot-NAT und Mobilfunknetz; jede engere Zuschreibung wäre eine Behauptung ohne Beleg.

6.2.3 Interkontinentale Pfade

Die fünfte Pfadstufe erweitert die Messbasis um zwei Fabrics großer Cloud-Anbieter und lange Transitzketten; Abbildung 6.7 ordnet die Endpunkte:

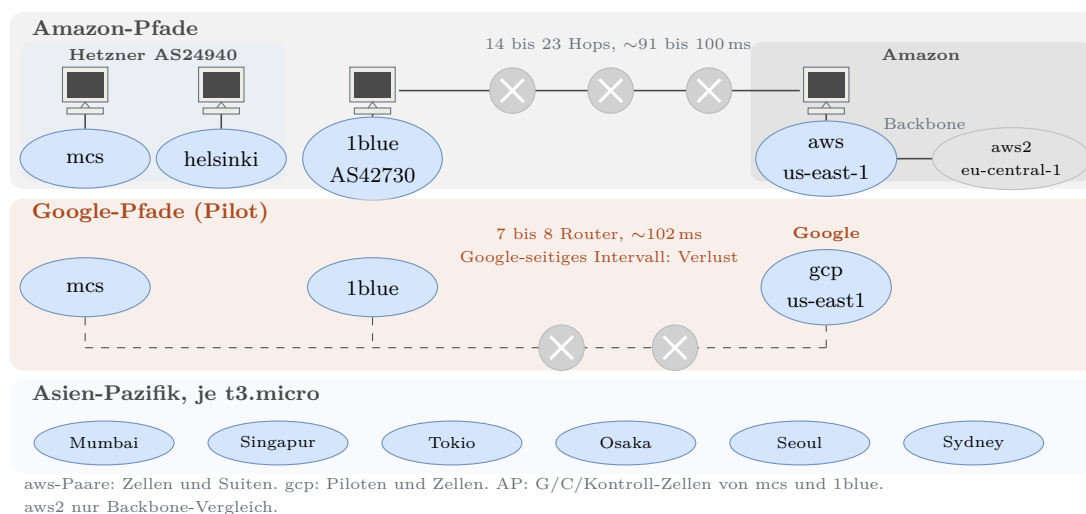


Abbildung 6.7: Interkontinentale Messendpunkte mit Belegstufe und Rundlaufzeiten

Die beiden untersuchten Anbieterumgebungen zeigten in ihren Messfenstern

gegensätzliche Ergebnisse, allerdings auf verschiedenen Evidenzstufen: Amazon mit Kampagnen und Zellenmatrix, Google mit Piloten und Zellen ohne Sendemanifest. Auf den Amazon-Pfaden aus Anbieter-Fabric, Adressumsetzung am Internet-Gateway und Transatlantik-Transit kamen in den sechs Pilotläufen 18 von 18 Kontrollen und 36 von 36 Optionsdatagramme unversehrt an; die anschließende Zellenmatrix mit sechs Zellformen bestätigte das mit 240 von 240 Paketen. Die erfolgreiche UDP-Prüfsummenprüfung nach der Adressumsetzung belegt deren Anpassung, während die übereinstimmenden Optionsbytes und die gültige OCS getrennt davon stützen, dass die Surplus Area unverändert blieb. Auch die vollständigen Suiten erreichten auf allen drei aws-Paaren die erwarteten funktionalen Zustellmengen einschließlich der MTU-Kante bei exakt 1500 Byte. Damit belegt die Stichprobe zweierlei: Ein Präsenz-Verwerfer ist keine notwendige Eigenschaft aller Umgebungen großer Cloud-Anbieter, und die drei untersuchten EU-US-Paarpfade transportierten die geprüften Szenarien beidseitig mit den erwarteten Ergebnissen. Bei einem Lauf des Szenarios S-50 auf dem Pfad von aws nach 1blue lagen 32 der 64 erwarteten Terminalfragmente außerhalb des vorregistrierten 30-Sekunden-Fensters. In den sechs Läufen dieser Richtung kamen insgesamt 966 von 966 Fragmenten an, und im betroffenen Lauf wurden alle 64 erwarteten Sätze vor dem 60-Sekunden-Cachetimeout zugestellt; der funktionale Cachebefund blieb also erhalten, der Lauf war dennoch nicht abweichungsfrei.

Im Google-seitigen Messintervall verschwand die Klasse in beiden Richtungen und unabhängig vom Inhalt. Die archivierten Empfänger-JSONs belegen 40 zugestellte Kontrollen, keine Surplus-Ankunft und keine leere Hülle. Der beabsichtigte Nenner von 200 Surplus-Sendungen folgt aus Messskript und Bericht; ein Sendemanifest für die Matrix und ein Sender-PCAP fehlen im Archiv. Die Mitschnitte an beiden Gast-Schnittstellen grenzen den Verlust auf das Intervall nach der sendenden und vor der empfangenden Gast-Schnittstelle ein. Der gleiche Pilotbefund gegen mcs und 1blue macht ein gemeinsames Google-seitiges Element zum sparsamsten Modell. Die konkrete Komponente ist ohne Zwischenmesspunkt nicht nachgewiesen. Damit ist ein **Präsenz-Verwerfer** im Google-seitigen Messintervall auf Pilotniveau belegt, nicht als nachgewiesene Eigenschaft einer bestimmten Cloud-Fabric.

Auf den Hinwegen nach Asien-Pazifik zeigte sich dieselbe Mechanismenklasse ein drittes Mal, nun quell- und zielabhängig auf interkontinentalen Paarpfaden; Abbildung 6.8 fasst die Lage zusammen:

Quelle	Mumbai	Singapur	Tokio	Osaka	Seoul	Sydney
mcs	✓	✓	×	✓	×	✓
hel (Pilot)	·	·	×	·	×	·
1blue	✓	✓	×	✓	×	×
aws-us (Pilot)	·	·	✓	✓	✓	✓

✓ Surplus-Klasse zugestellt; × Surplus-Klasse verworfen (Kontrollen kamen in allen Richtungen vollständig an); · nicht gemessen; dünner Rahmen: Pilotbeleg ohne Mitschnitt ($n = 6$ je Richtung), sonst Zellenbeleg mit Mitschnitten ($n = 20$ je Zelle und Richtung)

Abbildung 6.8: Verbleib der Surplus-Klasse auf den Hinwegen nach Asien-Pazifik je Quelle

Abbildung 6.8 zeigt, dass von der Quelle abhängt, ob die Surplus-Klasse ankommt. In den drei nachgeschalteten Zellenmatrizen kamen von mcs nach Tokio und Seoul sowie von 1blue nach Sydney jeweils alle Kontrollen an, aber keine der fünf Surplus-Zellen; die jeweilige Gegenrichtung blieb mit 120 von 120 Paketen sauber, zusammen also mit 360 von 360 Paketen. Von 1blue nach Tokio und Seoul zeigten bereits die Zellenläufe dasselbe Bild: alle Kontrollen kamen an, keine der Sendungen der Zellen G und C; Eskalationsmatrizen liefen für diese beiden Richtungen nicht. Die Helsinki-Piloten nach Tokio und Seoul lieferten je drei Kontrollen, aber keines der je sechs Optionsdatagramme, während die Amazon-Piloten nach Tokio, Seoul, Sydney und Osaka je drei Kontrollen und sechs Optionsdatagramme vollständig zustellten. Zwei Gegenproben stützen die Deutung: Osaka teilt mit Tokio die sichtbare AS-Kette und bleibt dennoch transparent, sodass die sichtbare Kette allein den Unterschied nicht erklärt, das Attributionsintervall mit Endpunktmitschnitten aber auch nicht weiter zu trennen ist; und die Sydney-Anomalie blieb im zeitversetzten Gegentest quellabhängig. Interkontinentale Erreichbarkeit der Surplus Area ist damit in den beobachteten Messfenstern paarabhängig.

6.2.4 Mechanismenklassen und Folgen

Aus den Vorversuchen und Messungen sind drei wiederkehrende Mechanismenklassen entstanden. Abbildung 6.9 zerlegt einen Messpfad in Kanten und ordnet die Klassen den beobachteten Attributionsintervallen zu (Abschnitt 4.3); die Zuordnung ist kein Gerätenachweis, und keine Klasse ist an die gezeigten Kanten gebunden, denn Kanten können Verkehr quellabhängig behandeln:

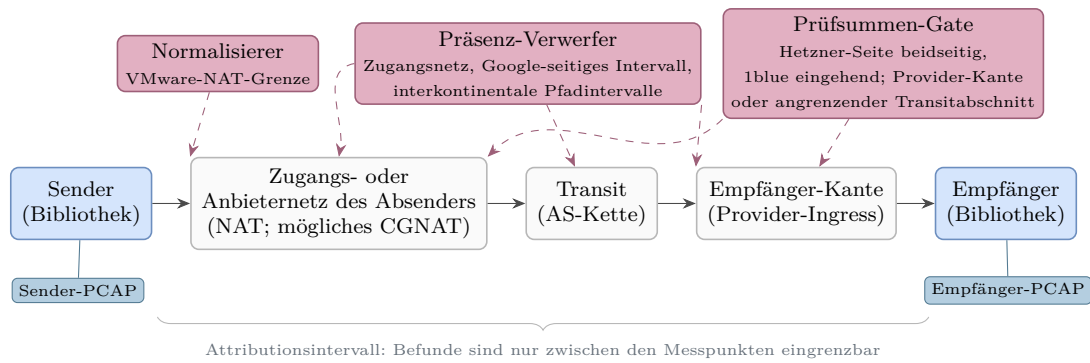


Abbildung 6.9: Kantenmodell eines Messpfads mit den beobachteten Attributionsintervallen der drei Mechanismenklassen

Abschnitt 25.6 des RFC beschreibt sowohl das Entfernen von Optionen durch UDP-Vermittler als auch das Verwerfen optionsbehafteter Datagramme auf bestimmten Pfaden [2, Sec. 25.6]. Dazu passen der **Normalisierer** an der VMware-NAT-Grenze (entfernt), das **Prüfsummen-Gate** in den Richtungen mit Hetzner-Beteiligung und am 1blue-Eingang (verworfen, wenn die Gate-Summe nicht aufgeht) und der **Präsenz-Verwerfer** im Mobilfunkzugang, in der Google-Endpunktumgebung und auf quell- und zielabhängigen interkontinentalen Paarpfaden (Verwerfen der gesamten Surplus-Klasse); jede Verortung gilt nur für ihr Attributionsintervall (Abschnitt 6.2.2). Die Amazon-seitigen Intervalle und der reine Amazon-Backbone zeigten kein Gate; die vollständigen Paarpfade mit Hetzner oder 1blue sind dennoch nicht gatefrei, denn das Gate liegt auf der jeweils entfernten Seite oder dem angrenzenden Transitabschnitt. Dazu tritt der Maskierungseffekt: Ein Normalisierer vor einem Präsenz-Verwerfer entfernt dessen Auslöser und macht ihn unsichtbar. Was das für ein einzelnes Datagramm bedeutet, bündelt Abbildung 6.10 an einem Datagramm mit den Nutzdaten odd wie im Leitbeispiel aus Kapitel 5, hier mit REQ statt MDS (42 statt 40 Byte), und seinen zwei härtesten Gegenstücken:

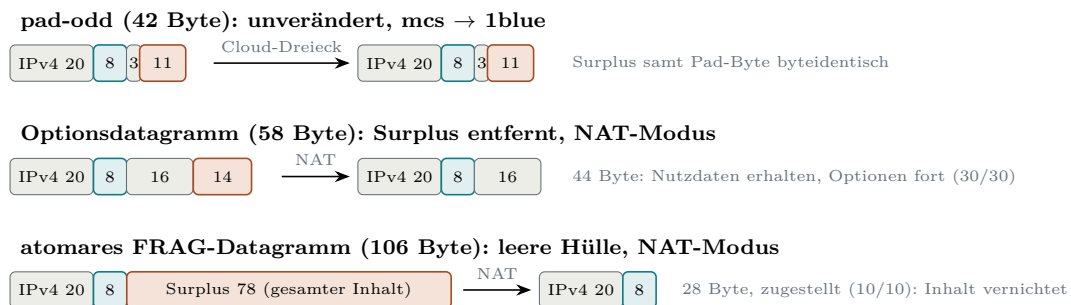


Abbildung 6.10: Drei beobachtete Ergebnisse für Beispieldatagramme auf realen Pfaden

Aus Abbildung 6.10 geht hervor, dass zwischen Erhalt und Verlust eine dritte, schwer erkennbare Form liegt: das zugestellte leere Datagramm, bei dem der Inhalt ohne Verlustsignal verschwindet. Die Einzelbefunde je Pfad fallen unterschiedlich aus;

sie tragen Abschnitt 6.2.2 und Abschnitt 6.2.3. FF2 wird deshalb als Typologie mit Einzelbefunden je Paar und Richtung beantwortet und nicht als Gesamtquote: Ohne vorher festgelegte Äquivalenzregel für Kampagnen, Messreihen, Zellen und Piloten ließe sich keine über die Stufen hinweg gültige Rate bilden. Ob die Anforderungen des Standards erfüllbar bleiben, bewertet der folgende Soll-Ist-Abgleich.

6.3 Soll-Ist-Abgleich mit RFC 9868

FF1 fragt, welche Anforderungen von RFC 9868 sich unter Linux und IPv4 mit einer Raw-Socket-Implementierung im Benutzerraum vollständig, teilweise oder nicht erfüllen lassen (Abschnitt 1.3). Bewertungsgrundlage sind die 67 Zeilen der Konformitätsmatrix aus Abschnitt 4.2, ein Arbeitsindex der Implementierung und kein Verzeichnis aller Pflichten des Standards; einzelne Zeilen bündeln mehrere Normsätze, Zeile 224 etwa eine Transit- und eine Endpunktpflicht, und werden nur mit ihrem FF1-Anteil bewertet. Die Bewertungsregeln aus Kapitel 4 und Kapitel 3 gelten unverändert, einschließlich unmarkierter normativer Sätze und der sichtbaren, aber unbewerteten nicht gewählten MAY-Funktionen. Die FR- und NFR-Kennungen dienen als Nachweisindex und nicht als zweiter Nenner; 58 der 67 Matrixzeilen tragen im eingefrorenen Arbeitsindex mindestens eine solche Kennung. Je bewerteter Zeile folgt der Nachweis einer festen Reihenfolge: Notiz der Matrixzeile, verknüpfte Anforderungskennung, Quelltext und Test, zuletzt, wo vorhanden, das Realpfadszenario. Damit ist auch die Frage aus Kapitel 5 zu beantworten, ob die zusammengefasste Meldung „kein Datagramm“ für die Messkampagnen ausreicht: allein nicht, denn nur bei den vollständigen Kampagnen stützt sich jeder Realpfadnachweis auf Mitschnitte, Sendemanifest und Auswerterlauf (Abschnitt 6.1). Ein Ausbleiben, etwa beim Reassemblierungs-Timeout ohne eigene Verwerfmeldung, ist nur dort als Kampagnenbefund belastbar, wo der Beleg für dieses Ausbleiben vollständig vorliegt.

Die am Primärtext korrigierten Bewertungen aller 67 Zeilen stehen in der in Abschnitt 4.2 verlinkten Einzelzuordnung. Tabelle 6.2 fasst die Summen zusammen.

Tabelle 6.2: Summen der geprüften Einzelzuordnung

Größe	Anzahl
Matrixzeilen insgesamt	67
anwendbar oder gewählt	57
technisch vollständig erfüllbar	57
technisch teilweise oder nicht erfüllbar	0
vollständig umgesetzt	50
teilweise umgesetzt	7
nicht anwendbar, nicht gewählt oder Leitlinie	10

Der Implementierungsstand bewertet den Endpunkt aus Bibliothek und aufrufender Schicht. Eine Zeile gilt als vollständig umgesetzt, wenn die Bibliothek alles leistet, was RFC 9868 der Optionsschicht zuweist. Weist der Primärtext eine Pflicht ausdrücklich der aufrufenden Schicht zu, genügt ein in der öffentlichen Schnittstelle dokumentierter Aufruervertrag. Weist er sie der umsetzenden Schicht selbst zu, zählt ein Vertrag ohne Durchsetzung nur als teilweise. Nach dieser Regel zählt Zeile 204 als vollständig. Abschnitt 11.7 stellt REQ und RES unter die Kontrolle der oberen Schicht: Die Optionsschicht antwortet nie selbsttätig mit einer RES, sondern liefert das REQ an die obere Schicht aus [2, Sec. 11.7]. Da die Bibliothek nie selbst eine RES erzeugt, trifft die Token-Pflicht des Abschnitts stets die aufrufende Schicht. Die öffentliche Schnittstelle dokumentiert die Herkunft des RES-Tokens aus einem zuvor empfangenen REQ als Aufruervertrag, den die Bibliothek nicht selbst erzwingt. Zeile 199 zählt dagegen als teilweise, weil Abschnitt 11.4 die Speichergrenze an die reassemblierende Schicht selbst richtet. Bei rein bibliotheksbezogener Bewertung ohne diese Unterscheidung lauteten die Summen 49 vollständig und acht teilweise.

Die technische Erfüllbarkeit ist getrennt bewertet: Für keine der 57 anwendbaren oder gewählten Zeilen findet die Einzelprüfung eine technische Grenze des gewählten Rahmens (57-mal vollständig, zehnmal nicht anwendbar). Davon getrennt zählt der Implementierungsstand 50 vollständig und sieben teilweise umgesetzte Zeilen. Neben Zeile 199 weichen auch die Zeilen 189, 204, 216, 221, 223 und 226 von der Selbstauskunft des Arbeitsindex ab; die in Abschnitt 4.2 verlinkte Einzelzuordnung führt Selbstauskunft und Urteil je Zeile nebeneinander. Kennungen und Einzelbegründungen folgen in den beiden Detailabsätzen.

Eine leere Kategorie „nicht erfüllbar“ besagt nur, dass im gewählten Prüfraster kein grundsätzlicher technischer Hinderungsgrund gefunden wurde: Die harten Gren-

zen des Benutzerraum-Zugangs treffen den Betrieb von Sende- und Empfangsweg (Abschnitt 5.5), aber keine einzelne Normaussage der Matrix. Sie belegt weder volle RFC-Konformität noch Fehlerfreiheit und gilt nur für Linux, IPv4, Benutzerraum und den gewählten Funktionsumfang.

Davon getrennt bleibt der Implementierungsnachweis des geprüften Protokollkerns belastbar: Ortung und Ausrichtung der Surplus Area, OCS-Arithmetik samt Nullregeln, TLV-Rahmung, die acht Pflichtoptionen und die FRAG-Verarbeitung einschließlich Reassemblierung, gestützt auf die Prüfmittel aus Abschnitt 3.4.1.

Die P1-Suiten der Realpfadkampagnen (Abschnitt 6.1) ergänzen 15 Empfängerklassen. Die archivierten P1-Rohdaten zeigen in jeder Klasse sechs gleichförmige Zustellungen ohne Senderfehler; die Grenzen der Auswerter, die diese Rohdatenprüfung nötig machten, nennt Abschnitt 6.5. Zu den durch Rohdaten und Auswerter gemeinsam bestätigten Erwartungen gehören das Verwerfen des Optionsbereichs bei Pad- und OCS-Fehlern, das stille Übergehen unbekannter SAFE-Optionen und das Verwerfen der reassemblierten Nutzdaten bei einer nicht unterstützten oder außerhalb eines Fragmentkontexts auftretenden UNSAFE-Option. Diese Klassen enthalten auch Kontrollen und bilden nicht alle berührten RFC-Anforderungen ab; insbesondere verarbeitet die Bibliothek den überlangen NOP-Lauf trotz Überschreiten der lokalen Meldeschwelle weiter. Spätere Robustheitsszenarien ergänzten EOL-Füllbytes, NOP-Läufe, erweiterte Längenfelder, ein atomares FRAG-Einzelfragment, parallele Fragmentsätze und ungleiche Optionssätze je Fragment.

Sieben Matrixzeilen sind **teilweise** umgesetzt: 199, 216, 218, 219, 221, 222 und 225. Zeile 199 korrigiert die damalige Repository-Selbstauskunft **Covered yes**: Das Modul **frag** verlangt einen Cache je UDP-Vierertupel als dokumentierte Aufruferpflicht. Der Typ **Peer** filtert jedoch nur die Ports. Daher können vollständige Datagramme anderer Adresspaare zugestellt werden und teilen dieselbe Mengengrenze (Abschnitt 5.5); der Adressanteil des **FragKey** verhindert nur die Vermischung von Fragmenten verschiedener Adresspaare. Bei den Zeilen 216 und 221 liefert die öffentliche API für ignorierte oder fehlgeschlagene Optionen Kind und Status, aber nicht immer die empfangenen Parameter. Die Empfangs-API prüft Pflichtoptionen zudem gebündelt je Datagramm oder Fragmentsatz und bietet weder getrennte Regeln je Fragment noch eine Ausschlussliste einzelner Optionsarten (Zeilen 218 und 219). Der Sende-API fehlen getrennte Optionslisten je Fragment und eine vom Aufrufer gewählte Mindestlänge (Zeile 222). Zeile 225 fasst teilweise umgesetzte Schutzgrenzen zusammen. Das sind Implementierungs- und Entwurfsentscheidungen, keine technischen Grenzen von Linux, IPv4 oder Raw Sockets.

Die NOP-Regel (Zeile 189) gilt nach der Primärtextprüfung dagegen als vollstän-

dig erfüllt. Sender sollten weder mehr als sieben aufeinanderfolgende NOP verwenden noch NOP als Ersatz für EOL mit Nullfüllung einsetzen; Empfänger sollten dauerhaft auftretende Läufe von mehr als sieben aufeinanderfolgenden NOP zumindest gelegentlich melden. Beide Aussagen tragen die Normstufe **SHOULD**: Eine Abweichung ist zulässig, verlangt aber eine dokumentierte Abwägung (Kapitel 3); einen Abbruch der Verarbeitung schreibt Abschnitt 11.2 weder vor noch verbietet er ihn. Die Bibliothek begrenzt die Sendeseite, erkennt und meldet überlange Empfangsläufe und verarbeitet weiter. Die bedingte Empfehlung zur Ressourcenbegrenzung steht in Abschnitt 25.2 und ist dort als Zeile 225 geführt, sodass dieselbe Lücke nicht zweimal zählt [2, Sec. 11.2 und 25.2].

Außerhalb der Matrix treten die erwarteten Betriebsgrenzen hinzu: Je unfragmentiertem IP-Datagramm trägt der Sendeweg bis zur MTU der ausgehenden Schnittstelle (auf allen Messendpunkten 1500 Byte, auf den Amazon- und Google-Instanzen ausdrücklich so eingestellt)⁴ und scheitert darüber am Kernel, weil `IP_HDRINCL` den Sendepuffer nicht fragmentiert. In der Voreinstellung teilt die Bibliothek größere logische Datagramme per FRAG; die markierte Pflicht aus Abschnitt 11.4, mindestens zwei je in 1500 Byte passende Fragmente zu bilden und zusammenzusetzen, bleibt davon unberührt (Matrixzeile 194) [2, Sec. 11.4]. Der Sweep S-35 schaltete die Fragmentierung bewusst ab: Je Richtung wurden elf Längensstufen bis 1500 Byte geprüft, mit je drei Kontroll- und drei Optionsdatagrammen, also 66 von 66 Zustellungen; in den fünf Längensstufen ab 1504 Byte scheiterten je Richtung alle 30 Sendeversuche, wobei die Berichte `EMSGSIZE` als Ursache ausweisen und die Kampagnenprotokolle nur den Rückgabewert festhalten. Dieses Bild wiederholte sich auf allen sechs Hostpaaren einschließlich der transatlantischen. Eine Pfad-MTU-Ermittlung fehlt. Die Bibliothek setzt den geprüften Protokollkern damit um, bildet aber nicht alle bewerteten API-Anforderungen vollständig ab.

Nicht anwendbar oder leitend sind zehn Zeilen: 188, 206, 207, 208, 209, 211, 223, 226, 227 und 228. Dazu gehören die nicht gewählte Empfangsprüfung der Bytes nach EOL, die nicht implementierten Optionen TIME, EXP und UEXP, die reservierten Arten AUTH, UCMP und UENC, die Entwurfs- und Namensregeln für neue Optionsarten, die Multicast-Betrachtungen sowie die API-Leitlinie zu Optionsreihenfolge und Fragmentgrenzen; letztere bleibt Leitlinie, weil der Primärtext sie ausdrücklich als nicht beabsichtigte Nutzersteuerung formuliert [2, Sec. 15]. Zeile 226 betrifft die bedingte Empfehlung aus Abschnitt 25.2, empfangene Optionen in einer von der Paketreihenfolge unabhängigen Ordnung zurückzugeben [2, Sec. 25.2]. Sie richtet sich an Implementierungen mit Bedenken wegen verdeckter Kanäle. Diese Arbeit verfolgt

⁴ Für die Google-Messung wurde ein eigenes virtuelles Netz (VPC) mit 1500 Byte MTU statt des Standardwerts von 1460 Byte angelegt.

kein solches Schutzziel (Abschnitt 1.4); die Empfangsausgabe der Bibliothek folgt bei gewöhnlichen Datagrammen der Paketreihenfolge. Die unmarkierte Sende-API-Form ist dagegen anwendbar und teilweise umgesetzt (Zeile 222). Eine formgültige TIME-Option lief auf dem Realpfad als unbekannte SAFE-Option still mit; das belegt das stille Übergehen einer unbekannten SAFE-Option, nicht die Umsetzung von TIME, und ändert die Umfangseinstufung nicht.

Vier **Differenzen zwischen Spezifikation, Modell, Programm und Laufzeit** verdienen eigene Sätze. Erstens ist die Kombination aus UDP-Prüfsumme null und OCS null aus dem Benutzerraum exakt konstruierbar und wurde auf der Leitung nachgewiesen; in der Matrix aus UDP-Prüfsumme und OCS wird diese Zelle regelkonform verarbeitet, wobei die Nichtnull-OCS nach Abschnitt 9 die Pflichtvoreinstellung bleibt und ihr Nullwert nur bei genullter UDP-Prüfsumme zulässig ist [2, Sec. 9]. Das geprüfte Datagramm trug weder eine APC noch den Integritätsschutz einer anderen Schicht, sodass seine UDP-Nutzdaten und seine Surplus Area ungeschützt blieben. Eine APC kann allgemein die UDP-Nutzdaten zusätzlich schützen, aber weder die Surplus Area noch UDP-Header und Pseudo-Header [2, Sec. 11.3]. Zweitens löst Erratum 8834 den Widerspruch zwischen den Abschnitten 10 und 14 zugunsten des Sec.-10-Verfahrens auf (Kapitel 3); die Laufzeit folgt dem nachweislich, denn im realen Überlängenfall wurde ein gültiges REQ vor der defekten Option nicht gemeldet, sondern der gesamte Optionsbereich verworfen. Drittens behandelt der Parser ein erweitert kodierte Längelfeld unter 255 strenger, als der RFC dem Empfänger vorschreibt; das ist als lokale Politik dokumentiert und in der Einzelzuordnung keine Abweichung, sondern eine ausgewiesene Verschärfung. Viertens beweisen die Lean-Theoreme Aussagen über Modelle der Wire-Regeln, nicht über das laufende Programm; die Brücke bleibt Prüfsache der Tests (Abschnitt 3.4.1), weshalb diese Arbeit keine formale Konformitätsaussage erhebt.

Fünf **Abweichungen** traten im Prüf- und Messbetrieb auf. Erstens beendete sich der Raw-Empfänger auf öffentlichen Hosts still beim ersten fremden UDP-Paket; Ursache war ein als Ende gewerteter Filterausgang, und die Korrektur besteht aus dem Weiterlesen nach gefiltertem Rauschen und einer Gesamtfrist. Die externe Kampagne vom 10.08. und die bidirektionale Kampagne vom 11.08. nutzten noch einen Stand vor beiden Korrekturen (Tabelle 6.1). In der bidirektionalen Kampagne hielt eine äußere Neustartschleife den Empfänger verfügbar; die Ankunftszeiten beruhen deshalb auf den PCAP-Dateien, während die lückenhaften JSONL-Dateien dort nur die Decodierung belegen. Die vollständigen P0-, P1- und P2-Kampagnen ab dem 14.08. liefen mit beiden Korrekturen. Zweitens überstand der Um-eins-Fehler der Surplus-Ortung 23 Einzelfalltests und fiel erst der Zweitprüfung im Pull-Request auf (Abschnitt 6.4); als Korrektur wurden Property-Tests und Fuzz-Ziele je Parse-

fläche verpflichtend. Drittens fand der RFC-Abgleich des Abschlussschritts trotz grüner Prüfkette weitere Konformitätslücken, darunter die Behandlung unbekannter UNSAFE-Optionen vor einem FRAG; die Korrektur erfolgte gesammelt und real nachgeprüft. Viertens verfehlte ein AWS-Lauf des Szenarios S-50 das vorregistrierte Zeitkriterium, obwohl der Kampagnentreiber Status 0 meldete (Abschnitt 6.5). Fünftens wich der Messbetrieb von der Hotspot-Vorregistrierung ab: verlangt war $DF=1$, gesendet wurde in allen drei Hotspot-Reihen durchgehend $DF=0$ (Abschnitt 6.2.2); da Kontrollen und Proben je Lauf dieselbe DF-Lage hatten, blieb die Diskriminatorfrage auswertbar, die Abweichung ist dennoch offenzulegen. Zusammen zeigen die fünf Fälle, dass eine grüne Prüfkette weder Konformität noch die vollständige Erfüllung aller Zellkriterien garantiert.

Damit kehrt die Bewertung zu der in Kapitel 2 offen gelassenen Frage zurück, was aus dem Verlust einzelner Fragmente wird, den auch FRAG nicht behebt. Die Messungen beantworten sie in vier Schritten. Erstens lässt ein fehlendes Fragment seinen Satz liegen, ohne Nachbarn zu stören: In der langen Serie aus 300 Zwei-Fragment-Sätzen mit gezielt zurückgehaltenem Terminalfragment kamen exakt die geplanten 270 Sätze an, die 30 beschädigten blieben ohne Zustellung, und die Quote blieb über die Serie drittelgenau konstant (570 von 570 Fragmenten auf dem Draht, keine Drift). Zweitens wirkt der Reassemblierungs-Timeout real, ist aber nur über das belegte Ausbleiben messbar. Drittens muss die Identification zusammen mit dem UDP-Vierertupel aus Quell- und Zieladresse sowie Quell- und Zielport über das erwartete Reassemblierungsfenster eindeutig sein [2, Sec. 11.4]. Der Versuch mit derselben Kennung im selben Vierertupel und innerhalb dieses Fensters verletzt diese markierte Senderpflicht absichtlich: Bei komplementären, nicht überlappenden Fragmenten setzte der Empfänger ein nie gesendetes Datagramm zusammen und lieferte es kommentarlos aus. Das ist keine Abweichung der Implementierung vom RFC, sondern eine Robustheitsgrenze gegenüber einem nichtkonformen Sender. Der RFC schreibt dem Empfänger keine zusätzliche Erkennung solcher Kollisionen vor. Viertens bietet die APC dafür eine optionale protokollinterne Erkennung beschädigter Nutzdaten; schlägt sie fehl, wird der Fehlschlag gemeldet, und die Nutzdaten werden standardmäßig dennoch zugestellt.

Fragmentierung und Soft-State lassen sich in vier getrennte Grenzen zerlegen, die Tabelle 6.3 Norm, Umsetzung und Beobachtung direkt zuordnet.

Da der RFC weder die Kapazität noch das Überlaufverfahren festlegt (Tabelle 6.3; [2, Sec. 11.4, 25.4]), folgen die 16 nicht zugestellten Sätze je S-50-Lauf der gewählten Abweisungspolitik: Bei vollem Cache wird ein Fragment, das einen neuen unvollständigen Satz anlegen würde, nicht aufgenommen; ein atomares, sofort vollständiges FRAG wird ohne neuen Cacheeintrag verarbeitet. Nach Freigaben können

Tabelle 6.3: Fragmentierung und Soft-State: Norm, Umsetzung und Messbeleg

Grenze	RFC 9868	Implementierung	Beobachtung
Menge und Schutz	Speicher für die Reassemblierung sollte begrenzt werden; kein Zahlenwert (Sec. 11.4, 25.4)	64 unvollständige Sätze je Cache; bereits gespeicherte Sätze bleiben erhalten	S-50: je Lauf 64 von 80 Sätzen, 16 ohne Zustellung; 72 von 72 Läufen (12 je Hostpaar) wie vorhergesagt; Kontrollen und atomares Fragment zugestellt
Zeit und Bereinigung	Standardtimeout sollte höchstens zwei Minuten betragen; sein Ablauf darf kein ICMP erzeugen (Sec. 11.4)	Grenze bei 120 Sekunden; Ablauf wird bei Einfügung oder durch <code>gc</code> wirksam; kein Hintergrundlauf	S-50 nutzte 60 Sekunden; in einem Lauf lagen 32 Terminale außerhalb von 30 Sekunden und wurden noch zugestellt
Trennung	Grenzen sollten nicht als gemeinsame Ressource über mehrere Sockets berechnet werden (Sec. 11.4)	ein Cache je <code>Peer</code> ; der Raw-Empfänger filtert nur Ports, nicht Adressen. Daher sind Fremdzustellung und ein geteiltes Budget möglich; der <code>FragKey</code> verhindert nur Fragmentmischung	Teilumsetzung der Matrixzeile 199; keine Messung konkurrierender Adresspaare
Auslieferung	einzelne Fragmente dürfen nie ausgeliefert werden; ein Reassemblierungsfehler sollte kein Leerdatagramm erzeugen (Sec. 11.4)	<code>Buffered</code> oder <code>Dropped</code> ; Berichte erst nach vollständiger Reassemblierung	S-49: 270 von 300 Sätzen geliefert, 30 unvollständige Sätze ohne Nutzerrahmen

spätere Terminalfragmente neue unvollständige Zustände anlegen; zugestellt wird keiner der 16 Sätze. Anwendungen müssen Ausbleiben, Kennungskollisionen und Cacheüberlauf deshalb selbst behandeln; APC und Optionsberichte ersetzen keine Ende-zu-Ende-Behandlung.

Für dieses Ergebnis gelten zwei Grenzen: saubere, kalibrierte Pfade ohne gemischte Störprüfung (Abschnitt 6.1) und die nur pfadneutral messbaren Empfängerpflichten für verfälschte Pads und Prüfsummen (Abschnitt 6.2.2). Damit ist der derzeitige Zuordnungsstand für FF1 zusammengefasst; die Einzelwerte enthält die in Abschnitt 4.2 verlinkte Einzelzuordnung. Wie diese Zuordnung unter durchgehender Beteiligung von Sprachmodellen zustande kam, beschreibt der folgende Abschnitt.

6.4 Reflexion der KI-gestützten Arbeitsweise

Dieser Abschnitt ergänzt die Offenlegung aus Kapitel 3 um beschreibende Kennzahlen der Zusammenarbeit. Er ist rein deskriptiv: Er berichtet belegte Fälle mit ihren Ausgängen und erhebt keine Wirksamkeitsaussage, denn eine Prüfung ohne Sprachmodelle als Vergleichsgruppe existiert nicht. Die Rollen sind die aus Kapitel 3: das umsetzende Werkzeug für Entwürfe und Quelltext, der getrennte Zweit-

prüfer mit dem Auftrag, Fehler und Gegenbeispiele zu suchen, und der Autor, der entscheidet und verantwortet. Grundlage ist ein Befundregister aus zwölf Fällen (`thesis/daten/ki-befundregister.csv` im Repository dieser Arbeit); es ist eine geprüfte Auswahl aus 27 Pull-Requests sowie weiteren Messläufen, kein Vollbestand. Vier Fallgruppen tragen das Bild:

- **Übernommene Funde vor dem Merge.** Im Wire-Modell-Review wurden zwei Funde übernommen, beim Aufbau des Lean-Gates drei; die Einzelheiten führt das Register. Ein Einzelfund gelang allein dem Zweitprüfer: der durch ein Byte-Palindrom maskierte Ausrichtungsfehler im unabhängigen Prüfprogramm der Wire-Prüfung (Abschnitt 3.4.1); ebenso fing allein die Zweitprüfung den Um-eins-Fehler der Surplus-Ortung (Abschnitt 6.3).
- **Abgelehnte Befunde.** Nicht jeder Befund wurde übernommen: Im Lean-Gate-Review lehnte der Autor einen Vorschlag mit dokumentierter Begründung ab (der bemängelte Bestand lag außerhalb des Änderungsumfangs), und im TLV-Parser-Review blieben Prüferbefunde insgesamt ohne Übernahme, weil sie den beabsichtigten Umfang betrafen und keinen Faktenirrtum zeigten. Die Ablehnungen sind in den Verläufen nachlesbar; sie gehören zum Bild derselben Arbeitsweise.
- **Prüfung nach dem Merge.** Die Reihenfolge hielt nicht immer: Ein Pull Request wurde vor der Zweitprüfer-Rückmeldung zusammengeführt. Die vom Zweitprüfer beanstandete Gesamtfrist des Empfangspfads folgte drei Tage später. Auch der Schlussaudit fand trotz grüner Prüfkette weitere Konformitätslücken; Schritt 18 behob sie gesammelt (Abschnitt 6.3).
- **Gemeinsam übersehene Fehler.** Der Zählfehler der bidirektionalen Kampagne gab für den Hinweg zunächst 15 statt korrekt 18 optionsfreie Kontrollpakete an (Ergebnis 18/18). Die vorab beauftragte Prüfung erfasste diese Zahl nicht, weil keiner der vorgelegten Prüfpunkte sie betraf; eine spätere, gesonderte Codex-Prüfung fand den Fehler, der als Erratum im versiegelten Archiv berichtigt wurde. Ein Faktencheck dieses Textes widerlegte zudem eine eigene Kapitelaussage am Quelltext (der nur teilweise eingelöste Cachevertrag aus Abschnitt 6.3), und die Nachprüfung der Grundlagen führte zu zwei ausdrücklich als lokale Politik dokumentierten Entscheidungen samt einer geschlossenen Testlücke. Außerhalb des Quelltexts ließ ein Abrufwerkzeug bei der Prüfung der Referenzfälle aus Abschnitt 3.1 den entscheidenden Satz eines Abstracts aus, sodass die Erstbewertung falsch war und nach einem Autorenhinweis berichtigt wurde.

Die Kennzahlen des Registers: Zehn der zwölf Fälle endeten mit Übernahme, einer teils mit Übernahme und teils mit begründeter Ablehnung, einer vollständig mit Ablehnung; sechs Fälle lagen vor dem Merge, fünf danach, einer außerhalb

des Repositorys. Zwei Funde führt das Register allein beim Zweitprüfer, ein dritter stammt aus einem Zweitprüferlauf des Faktenchecks; die Fundserie eines Falls führt es allein beim umsetzenden Werkzeug. Das Register kennzeichnet nur den Kampagnen-Zählfehler ausdrücklich als zunächst von allen drei Beteiligten übersehen. Für die frühen Projektphasen fehlen vollständige Modell- und Konfigurationsangaben; diese Fälle tragen den Vermerk der lückenhaften Provenienz und werden nicht nachträglich vervollständigt (Kapitel 3). Die KI-Prüfungen waren angefordert, nicht technisch erzwungen; Sammelcommits verschmelzen die innere Reihenfolge eines Schritts.

Das Schlussbild ist eine gemischte Bilanz: Die Arbeitsweise erzeugte nachweisbare Korrekturen bis hinein in Prüfprogramm und Kapiteltext, sie produzierte Fehlbefunde, die der Autor mit Begründung ablehnte, und sie übersah Fehler, die erst eine andere Ebene fand. Mehr als diese Beschreibung gibt der Aufbau nicht her, und mehr behauptet diese Arbeit nicht.

6.5 Diskussion und Grenzen der Aussagekraft

Technische Umsetzbarkeit am Endpunkt und Durchlässigkeit des Pfads sind getrennte Eigenschaften (Abschnitt 6.3): Normalisierer, Prüfsummen-Gates und Präsenz-Verwerfer entlang des konkreten Pfads entscheiden, ob eine Surplus Area ankommt, und Erreichbarkeit ist deshalb paarbezogen, also eine Eigenschaft aus Quelle und Ziel und nicht des Ziels allein. Die drei Mechanismenklassen lassen sich an einer Messstudie einordnen, die RFC 9868 in den Abschnitten 9 und 25.6 zitiert [43]: Zullo, Jones und Fairhurst fanden 2018 von einem europäischen Rechenzentrum aus auf Pfaden zu STUN-, DNS- und HTTP-Zielen als häufigste Störung eine UDP-Prüfsummenprüfung über die gesamte IP-Nutzlast; auf mehr als 80 Prozent der von Optionsdatagrammen durchquerten Pfade beobachteten sie mindestens eine Prüfung nach einem falschen Schema. Dagegen schlugen sie eine Kompensationsoption vor, die nach Angabe der Autoren als umgestaltete OCS in die Spezifikation einging; die OCS nennt dieselbe Neutralisierung als Entwurfsziel [2, Sec. 9]. Das Prüfsummen-Gate dieser Arbeit ist genau diese Pathologie, in den Randintervallen zweier Endpunktnetze repliziert. Die dort als Prüfung der Längenkonsistenz mit Verwerfen beschriebene Störung entspricht dem Präsenz-Verwerfer, der hier zusätzlich als quell- und zielabhängige Paareigenschaft auftritt. Ein Entfernen der Surplus Area fand die Vorarbeit dagegen nicht, obwohl Abschnitt 25.6 des RFC für dieselbe Quelle ein Entfernen als wahrscheinliche Erklärung nennt [2, Sec. 25.6]; der Normalisierer geht damit über die Vorarbeit hinaus, wurde hier aber nur an einer VMware-NAT-Grenze beobachtet. Die Gates stützen zugleich den Entwurf der OCS: Ihre Prüfsummenneutralität lässt regelkonforme Datagramme passieren (Abschnitt 6.2.2); gegen einen Präsenz-Verwerfer

hilft sie nicht.

Für die Betroffenen folgt daraus zweierlei. Anwendungen müssen auf vergleichbaren Pfaden mit klassenbasiertem Totalverlust und mit stiller Inhaltsentfernung rechnen, denn ein empfangenes leeres Datagramm zeigt nicht an, dass sein Inhalt unterwegs entfernt wurde; Ende-zu-Ende-Bestätigung, Empfangsberichte und im Zweifel eine Kapselung, die die Klasse vor dem Pfad verbirgt, mindern das Risiko. Wer Netze betreibt, sollte beide Richtungen getrennt prüfen, weil eine Adressumsetzung einen dahinterliegenden Verwerfer maskieren kann. Beide Folgerungen ziehen die Messbefunde dieser Arbeit; Betreiberpflichten von RFC 9868 sind sie nicht.

Diesen Aussagen stehen klare Grenzen gegenüber. Jeder Pfadbefund gilt für das beobachtete Paar, die Richtung und das Messfenster; eine Aussage über „das Internet“ trifft diese Arbeit nicht, Formulierungen wie Hetzner-Kante oder NAT-Vermittler sind sparsame Modelle und keine Gerätenachweise, und die Messgrenzen aus Abschnitt 6.1 (ein einziges Zugangsnetz, kalibrierte Pfade, einzeln statt gemischt geprüfte Störmechanismen) gelten unverändert. Auf der Prüfseite beweisen die Lean-Theoreme Modelle statt des Programms, findet Fuzzing Fehler statt Fehlerfreiheit, und die Folge der Modellausgaben früher Projektphasen ist nur lückenhaft überliefert; eine Wirksamkeitsaussage zur KI-gestützten Arbeitsweise folgt daraus nicht (Abschnitt 6.4).

Auf Implementierungsseite bleiben die sieben Teilumsetzungen aus Abschnitt 6.3, darunter die Paarbindung des Reassemblierungs-Caches, die der Typ `Peer` nur teilweise einlöst, und die Parameter, die für ignorierte oder fehlgeschlagene Optionen nicht bereitgestellt werden. Hinzu treten zwei Lücken der Prüfkette. Erstens überführen die Auswerter für P0, P1 und P2 semantische Abweichungen nicht in einen Fehlerstatus: P1 und P2 sammeln solche Befunde, beenden den Lauf aber mit Status 0, und P0 gibt negative Prüfergebnisse aus, ohne den Rückgabewert zu ändern; das zeigte sich unter anderem bei S-14, S-36 und S-50. Zweitens prüfen die Auswerter semantische Felder höchstens am ersten Datensatz je Klasse, und nicht alle werten Senderfehler aus. Bei gültigen Eingaben belegt ein Status 0 nur den Abschluss des Auswerterlaufs, nicht die Erfüllung aller Zellkriterien. Damit ist die Darstellung der Ergebnisse und ihrer Grenzen abgeschlossen; die wörtliche Beantwortung beider Forschungsfragen und die Empfehlungen übernimmt Kapitel 7.

7. Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

FF1 fragt, welche Anforderungen von RFC 9868 sich unter Linux und IPv4 mit einer Raw-Socket-Implementierung im Benutzerraum vollständig, teilweise oder nicht erfüllen lassen. Die Zuordnung im geprüften Umfang steht in Tabelle 7.1. Die sieben Teilumsetzungen sind Implementierungs- und Entwurfsgrenzen, keine nachgewiesenen Plattformgrenzen. Der Index selbst mischt markierte BCP-14-Anforderungen, unmarkierte normative API-Formen, Beschreibungen, Leitlinien und zusammengesetzte Aussagen; er ist ein Arbeitsindex und kein Verzeichnis aller Pflichten des Standards. Das Ergebnis beantwortet FF1 deshalb auf genau diesem erklärten Prüfraster und beweist weder die Satzgenauigkeit der Zerlegung noch vollständige Konformität mit allen Aussagen von RFC 9868.

FF2 fragt, in welchem Umfang die Surplus Area auf den untersuchten realen Netzpfeilen bis zur Gegenstelle unverändert erhalten bleibt. Im europäischen Messdreieck und auf den drei transatlantischen AWS-Hostpaaren blieb sie in den jeweiligen Messfenstern erhalten. An der Grenze aus VMware-NAT und VMnet wirkte ein **Normalisierer** und entfernte sie. Im gebrochene Mobilfunk-Zugangspfad und, auf Pilotniveau, im Google-seitigen Messintervall wirkte ein **Präsenz-Verwerfer** und verwarf die gesamte Klasse in beiden Richtungen; auf einzelnen Hinwegen zu AWS-Regionen in Asien-Pazifik trat derselbe Totalverlust quell- und zielabhängig im Intervall zwischen den eigenen Messpunkten auf. Die beobachteten **Prüfsummen-Gates** verwarfen Datagramme, deren Einerkomplementsumme über Pseudo-Header und gesamte IP-Nutzlast nicht aufging; regelkonforme Datagramme mit Null-Pad und gültiger OCS passierten sie. Jede dieser Aussagen gilt nur für das beobachtete Paar, die Richtung und das Messfenster.

Die Leitfrage, wie sich UDP um Transportoptionen erweitern lässt, ohne seine grundlegenden Eigenschaften aufzugeben, ist zweigeteilt zu beantworten. Auf der Entwurfsseite bleibt der UDP-Header unverändert, die Optionen liegen in der Surplus Area, und Nutzdaten werden auch bei fehlgeschlagener SAFE-Option zugestellt; Nachrichtenorientierung, Verbindungslosigkeit und der Verzicht auf Sequenznummern, Fenster und Zeitgeber bleiben erhalten (Abschnitt 2.2.2). Die einzige belegte Ausnahme ist die Reassemblierung von FRAG-Datagrammen: Sie legt am Empfänger einen zeitlich und mengenmäßig begrenzten Zustand an, dessen Verhalten an Mengen-

Tabelle 7.1: Synthese der Forschungsfragen im geprüften Umfang

FF	Ergebnis	Beobachtete Grenze	Geltungsbereich
FF1	57 technisch vollständig erfüllbar; 0 teilweise; 0 nicht erfüllbar	Referenzimplementierung: 50 vollständig, 7 teilweise; 10 nicht anwendbar oder Leitlinie	67-zeiliger Arbeitsindex; Linux, IPv4, Benutzerraum, Raw Sockets
FF2	Surplus Area je nach Pfad erhalten, entfernt oder als Klasse verworfen	Normalisierer, Prüfsummen-Gates und Präsenz-Verwerfer	beobachtetes Paar, Richtung und Messfenster

grenze und Zeitablauf Tabelle 6.3 zeigt. Auf der Pfadseite zeigen die Messungen, dass die Abwärtskompatibilität des Wire-Formats keine allgemeine Zustellbarkeit garantiert: Technische Umsetzbarkeit am Endpunkt und Durchlässigkeit des Pfads sind getrennte Eigenschaften.

7.2 Ausblick

Für eine Weiterentwicklung der Bibliothek liegen die sieben Teilumsetzungen aus Abschnitt 6.3 nahe; die hohe Sende-API könnte zusätzlich die Leitlinie zur paketweisen Fragmentierungssteuerung abbilden. Eine Prüfkette, die semantische Abweichungen ihrer Auswerter in einen Fehlerstatus überführt, würde den Rückgabewert wieder aussagekräftig machen.

Für künftige Untersuchungen bleiben die Grenzen dieser Arbeit die Ansatzpunkte: eine unabhängige Implementierung, weitere Zugangsnetze, IPv6, andere Messzeitpunkte und das Zusammenspiel mit RFC 9869 [3] könnten Interoperabilität und zeitliche Stabilität der Befunde einordnen.

7.3 Schlusswort

Die Arbeit belegt, dass die Umsetzung der UDP-Optionsmechanik im Benutzerraum und ihre Zustellbarkeit auf realen Pfaden getrennte Fragen sind. Eine funktionierende Referenzimplementierung beseitigt die pfadabhängigen Normalisierer und Verwerfer nicht. Die getrennte Bewertung verhindert, dass ein grüner Implementierungstest als Netzbeleg oder eine erfolgreiche Pfadmessung als Konformitätsbeleg missverstanden wird.

Literaturverzeichnis

- [1] J. Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force, Aug. 1980. DOI: [10.17487/RFC0768](https://doi.org/10.17487/RFC0768).
- [2] J. Touch und C. Heard. *Transport Options for UDP*. RFC 9868. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9868](https://doi.org/10.17487/RFC9868).
- [3] G. Fairhurst und T. Jones. *Datagram Packetization Layer Path MTU Discovery (DPLPMTUD) for UDP Options*. RFC 9869. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9869](https://doi.org/10.17487/RFC9869).
- [4] A. Stephan und L. Wüstrich. „The Path of a Packet Through the Linux Kernel“. In: *Seminar Innovative Internet Technologies and Mobile Communications (IITM), WS 23*. Technische Universität München, Chair of Network Architectures und Services, 2024, S. 89–93. DOI: [10.2313/NET-2024-04-1_16](https://doi.org/10.2313/NET-2024-04-1_16).
- [5] J. Postel. *Internet Protocol*. RFC 791. Internet Engineering Task Force, Sep. 1981. DOI: [10.17487/RFC0791](https://doi.org/10.17487/RFC0791).
- [6] F. Gont, R. Atkinson und C. Pignataro. *Recommendations on Filtering of IPv4 Packets Containing IPv4 Options*. RFC 7126. Internet Engineering Task Force, Feb. 2014. DOI: [10.17487/RFC7126](https://doi.org/10.17487/RFC7126).
- [7] L. Eggert, G. Fairhurst und G. Shepherd. *UDP Usage Guidelines*. RFC 8085. Internet Engineering Task Force, März 2017. DOI: [10.17487/RFC8085](https://doi.org/10.17487/RFC8085).
- [8] W. Eddy. *Transmission Control Protocol (TCP)*. RFC 9293. Internet Engineering Task Force, Aug. 2022. DOI: [10.17487/RFC9293](https://doi.org/10.17487/RFC9293).
- [9] V. Cerf, Y. Dalal und C. Sunshine. *Specification of Internet Transmission Control Program*. RFC 675. Internet Engineering Task Force, Dez. 1974. DOI: [10.17487/RFC0675](https://doi.org/10.17487/RFC0675).
- [10] J. F. Kurose und K. W. Ross. *Computer Networking: A Top-Down Approach*. 8. Aufl. Pearson, 2021.
- [11] *Requirements for Internet Hosts - Communication Layers*. RFC 1122. Internet Engineering Task Force, Okt. 1989. DOI: [10.17487/RFC1122](https://doi.org/10.17487/RFC1122).
- [12] R. Braden, D. Borman und C. Partridge. *Computing the Internet Checksum*. RFC 1071. Internet Engineering Task Force, Sep. 1988. DOI: [10.17487/RFC1071](https://doi.org/10.17487/RFC1071).
- [13] RFC Editor. *RFC Erratum 8834 for RFC 9868*. Technical; Verified; gemeldet am 17.03.2026. RFC Editor. 23. März 2026. URL: <https://www.rfc-editor.org/errata/eid8834> (besucht am 26.08.2026).

- [14] Linux Kernel Community. *Linux 5.10: net/ipv4/udp.c*. Version 5.10. 13. Dez. 2020. URL: <https://elixir.bootlin.com/linux/v5.10/source/net/ipv4/udp.c> (besucht am 19.08.2026).
- [15] Linux man-pages project. *raw(7): IPv4 raw sockets*. Version 6.18. 8. Feb. 2026. URL: <https://git.kernel.org/pub/scm/docs/man-pages/man-pages.git/plain/man/man7/raw.7?h=man-pages-6.18> (besucht am 17.08.2026).
- [16] T. Hubert, R. Mehta, L. Sartran u. a. „Olympiad-level formal mathematical reasoning with reinforcement learning“. In: *Nature* 651.8106 (2026). Online veröffentlicht am 12.11.2025, S. 607–613. DOI: [10.1038/s41586-025-09833-y](https://doi.org/10.1038/s41586-025-09833-y).
- [17] M. D. Schwartz. *Resummation of the C-Parameter Sudakov Shoulder Using Effective Field Theory*. arXiv:2601.02484, eingereicht am 05.01.2026. 2026. URL: <https://arxiv.org/abs/2601.02484> (besucht am 13.08.2026).
- [18] J. Sumner. *Rewriting Bun in Rust*. Oven (Bun). 8. Juli 2026. URL: <https://bun.com/blog/bun-in-rust> (besucht am 13.09.2026).
- [19] *Rewrite Bun in Rust*. oven-sh/bun, Pull Request 30412. Gemergt am 14.05.2026. GitHub. 2026. URL: <https://github.com/oven-sh/bun/pull/30412> (besucht am 13.08.2026).
- [20] *PathString::slice dangling reference UB - add Miri to CI (oven-sh/bun, Issue 30719)*. Gemeldet am 14.05.2026, nach dem Merge des Rust-Ports. GitHub. 2026. URL: <https://github.com/oven-sh/bun/issues/30719> (besucht am 13.08.2026).
- [21] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Internet Engineering Task Force, März 1997. DOI: [10.17487/RFC2119](https://doi.org/10.17487/RFC2119).
- [22] B. Leiba. *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174. Internet Engineering Task Force, Mai 2017. DOI: [10.17487/RFC8174](https://doi.org/10.17487/RFC8174).
- [23] RFC Editor. *RFC Erratum 8708 for RFC 9868*. Editorial; Held for Document Update; gemeldet am 17.01.2026. RFC Editor. 20. Jan. 2026. URL: <https://www.rfc-editor.org/errata/eid8708> (besucht am 26.08.2026).
- [24] A. T. Kalai, O. Nachum, S. S. Vempala und E. Zhang. *Why Language Models Hallucinate*. arXiv:2509.04664. 2025. URL: <https://arxiv.org/abs/2509.04664> (besucht am 13.08.2026).
- [25] E. Perez u. a. *Discovering Language Model Behaviors with Model-Written Evaluations*. arXiv:2212.09251. 2022. URL: <https://arxiv.org/abs/2212.09251> (besucht am 13.08.2026).
- [26] M. Cheng, C. Lee, P. Khadpe u. a. „Sycophantic AI decreases prosocial intentions and promotes dependence“. In: *Science* 391.6792 (2026). Artikelkennung eaec8352. DOI: [10.1126/science.aec8352](https://doi.org/10.1126/science.aec8352).

- [27] proptest-rs-Projekt. *Proptest: Hypothesis-like property testing for Rust*. GitHub. 2026. URL: <https://github.com/proptest-rs/proptest> (besucht am 23.08.2026).
- [28] rust-fuzz-Projekt. *cargo-fuzz: Command line helpers for fuzzing*. GitHub. 2026. URL: <https://github.com/rust-fuzz/cargo-fuzz> (besucht am 23.08.2026).
- [29] LLVM Project. *libFuzzer: A Library for Coverage-Guided Fuzz Testing*. Undatierte, fortlaufend gepflegte Dokumentation. LLVM Project. URL: <https://llvm.org/docs/LibFuzzer.html> (besucht am 20.08.2026).
- [30] L. de Moura und S. Ullrich. „The Lean 4 Theorem Prover and Programming Language“. In: *Automated Deduction (CADE 28)*. Bd. 12699. Lecture Notes in Computer Science. Springer, 2021, S. 625–635. DOI: [10.1007/978-3-030-79876-5_37](https://doi.org/10.1007/978-3-030-79876-5_37).
- [31] G. Thomas. *A proactive approach to more secure code*. Microsoft Security Response Center. 16. Juli 2019. URL: <https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code/> (besucht am 13.09.2026).
- [32] The Chromium Project. *Memory safety*. Undatierte, fortlaufend gepflegte Projektseite. The Chromium Project. URL: <https://www.chromium.org/Home/chromium-security/memory-safety/> (besucht am 31.05.2026).
- [33] National Security Agency. *Software Memory Safety*. Cybersecurity Information Sheet U/OO/219936-22; PP-23-0782. Version 1.1. Unter der Adresse der Erstveröffentlichung vom 10. November 2022. National Security Agency (NSA), Apr. 2023. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF (besucht am 13.09.2026).
- [34] ISO/IEC JTC 1/SC 22/WG14. *Programming languages: C*. N2310. C2x-Arbeitsentwurf mit Änderungsmarkierungen gegenüber ISO/IEC 9899:2018. ISO/IEC JTC 1/SC 22/WG14. 6. Nov. 2018. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (besucht am 13.09.2026).
- [35] B. Stroustrup und H. Sutter. *C++ Core Guidelines*. Fortlaufend gepflegte Online-Fassung, insbesondere Regel R.1 zur Ressourcenverwaltung. 14. Juni 2026. URL: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines> (besucht am 13.09.2026).
- [36] Zig Software Foundation. *Zig Language Reference*. Version 0.16.0. Zig Software Foundation. 2026. URL: <https://ziglang.org/documentation/0.16.0/> (besucht am 24.08.2026).
- [37] S. Klabnik, C. Nichols und C. Kryocho. *The Rust Programming Language*. Undatierte, fortlaufend gepflegte Online-Fassung. URL: <https://doc.rust-lang.org/book/> (besucht am 16.02.2026).

- [38] The Rust Project. *Rust Programming Language*. Undatierte, fortlaufend gepflegte Projektseite. URL: <https://www.rust-lang.org/> (besucht am 16.02.2026).
- [39] Linux man-pages project. *packet(7): packet interface on device level*. Version 6.18. 8. Feb. 2026. URL: <https://git.kernel.org/pub/scm/docs/man-pages/man-pages.git/plain/man/man7/packet.7?h=man-pages-6.18> (besucht am 26.08.2026).
- [40] Linux Kernel Community. *Linux 5.10: Universal TUN/TAP device driver*. Version 5.10. 13. Dez. 2020. URL: <https://www.kernel.org/doc/html/v5.10/networking/tuntap.html> (besucht am 26.08.2026).
- [41] Linux Kernel Community. *Linux 5.10: AF_XDP*. Version 5.10. 13. Dez. 2020. URL: https://www.kernel.org/doc/html/v5.10/networking/af_xdp.html (besucht am 26.08.2026).
- [42] Linux man-pages project. *capabilities(7): overview of Linux capabilities*. Version 6.18. 8. Feb. 2026. URL: <https://git.kernel.org/pub/scm/docs/man-pages/man-pages.git/plain/man/man7/capabilities.7?h=man-pages-6.18> (besucht am 23.08.2026).
- [43] R. Zullo, T. Jones und G. Fairhurst. „Overcoming the Sorrows of the Young UDP Options“. In: *Proceedings of the 4th Network Traffic Measurement and Analysis Conference (TMA 2020)*. In RFC 9868 als [Zu20] zitiert. IFIP, 2020. URL: <https://dl.ifip.org/db/conf/tma/tma2020/tma2020-camera-paper70.pdf> (besucht am 02.09.2026).

Erklärung

Ich erkläre, dass ich die Masterarbeit selbstständig und ohne unzulässige Inanspruchnahme Dritter verfasst habe. Ich habe dabei nur die angegebenen Quellen und Hilfsmittel verwendet und die aus diesen wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht. Die Versicherung selbstständiger Arbeit gilt auch für enthaltene Zeichnungen, Skizzen oder graphische Darstellungen. Die Arbeit wurde bisher in gleicher oder ähnlicher Form weder derselben noch einer anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

.....

Ort, Datum

.....

Unterschrift